



國立臺北科技大學

資訊與財金管理系碩士班

碩士學位論文

**多代理 RESTful API 自動化測試框架：語
義生成、工作流程恢復與序列決策策略之
設計與評估**

**A Multi-Agent Framework for Automated RESTful
API Testing: Design and Evaluation of Semantic
Generation, Workflow Recovery, and Sequential
Decision-Making Strategies**

研究生：鄭亦恩

指導教授：彭祖乙博士

中華民國一百一十五年六月



國立臺北科技大學

資訊與財金管理系碩士班

碩士學位論文

**多代理 RESTful API 自動化測試框架：語
義生成、工作流程恢復與序列決策策略之
設計與評估**

**A Multi-Agent Framework for Automated RESTful
API Testing: Design and Evaluation of Semantic
Generation, Workflow Recovery, and Sequential
Decision-Making Strategies**

研究生：鄭亦恩

指導教授：彭祖乙博士

中華民國一百一十五年六月

摘要

關鍵詞：大型語言模型、多代理系統、自動化軟體測試、RESTful API、強化學習、模型上下文協定 (MCP)、代理人對代理人協定 (A2A)、工作流程決策、軟體品質保證

隨著軟體開發邁向敏捷與微服務架構，RESTful API 已成為系統整合與服務交換之核心介面。然而，現有自動化測試方法在面對多步驟操作依賴、語意限制條件與真實執行環境變動時，仍常出現語意理解不足、跨操作依賴推理困難，以及測試流程缺乏恢復能力等問題，導致無效請求比例偏高，並限制整體測試可完成度。為因應上述問題，本研究提出一套以後執行工作流程恢復層 (Post-Execution Workflow Recovery Layer) 為核心之多代理 RESTful API 自動化測試框架。此框架以大型語言模型 (Large Language Model, LLM) 為語意推理核心，整合 AutoGen 之代理協作能力、LangGraph 之工作流程控制機制，以及模型上下文協定 (Model Context Protocol, MCP) 與代理人對代理人協定 (Agent-to-Agent Protocol, A2A)，建立由 OpenAPI 規格解析、測試案例生成、測試程式產生、API 執行、工作流程決策到結果分析之完整測試流程。本研究之重點不在於單純產生測試請求，而在於處理 API 執行後之工作流程決策問題。為此，本文將執行後狀態轉移形式化為馬可夫決策過程 (Markov Decision Process, MDP)，使系統能根據 HTTP 狀態碼、錯誤類型、依賴狀態、授權狀態與資源可用性等訊號，選擇後續工作流程行動，例如重試、更新授權、資源解析、重新產生測試資料或進入結果回報，以提升測試流程之持續執行能力與恢復能力。其中，工作流程恢復層負責在 404、資源缺失與依賴失敗等情境下提供可持續執行之恢復機制；工作流程決策策略則負責在候選恢復行動中選擇下一步動作。在實證評估方面，本研究以 TMDB 資料集之 100 筆 manual-doc-grounded 任務作為主實驗基準，比較 v1 至 v5 不同版本之測試表現。結果顯示，v1 至 v5 之通過率分別為 0.25、0.72、0.69、0.69 與 0.79。其中，v5 之 Strict Pass Rate 仍為 0.69，與 v3 及 v4 相同，但其 Final Pass Rate 提升至 0.79，且新增 10 筆 Workflow Recovered Pass，顯示其主要效益來自執行後工作流程恢復，而非前段測試生成能力之提升。進一步分析顯示，目前恢復增益主要集中於 404 資源依賴缺失情境。在離線強化學習方面，本研究利用工作流程轉移事件建立離線資料集，並以表格式擬

合 Q 值迭代訓練 Offline-Q Policy。此部分之主要目的，在於驗證工作流程決策問題是否可被形式化為 learning-based workflow decision strategy 之可行性探索。結果顯示，學習型策略已能掌握部分工作流程決策模式，並於特定恢復情境中提供額外效益；然而，受限於單一 API 領域、Live API 環境波動與離線資料規模有限，其整體表現尚不足以宣稱全面超越 rule-based baseline。整體而言，本研究驗證了大型語言模型、多代理協作與工作流程決策策略整合於 RESTful API 自動化測試之可行性，並為後續智慧化 API 測試系統之設計與評估提供具體參考。



Abstract

Keyword: Large Language Models, Multi-Agent Systems, RESTful API, Automated Software Testing, Reinforcement Learning, Model Context Protocol, Agent-to-Agent Protocol, AI Agents, Workflow Orchestration, Software Quality Assurance

As software development shifts toward agile methodologies and microservice architectures, RESTful APIs have become the core interface for system integration and service exchange. However, existing automated testing approaches continue to struggle with multi-step operation dependencies, semantic constraints, and dynamic real-world execution environments, resulting in insufficient semantic understanding, difficulties in cross-operation dependency reasoning, and a lack of recovery capability in testing workflows. These limitations lead to a high proportion of invalid requests and constrain overall test completion rates.

To address these challenges, this study proposes a multi-agent RESTful API automated testing framework centered on a Post-Execution Workflow Recovery Layer. The framework employs Large Language Models (LLMs) as the semantic reasoning core, integrating AutoGen for multi-agent collaboration, LangGraph for workflow control, Model Context Protocol (MCP), and Agent-to-Agent Protocol (A2A) to establish a complete testing pipeline spanning OpenAPI specification parsing, test case generation, test code materialization, API execution, workflow decision-making, and result analysis.

The primary focus of this study is not merely generating test requests, but addressing the workflow decision problem that arises after API execution. To this end, post-execution state transitions are formalized as a Markov Decision Process (MDP), enabling the system to select subsequent workflow actions based on signals including HTTP status codes, error types, dependency states, authentication states, and resource availability. Candidate actions include retry, token refresh, resource resolution, test data regeneration, and result reporting, with the goal of improving workflow continuity and recovery capability. Within this design, the workflow recovery layer provides executable recovery mechanisms for scenarios such as 404 errors, resource

unavailability, and dependency failures, while the workflow decision policy is responsible for selecting the next action among candidate recovery operations.

For empirical evaluation, this study uses 100 manual-doc-grounded tasks from the TMDb dataset as the primary benchmark, comparing test performance across versions v1 to v5. Results show pass rates of 0.25, 0.72, 0.69, 0.69, and 0.79 for v1 through v5 respectively. Notably, v5 maintains a Strict Pass Rate of 0.69, identical to v3 and v4, while its Final Pass Rate improves to 0.79 with 10 additional Workflow Recovered Pass cases, indicating that the primary benefit derives from post-execution workflow recovery rather than improvements in upstream test generation. Further analysis reveals that current recovery gains are concentrated in 404 resource dependency failure scenarios.

Regarding offline reinforcement learning, this study constructs an offline dataset from workflow transition events and trains an Offline-Q Policy using Tabular Fitted Q Iteration. The primary purpose of this component is to explore the feasibility of formalizing the workflow decision problem as a learning-based workflow decision strategy. Results demonstrate that the learned policy successfully captures partial workflow decision patterns and provides incremental benefits in specific recovery scenarios. However, due to constraints including a single API domain, Live API environment variability, and limited offline dataset scale, the overall performance is insufficient to claim comprehensive superiority over the rule-based baseline. Overall, this study validates the feasibility of integrating large language models, multi-agent collaboration, and workflow decision strategies into RESTful API automated testing, and provides a concrete reference architecture for the design and evaluation of future intelligent API testing systems.

目錄

摘要	i
Abstract	iii
目錄	v
圖目錄	vii
表目錄	viii
第一章 緒論	1
1.1 軟體開發中的系統測試	1
1.2 RESTful API 語法正確性與語意一致性	2
1.3 標準化且模組化的人工智慧代理生態協定	3
1.4 自動化測試生成與驗證流程概述	4
1.5 研究目標	5
1.6 研究架構	5
第二章 文獻探討	9
2.1 系統測試的發展歷史	9
2.2 RESTful API 自動化測試技術與侷限性	10
2.3 LLM 驅動的 RESTful API 自動化測試核心方法	13
2.4 結合 LLM 的語義增強與序列優化技術	14
2.5 馬可夫決策過程與強化學習於工作流程決策之應用	16
2.6 大型語言模型領域中 AI Agent 測試技術發展歷程與里程碑	18
2.6.1 多代理系統與傳統架構的挑戰	19
2.6.2 Multi-Agent 自動化生成測試案例	20
2.6.3 Hybrid Architecture 協作機制的發展與創新理念	20
第三章 研究方法	23
3.1 研究架構與整體流程	23
3.1.1 環境架構之職責分離	24
3.1.2 Agent 角色與責任分工	30
3.1.3 系統架構之五個功能模組	32

3.1.4	工作流程節點設計架構	35
3.1.5	測試案例生成模組	42
3.1.6	API 執行與結果蒐集模組	44
3.1.7	實驗版本與策略演化設計	46
3.2	工作流程決策策略設計	49
3.2.1	馬可夫決策過程建模	53
3.2.2	狀態空間設計	55
3.2.3	動作空間設計	57
3.2.4	Rule-based Workflow Decision Baseline	59
3.2.5	Offline-Q Workflow Decision Policy	60
3.2.6	Process Reward Modeling 觀點之獎勵設計	66
第四章	實驗過程與結果	74
4.1	開發環境與系統配置	74
4.2	資料來源與測試目標	75
4.3	Benchmark 設計	77
4.3.1	Evaluation Metrics	79
4.4	實驗流程	80
4.4.1	Offline-Q 工作流程決策策略設定	84
4.5	Manual Document Benchmark 主實驗結果	84
4.5.1	Workflow Recovery Analysis	88
4.5.2	v4 與 v5 之差異分析	90
4.5.3	Offline-Q 策略學習之深度診斷	91
第五章	結論與未來展望	99
5.1	實驗成果總結	99
5.2	研究貢獻與限制	100
5.2.1	研究貢獻	100
5.2.2	研究限制	101
5.3	未來研究方向	102
參考文獻		105

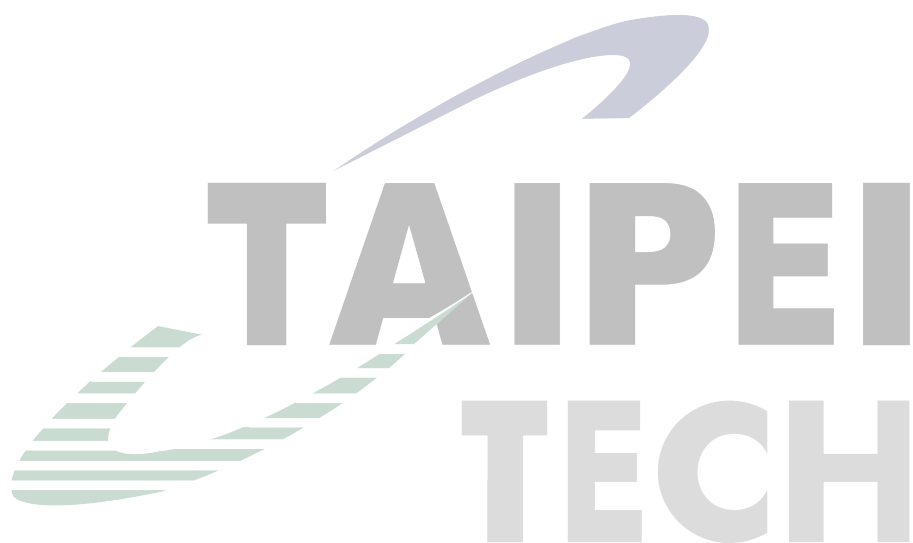
圖目錄

1.1	System Architecture Diagram	7
3.1	Environment Architecture Diagram	26
3.2	本研究代理架構與流程階段配置圖	30
3.3	Workflow & Layer Architecture Diagram	39
3.4	v5 版本之測試生成、執行與 Offline-Q 決策資料流	61
3.5	Decision Agent 與 Offline-Q Policy 之決策資料流	62
4.1	Offline-Q Policy 之 Q-table	94
4.2	Offline-Q Policy 之 Q-value 分布	95
4.3	Offline-Q 各決策行動之完整 Q-value 統計摘要	96
4.4	不同決策行動之 Q-value 分布	97

表目錄

2.1	本研究相關文獻定位表	12
2.3	本研究相關 HTTP 狀態碼與工作流程意涵	17
2.5	MCP, A2A, ACP, ANP Protocol 模型架構、特點與應用範疇比較表	21
3.1	環境架構之技術元件職責分離	28
3.2	本研究代理角色與責任分工	32
3.4	自動化測試之五層功能架構	35
3.6	工作流程引擎主要節點	40
3.8	工作流程狀態主要欄位流動	41
3.10	測試案例生成模組核心資料結構	43
3.12	Telemetry 主要蒐集欄位	46
3.13	v1 至 v5 實驗版本設計	48
3.15	Workflow Decision 與相關方法比較	51
3.17	Decision Input、Decision State、Feature State 與 Metadata 之對照	52
3.19	Offline-Q 策略之標準化動作空間	58
3.21	Rule-based Workflow Decision Baseline 之決策規則	59
3.23	Offline-Q 主要狀態特徵	64
3.24	Offline-Q Action Space	65
3.25	人工文件欄位與 METEOR 參考描述來源範例	69
3.27	v5 輸出紀錄範例	72
4.1	系統開發環境與主要元件	75
4.2	研究使用之 API 資料來源	76
4.3	100-Case Manual Document Benchmark 定義	77
4.4	v1~v5 版本定義	78
4.5	系統主要模組與功能	83
4.7	Offline-Q 工作流程決策策略設定	84
4.9	Version Benchmark Results	86

4.11 Workflow Recovery Cases in Version 5	89
4.13 Offline-Q 訓練結果摘要	92
4.14 Offline-Q 各決策行動 Q-value 統計	95



第一章 緒論

1.1 軟體開發中的系統測試

近十年間，軟體產業在敏捷開發、持續整合與持續部署（Continuous Integration / Continuous Deployment, CI/CD）以及微服務（Microservices）架構的推動下，逐步形成高頻率更新與快速交付的開發模式。[1] 在極短時間內完成程式修改、測試到上線部署的全流程，這種追求開發速度的飛躍，使系統功能得以持續演進，亦提升了產品迭代速度與市場回應能力；然而，開發效率的提升並未同步降低品質保證（Quality Assurance, QA）的難度，反而使測試活動面臨更高的即時性與覆蓋壓力。當系統由多個可獨立部署之服務構成時，每一次版本調整皆可能造成介面契約、資料狀態與服務相依關係的連動變化，使測試目標呈現高度動態化與組合爆炸特性（State explosion）。於此情境下，測試案例不僅必須快速反映最新規格與實作，亦須在有限時程內覆蓋跨模組、跨服務之關鍵互動路徑。若仍高度依賴人工撰寫與維護測試腳本，則常出現測試更新速度落後部署節奏、覆蓋範圍集中於正常流程（happy path），以及異常與邊界情境驗證不足等問題。[1] 這種不斷變化的系統狀態，使得經典的驗證與確認（Systematic Verification and Validation, V&V）方法難以執行。儘管系統能夠迅速上線，但傳統測試腳本、驗證邏輯與技術文件往往無法匹配此節奏，造成測試覆蓋率不足、缺陷追蹤效率下降，並形成技術文件與實際系統狀態之間的「知識漂移」（Documentation Drift），加劇了系統複雜性與可用能力之間的鴻溝。此即形成了所謂的「速度與品質落差」，開發節奏不斷加快，而品質驗證與知識管理機制卻相對滯後，使得組織在追求開發效率與維持系統可靠性之間陷入結構性取捨。系統測試之目的在於檢查整合後之軟體產品是否符合規格需求，並驗證其在功能性（Functional suitability）、安全性（Security）、可用性（Usability）與可維護性（Maintainability）等面向之品質表現。[2] 對於高度服務化之系統而言，系統測試已不再只是開發後期的附屬活動，而是必須在需求分析、設計、實作與部署過程中持續納入之核心工程機制。尤其在以應用程式介面（Application Programming Interface, API）為主要互動媒介之系統中，功能測試（functional testing）、整合測試（integration testing）、回歸測試（regression testing）與部分安全性測試（security testing），往往皆以 API 行為作為驗證基礎；因此，API 測試能

力的不足，將直接削弱整體系統品質評估之可信度。綜合所述，現代軟體開發產業的核心矛盾在於，系統迭代速度持續提高，但傳統測試方法在案例生成效率、跨服務覆蓋能力與維護可持續性方面逐漸失衡。[1][2] 因此，如何於快速演化之 API 驅動環境中，建立兼具效率、覆蓋能力與可維護性的系統測試機制，已成為軟體工程與自動化測試領域的重要研究議題。

1.2 RESTful API 語法正確性與語意一致性

隨著軟體系統由單體式架構（Monolithic Architecture）逐步演進為微服務（Microservices）架構，具表述性狀態轉移（Representational State Transfer, REST）風格之應用程式介面（Application Programming Interface, API）的特性輕量、快速且可擴展，成為雲端服務對外提供功能之重要入口。[3][4] 現代軟體系統普遍透過 HTTP 協議，承載雲端資源的 CRUD（Create, Read, Update, Delete）操作服務。[5] 由於 REST API 在現代雲端跨服務流程執行的關鍵節點，測試之困難不僅在於能否自動送出大量請求，更在於所生成之請求是否具備可被後端邏輯接受的語法正確性與語意一致性。[6] 所謂語法正確性，係指請求結構、欄位型別、參數格式與基本約束須符合 API 規格要求；語意一致性則進一步要求輸入內容須符合業務情境、資源狀態與跨操作依賴關係。[4][7] 若請求僅滿足結構格式，而未能反映實際商業邏輯與資源條件，則即使可成功通過部分欄位驗證，亦難以深入觸發後端關鍵流程與狀態轉移。傳統 RESTful API 自動化測試方法，多以 OpenAPI 規格作為輸入，透過靜態規格分析、自動推論操作依賴關係以及依序產生測試序列，以進行黑箱測試。[7][8][6] 然而，既有工具在實務上常面臨兩項關鍵限制。其一，測試資料多仰賴隨機值、字典或手動定義規則，導致生成之輸入缺乏語義真實性，難以通過 API 閘道對請求內容之實質檢查，因而產生大量無效請求。[4][7] 其二，序列生成通常偏短，缺乏跨資源、跨步驟之依賴推論能力，因此不易觸發較深層之系統狀態與端對端互動邏輯。[9][10][11] 近年來，大型語言模型被引入 RESTful API 測試領域，逐漸成為改善上述限制的重要方向。相關研究指出，大型語言模型可根據參數名稱、欄位描述與操作語境，生成較具業務合理性之輸入資料與操作序列，進而提升有效請求比例，並改善測試序列之可達性。[4][5] 然而，即使語義增強方法有助於提高輸入品質，真實執行環境中仍會面臨授權失效、資源不存在、驗證失敗、伺服

器異常與環境波動等動態問題。此類狀況並不必然表示測試流程應立即終止，而是需要根據當前執行狀態進行後續決策，例如重試、重新產生資料、解析資源依賴或進行失敗分類。因此，本研究將此問題定義為 RESTful API 測試之核心挑戰，已由單純之請求生成問題，延伸為測試工作流程能否持續且合理推進之決策問題。

1.3 標準化且模組化的人工智慧代理生態協定

大型語言模型（Large Language Models, LLMs）的高速發展，使其逐步從單純文字語言理解、生成工具的「專用 AI」，轉變為可協助規劃、推理、整合外部資源並能夠執行複雜任務的「通用 AI」（或通用認知元件）。[12][13] 在軟體工程領域中，需求迅速轉化促成代理式工作流程（Agentic Workflow）與多代理系統之興起，使測試生成、規格解析、程式碼產生、執行分析與報告整理等任務，得以透過多個具專責角色之代理分工完成。[14][15] 多代理系統之優勢，在於能將複雜任務拆解為多個具明確責任的子任務、規劃多步驟流程，並藉由代理間協作、透過工具調用與外部服務互動，提升問題處理能力、任務追蹤性與系統可擴展性，實現從測試生成到結果收斂的全流程自動化。[16][13] 然而，當前許多大型語言模型原生框架仍高度依賴提示鏈與集中式協調，將語義理解、流程編排與工具調用混合於同一推理流程中，導致例行性流程控制仍需耗費大量模型推理成本，進而限制整體可擴展性與穩定性。[13] 為改善上述問題，近年開始出現標準化且模組化之代理生態協定，以建立可互通、可追蹤且可整合之外部協作環境。其中，模型上下文協定（Model Context Protocol, MCP）主要處理模型與外部工具、文件、資料源之垂直整合，提供一致性的上下文存取與工具調用機制。[15][17] 透過此協定，模型可在統一介面下使用規格檔、技術文件、檢索系統與輔助工具，降低不同工具鏈之整合成本。相對而言，代理對代理協定（Agent-to-Agent Protocol, A2A）則著重於代理之間的水平協作，負責任務委派、狀態交換、能力發現與執行結果回傳，使多代理系統能以較一致之資料結構與互動模式進行協作。[13][17] 此類協定的價值，在於將代理協調機制自模型提示內容中抽離，使系統架構更具可觀測性、可維護性與可驗證性。因此，代理式工作流程、多代理系統、模型上下文協定與代理對代理協定，實質上形成一個彼此互補之技術生態：代理式工作流程負責定義整體任務流程，多代理系統負責任務分工與協作，模型上下文協定負責外部工具與知識資源調用，代理對

代理協定則負責代理間資訊交換與任務協調。[14][15][17] 此一標準化與模組化趨勢，為建構可落地之智慧化自動測試架構提供了技術基礎。

1.4 自動化測試生成與驗證流程概述

在自動化測試生成方面，OpenAPI 規格提供了端點、參數、請求格式、回應結構與驗證需求等結構化資訊，因此成為 RESTful API 測試生成的重要輸入基礎。[5][6] 然而，僅依賴規格檔進行靜態分析，往往仍不足以生成具語義合理性之測試輸入與跨操作測試序列。為此，近年研究開始結合大型語言模型，以補足自然語言描述理解、跨欄位語意推論與測試案例具像化能力。[12][18] 在此基礎上，多代理架構進一步將測試自動化流程拆解為多個功能模組，例如規格討論、規格整合、測試策略形成、測試案例生成、程式碼產生、執行控制、結果分析與報告輸出等，使不同代理得以依據角色進行協作。[16][14] 此種設計有助於降低單一模型在單次推理中同時承擔規劃、生成、執行與分析之負擔，亦提升整體流程之模組化程度。就工具整合而言，模型上下文協定可作為規格檔、人工文件、知識來源與輔助工具之統一調用介面；代理對代理協定則負責代理間任務交換與結果回傳；而工作流程編排框架則負責控制節點順序、狀態遞移與條件分支，使整體流程具備可追蹤與可重播特性。[15][13][19] 在此架構下，OpenAPI 規格與人工文件可先經由規格討論與整合代理轉換為較完整之語義測試上下文，再由測試策略與案例生成代理產生具體測試輸入與執行腳本，最後進入實際 API 執行與結果分析階段。然而，測試自動化之挑戰並不止於案例生成。當測試進入執行階段後，系統仍須面對動態環境中之失敗狀態，例如資源未建立、授權逾期、請求資料不一致或服務暫時異常。此時，若能將工作流程決策層與測試案例生成層分離，並建立一個後執行工作流程恢復層（Post-Execution Workflow Recovery Layer），負責在 404、資源缺失、依賴失敗或部分可恢復錯誤情境下維持流程可持續執行，再由工作流程決策策略於候選恢復行動中選擇下一步，例如重試、重新生成資料、解析資源、繼續執行或進行失敗分類，則可使測試流程更具持續性與可解釋性。[20][21] 因此，本研究將大型語言模型、多代理協作、OpenAPI 規格分析與工作流程決策策略整合為一套完整之 RESTful API 自動化測試流程。

1.5 研究目標

本研究以「基於強化學習工作流程決策策略之多代理 RESTful API 自動化測試框架」為核心，探討大型語言模型、多代理協作機制、模型上下文協定、代理對代理協定與工作流程決策策略，如何共同提升 RESTful API 自動化測試之流程穩定性。為使研究問題具體化，本研究提出下列研究問題：

1. **RQ1**：結合 OpenAPI 規格、人工文件與大型語言模型語義推理，是否能提升 RESTful API 測試案例生成之語義合理性與可執行性？
2. **RQ2**：在多代理架構下，MCP 與 A2A 協定的職責分離設計如何支援測試流程的模組化與可追蹤性？本研究以架構分析與執行軌跡觀察回答此問題，不涉及移除協定的消融比較。
3. **RQ3**：將執行後工作流程恢復形式化為可決策的序列問題後，rule-based 策略與 learning-based 策略各自能在何種情境下改善測試流程的連續性？learning-based 策略在當前資料規模下的可行性與限制為何？

本研究之學習型決策策略主要定位為 workflow decision 之可行性探索，重點在於驗證該問題是否可被形式化為可學習之序列決策問題，而非直接宣稱其已全面優於規則式基準方法。為回應上述研究問題，本研究設定以下目標：其一，建立以 OpenAPI 規格為核心輸入之 RESTful API 自動化測試流程；其二，導入大型語言模型與多代理協作機制，以提升測試案例生成與驗證之語義品質；其三，結合模型上下文協定與代理對代理協定，建構具模組化與可擴展性之代理測試架構；其四，設計以工作流程決策為核心之執行處理機制，以提升動態 API 環境下之流程持續性；其五，透過實驗比較驗證本研究方法之可行性、恢復能力與學習型決策策略之限制。

1.6 研究架構

本研究提出一套結合多代理協作與工作流程決策策略機制之 RESTful API 自動化測試框架，其整體系統以 OpenAPI 規格與人工文件為主要輸入來源，並透過多代理協作、

工具整合與工作流程編排，形成可執行、可觀測且可重複驗證之測試流程。在整體架構上，系統首先讀入 OpenAPI Specification，作為測試流程之結構化規格基礎。接著由規格討論代理與規格整合代理，綜合 OpenAPI 規格與人工文件內容，整理測試任務所需之語義限制、操作依賴與輸入上下文。完成規格整合後，由 OpenAPI 解析模組建立可供後續節點使用之結構化資料，再交由測試策略代理與測試案例生成代理產生具體測試案例與執行內容。在代理協作層面，多代理框架負責代理角色之協作與任務分工；代理對代理協定負責代理間任務委派與資訊交換；模型上下文協定則負責連結外部工具、文件與知識資源，使代理可在一致介面下進行規格檢索、文件讀取與工具調用；工作流程編排框架則負責控制節點順序、狀態傳遞與條件分支。透過上述分工，系統可將語義推理、流程控制與工具使用明確區隔，降低整體耦合度。在測試執行與決策層面，系統於產生測試案例後執行實際 HTTP 請求，並將回應狀態、錯誤訊號、資源狀態、授權狀態與執行結果交由結果分析模組處理。其後，工作流程決策代理根據當前狀態選擇後續行動，例如重試、重新產生資料、解析依賴資源、繼續執行或進行失敗分類。此一決策層之核心目的，並非最佳化測試案例生成，而是提升測試工作流程在動態 API 執行環境下之持續性、穩定性與可解釋性。其中，工作流程恢復層負責提供資源解析、第二次執行與錯誤後續處理等恢復機制；工作流程決策策略則負責於候選恢復行動中選擇下一步動作。本研究之整體系統架構與工作流程如圖 1.1 所示。其核心模組可概分為規格輸入與解析模組、代理協作與規格討論模組、測試策略與案例生成模組、HTTP 執行模組、結果分析模組、工作流程決策模組以及報告生成模組。各模組之功能分工如下：規格輸入與解析模組負責整合 OpenAPI 規格與人工文件；代理協作模組負責多代理任務協調；測試生成模組負責產生具語義合理性之測試案例；執行與分析模組負責送出請求並解讀執行結果；工作流程決策模組負責於失敗或異常狀態下選擇後續行動；報告生成模組則負責彙整測試結果與決策歷程，輸出可供後續分析之測試報告。

本論文共分為五章，各章內容安排如下。第一章為緒論，說明研究背景、研究問題、研究動機、研究目標與整體研究架構，並介紹 RESTful API 自動化測試、大型語言模型、多代理系統以及標準化代理協定之相關概念。第二章為文獻探討，回顧系統測試、RESTful API 自動化測試、大型語言模型應用於測試生成、多代理系統、代理式工作流程、模型上下文協定與代理對代理協定等相關研究，據以整理現有方法之限制與

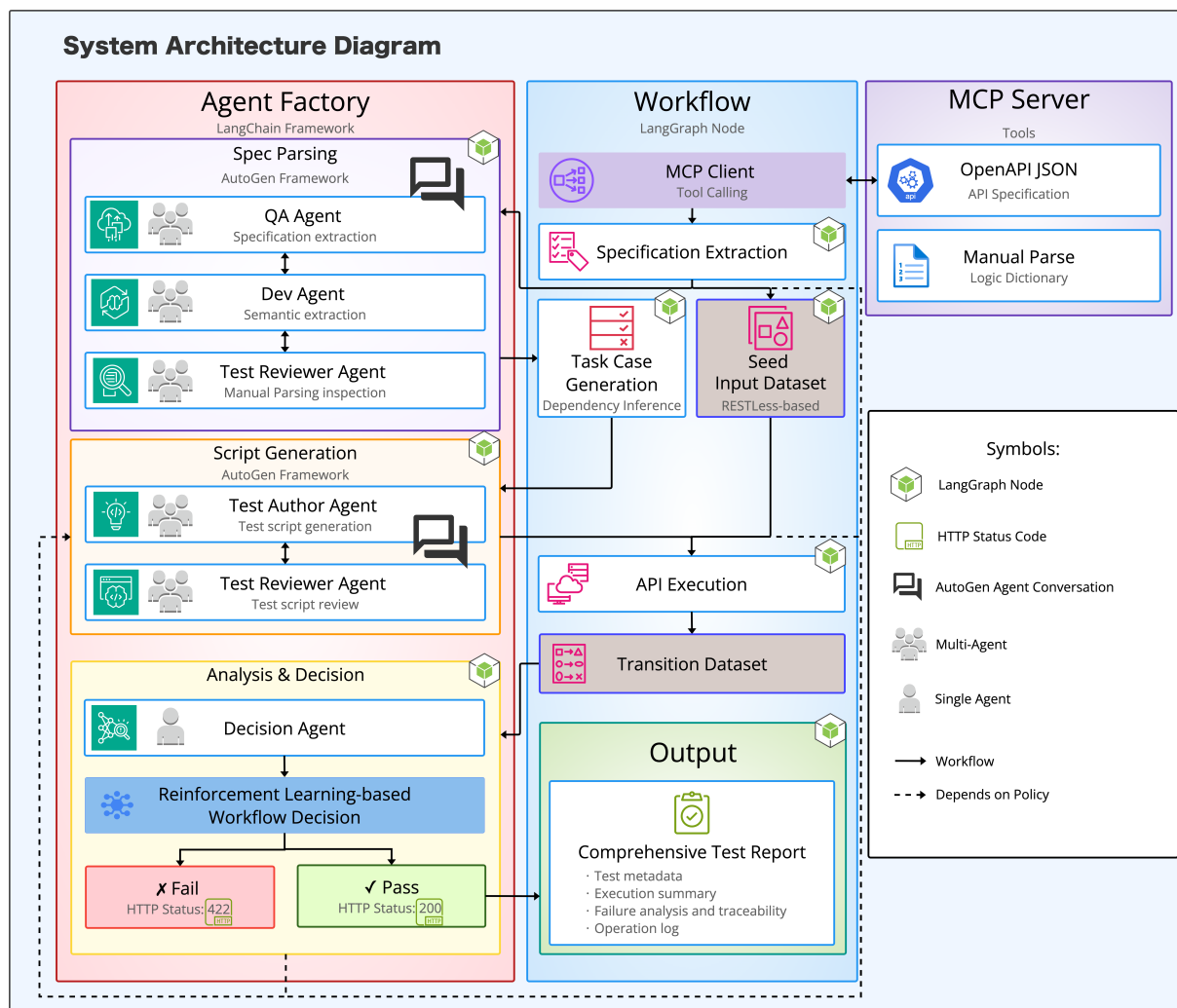
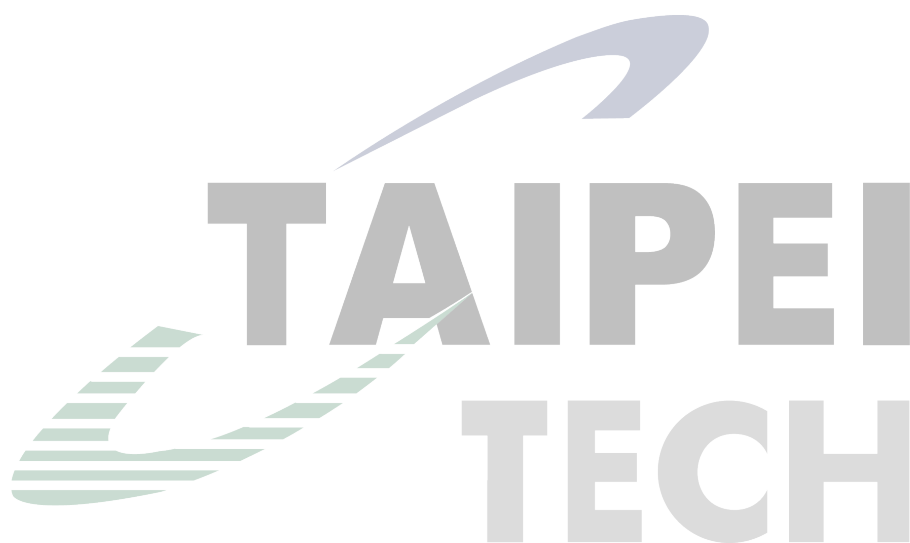


圖 1.1: System Architecture Diagram

本研究之切入點。第三章為研究方法，詳細說明本研究所提出之多代理 RESTful API 自動化測試框架，包括系統流程、代理角色分工、工作流程編排、工具調用機制、工作流程決策設計以及強化學習策略建構方式。第四章為實驗結果與討論，說明實驗設計、評估指標與資料來源，並展示不同版本測試流程與工作流程決策策略之實驗結果，分析其在測試成功率、恢復能力、執行品質與決策效果等面向之表現。第五章為結論與未來工作，總結本研究之主要發現與貢獻，說明研究限制，並提出未來在多代理測試架構、工具整合與工作流程決策深化方面之後續發展方向。



第二章 文獻探討

2.1 系統測試的發展歷史

在傳統軟體開發生命週期中，系統測試（System Testing）負責驗證完全整合後的產品是否符合規格要求，是品質保證（Quality Assurance, QA）對照初期需求進行全面驗收的重要階段。然而，隨著對開發速度與彈性的需求不斷提升，特別是在敏捷（Agile）方法興起後，耗時且成本高昂的測試流程（例如大規模回歸測試）逐漸被視為難以持續。因此，系統測試逐漸從傳統の後置活動，轉變為嵌入短週期開發衝刺（Sprint）中的持續性驗證與確認（Continuous V&V）流程。這一轉變促使自動化測試（Automated Testing）在軟體開發中迅速普及，品質保證團隊開始撰寫可重複執行的測試腳本，以確保功能在頻繁修改下仍保持正確性。隨著持續整合與持續部署（Continuous Integration / Continuous Deployment, CI/CD）的導入，快速部署與持續測試（Continuous Testing）進一步成為現代開發流程中的核心實踐，自動化的系統測試與整合測試亦被內嵌於 CI/CD 管線（Pipeline）中，成為穩定交付品質的重要條件。[2] 在此脈絡下，自動化測試逐漸扮演開發流程中的品質守門員角色，使團隊得以在高頻率版本釋出條件下維持基本品質保證能力。系統測試通常採用黑箱測試（Black-box Testing）方法，不要求測試者理解內部程式邏輯，而是透過檢驗輸入與預期輸出之間的一致性來驗證系統行為。[5][22] 於現代 RESTful API 的情境中，多數方法從 API 規範（OpenAPI Specification, OAS）推導測試案例。整體而言，系統測試主要涵蓋兩大類別：

1. 功能測試（Functional Testing）：檢驗系統是否如需求所述運作，確保功能性（Functional suitability）符合明示或隱含的需求。[2]
2. 非功能測試（Non-Functional Testing, NFT）：評估系統執行功能的品質，包括效能效率（Performance efficiency）、安全性（Security）、可用性（Usability）、可維護性（Maintainability）等面向。

儘管自動化測試在 CI/CD 流程中已廣泛涵蓋單元測試、整合測試與部分非功能測試，黑箱系統測試在實務上仍存在若干結構性限制。其中特別突出的問題之一為測試預言機（Test Oracle）不足，使得測試結果是否符合預期常難以以完全自動化方式判

定 [23][18][24] 傳統自動化測試多依賴 API 回應（例如 HTTP 狀態碼）或 API 規範（如 OAS）的不一致性偵測來判定結果，但這不足以支撐端到端（End-to-end）的正確性驗證。[7][22] 此外，在複雜的 RESTful API 系統中，既有方法難以處理操作間的複雜依賴性（Inter-operation Dependencies）和參數間依賴性（Inter-parameter Dependencies, IPDs）[5][3]，使得系統測試雖可自動產生大量請求，卻未必能有效形成具業務意義的測試流程。尤其在微服務架構下，雖然開發獨立性與部署彈性有所提升，但系統也同時呈現低可觀察性與行為不穩定性。微服務架構雖然增強了開發的獨立性與部署的彈性，但也帶來了低可觀察性（low observability）系統行為不穩定性。現有的微服務描述語言（如 OpenAPI Specification, OAS）多聚焦於單一微服務的 API 描述，卻缺乏對特定業務情境下行為預期或跨服務呼叫依賴關係的全面描述，這使得確保規格與實作之間的一致性（Conformance）成為一大挑戰。[25]

綜合相關研究可知，系統測試已由傳統後置驗證活動，轉變為持續交付環境下不可或缺的自動化品質機制；然而，現有黑箱自動化方法在語意驗證、跨操作依賴推論與端對端流程驗證方面仍存在明顯限制。此一缺口亦說明，若要支援 API 驅動之現代軟體系統，系統測試不僅需要更高程度的自動化，亦需要能同時處理規格理解、流程相依與執行階段決策之整合性方法，這也構成本研究後續文獻探討與方法設計之基礎。

2.2 RESTful API 自動化測試技術與侷限性

隨著軟體即服務（Software-as-a-Service, SaaS）成為主流，RESTful API 是可促使不同軟體系統進行資料交換與功能呼叫的標準介面，實現系統間的溝通與整合，已成為現代軟體架構中的核心互動介面，其品質驗證的重要性亦日益提高。[6][26] 然而，當開發流程朝向敏捷（Agile）與持續整合及持續部署（Continuous Integration / Continuous Deployment, CI/CD）的短週期模式發展時，自動化測試面臨的挑戰也隨之增加。這些挑戰不僅存在於測試腳本生成階段，亦延伸至實際執行與結果判定階段，反映出現行品質保證機制與開發節奏之間仍存在明顯落差。業界長期面臨的重要瓶頸之一，在於測試腳本與測試預言機（Test Oracle）難以完全自動化。尤其在缺乏形式化規格、完整斷言（Assertion）或可機器判定之輸出條件時，系統輸出是否屬於可接受之正確行為，往往難以透過自動化方式判斷。[22] 對於複雜功能之端對端（End-to-End）正確性驗證，

許多 CI/CD 流程仍需依賴人工檢查，形成持續性的維護成本。[6]

在腳本撰寫階段，主要困難來自如何由非結構化需求生成可執行之測試案例：

1. 傳統自動化方法往往假設系統需求可由統一塑模語言（Unified Modeling Language, UML）行為模型，如活動圖或循序圖，加以表達。然而在真實工業環境中，要建立足夠精確且可支撐測試生成之行為模型，其規格化成本通常過高，難以廣泛採用。[6]
2. 多數需求規格仍以自然語言（Natural Language, NL）撰寫使用案例規範（use case specifications）。若需由此類文件手動萃取執行場景與輸入資料，不僅成本高昂且容易出錯；即便使用自然語言處理技術進行測試模型生成，也常需大量人工修補，導致可擴展性不足。[6]

即使測試腳本已生成完成，在執行階段仍須面對現代 RESTful API 所帶來的多重困難：

1. 同時確保輸入資料具備語法正確性（Syntactic Validity）與語意一致性（Semantic Coherence）仍屬具挑戰性的問題。RESTLess 指出，許多模糊測試工具依賴字典或隨機產生參數值，使輸入缺乏語義真實性，導致大量請求在進入系統前即被阻擋。此類無效請求不僅降低執行效率，也浪費測試資源。[4]
2. 傳統自動化測試方法亦難以有效處理 RESTful API 的跨操作依賴與跨參數依賴。當欄位命名、資源關聯或參數語意稍有差異時，啟發式方法即可能誤判或忽略依賴關係；若改以人工補充依賴資訊，則將造成測試工程師額外的維護負擔。[5]
3. 由於無法精確處理複雜依賴關係，多數工具傾向生成較短且變化有限的測試序列，導致難以觸發隱藏於複雜操作組合下的深層狀態與錯誤，進而限制測試成效。[4][27]

近年來，大型語言模型（Large Language Models, LLMs）逐漸被應用於測試案例準備、測試預言機輔助生成、測試資料生成與依賴關係辨識等任務，展現出顯著潛力。[6][24] 相關研究指出，大型語言模型不僅可協助生成測試腳本、約束驗證腳本、測試案例與測試資料，亦可透過自然語言理解能力辨識較複雜的依賴關係。[5] 此外，大型語言模型亦可產生較具語義相關性與真實性的測試輸入值，對提升 RESTful API 自動

化測試之覆蓋能力，以及降低隨機輸入所造成之無效請求，具有明顯幫助。[3] 另一方面，結合外部工具之大型語言模型方法，也使測試流程中的推理與驗證能力得以提升。例如透過 Python 直譯器或外部工具協助模型逐步進行資料處理與結果驗證，可提高測試案例之準確性，並減少傳統腳本維護的負擔。[6] 然而，現有方法多半仍聚焦於測試案例生成、測試資料準備或局部依賴推論，對於整體測試執行流程中的動態控制與後續決策支援仍相對不足。因此，現有 RESTful API 自動化測試技術雖已在規格解析、腳本生成與語義輸入生成方面取得進展，但在真實執行環境中，仍面臨語意一致性不足、依賴推論不完整、端對端流程可達性有限，以及執行階段缺乏有效決策機制等問題。此一現象顯示，研究焦點不應僅侷限於測試資料或腳本生成本身，亦應進一步處理測試流程在動態 API 生態中的持續執行與決策問題，這亦為本研究後續探討之重要切入點。

表 2.1: 本研究相關文獻定位表

分類	代表文獻	可直接支持之主張	與本研究之關係	定位
REST API Testing	RESTler[9], EvoMaster[8]	有狀態或規格導向之 RESTful API 測試、錯誤觸發與覆蓋提升	可作為現有 RESTful API 測試基線與侷限比較	直接支持
OpenAPI Testing	RESTTestGen[11, 10]	由 OpenAPI 規格自動生成測試案例與請求	直接支持規格驅動測試生成之研究脈絡	直接支持
LLM API Testing	RESTLess[4], KAT[5], LlamaRestTest[3]	語義輸入增強、依賴推論、回應導向修正	直接支撐本研究之前置能力來源，但未直接處理執行後工作流程決策	直接支持
Multi-Agent	AutoGen[28], Bandi[14]	多代理分工、協作與角色化任務處理	支持本研究採多代理架構之合理性	直接支持
Agentic Workflow	RestGPT[29], Borghoff[13]	多步驟任務規劃、流程協調、回應導向後續行動	可作為流程編排與多步驟執行之借鏡，但非 API 測試 workflow decision 直接證據	借鏡
Offline RL	[30]	序列決策、狀態轉移與回饋學習之理論基礎	支持本研究以離線資料學習 workflow decision policy	直接支持
MCP	[17, 15]	大型語言模型與工具、資源及上下文之標準化整合	支持本研究之工具調用與上下文存取機制設計	直接支持
A2A	[17, 31]	代理間任務委派、能力描述與狀態交換	支持本研究之代理協作與跨代理狀態交換設計	直接支持

表 2.1 整理本研究所引用文獻之定位。整體而言，RESTler、RESTTestGen、EvoMaster、RESTLess、KAT 與 LlamaRestTest 可直接支撐 RESTful API 測試、OpenAPI 規格導向測試、語義輸入增強與依賴推論等研究脈絡；RestGPT、代理式工作流程與部分強化學習文獻則主要作為多步驟流程規劃與序列決策建模之借鏡，而非直接處理 API 測試執行後工作流程決策之研究。

2.3 LLM 驅動的 RESTful API 自動化測試核心方法

大型語言模型（Large Language Models, LLMs）驅動的自動化測試方法，近年已逐漸成為改善 RESTful API 測試限制的重要方向。相關研究顯示，此類方法主要可由三個核心層面提升既有自動化測試流程之能力，分別為依賴性解析與操作序列推導、語義測試輸入生成，以及結合外部工具之執行輔助與驗證強化。[5]

1. 依賴性解析與操作序列推導方面，大型語言模型之自然語言理解能力，使其得以處理傳統啟發式方法較難捕捉之複雜依賴關係。以 KAT 為代表的方法，透過模型解析 OpenAPI 規格中的自然語言描述，系統性辨識操作間依賴性（Inter-operation Dependencies）與參數間依賴性（Inter-parameter Dependencies, IPDs），並進一步建構操作依賴圖（Operation Dependency Graph, ODG），以支援較符合業務邏輯之測試序列生成。[5]
2. 在語義測試輸入生成方面，傳統方法的一項主要限制，在於缺乏具真實意義且語義有效之測試輸入。[7] 大型語言模型可根據參數名稱、欄位描述與上下文語意產生較具真實性的輸入值，從而提升有效請求比例並減少無效請求。[16][7] 代表性研究包括 ARTE、RESTLess 與 LlamaRestTest。ARTE 結合知識庫與自然語言處理技術，由 API 規範中抽取可用之測試輸入；RESTLess 則利用大型語言模型進行資料增強，建立高語義參數值資料集 RTSet，以補充 OAS 中不足之輸入範例；LlamaRestTest 進一步採用模型微調，使模型更聚焦於特定領域參數值之生成。[7][4][3]
3. 在執行輔助與驗證強化方面，大型語言模型亦可結合外部工具或執行器，以提升測試過程中的推理與驗證能力。例如，相關方法可透過程式執行環境協助模型逐步完成資料處理與結果分析，進而提高測試案例之準確度。[6] 此外，部分研究亦開始嘗試在執行階段動態利用伺服器回應、錯誤訊息與參數描述，修正輸入值或依賴規則，使測試流程具備有限度之動態調整能力。[3]

儘管大型語言模型方法在上述三個層面均展現出明顯潛力，既有研究仍指出傳統

黑箱測試工具在語義與序列長度方面存在明顯限制。一項針對七種代表性模糊測試工具的研究報告結果證實了現有工具在語義和序列長度方面的挑戰，在 18 個 RESTful API 上的實證比較顯示，這些工具的程式碼覆蓋率普遍低於 60%，反映出大量未覆蓋程式碼區段仍未被有效觸及。[32] 其原因之一，在於傳統方法生成的序列長度不足，因而難以觸及複雜、深層次的 API 操作組合。[33] 此外，由於 OpenAPI 規格本身通常不完整涵蓋參數間依賴性，且常存在規格不足（Underspecified Schemas）與語義要求未明確描述之情形，測試序列容易在第一層輸入驗證即失敗。[34][32] 在許多情境下，API 服務需以前序請求產生之動態物件，例如資源識別碼或驗證碼，作為後續請求輸入；若僅依靠隨機輸入或靜態字典生成，則極可能導致 404 等錯誤狀態，而無法深入觸發後端邏輯。[33][34] 因此，如何生成同時具備語法正確性、語意一致性與依賴關係合理性的測試輸入與操作序列，仍是 RESTful API 自動化測試的重要挑戰。[3][4] 即使語義增強方法已能有效提升測試輸入品質，例如 ARTE 所生成之語義有效輸入顯著優於隨機方法，[7] 相關方法仍未完全解決現實 API 中複雜輸入約束、多步驟操作依賴與執行階段動態變化等問題。[5] 綜合而言，現有大型語言模型驅動方法已在規格理解、依賴推論與語義輸入生成方面取得實質進展，並顯著改善傳統模糊測試工具所面臨的大量無效請求問題。然而，這些方法多聚焦於測試案例生成與請求有效性提升，對於真實 API 執行環境中之動態流程控制、失敗後續處理與跨模組協作機制仍著墨有限。此一缺口顯示，未來研究不僅需要更高品質之測試生成能力，亦需要能處理執行階段決策與工作流程連續性之整合式架構，而此亦構成本研究的重要研究定位。

2.4 結合 LLM 的語義增強與序列優化技術

RESTLess 是近年大型語言模型增強 RESTful API 測試中具代表性的研究之一，其主要目標在於改善傳統 REST API 模糊測試工具在輸入語義不足與測試序列可達性不足方面的限制。[4] 既有黑箱 REST API 測試工具多以 OpenAPI 規格作為輸入，並依據端點結構、參數型別與操作依賴關係自動產生請求。然而，OpenAPI 規格通常僅能描述欄位型別、必要性、路徑與基本回應格式，對於參數語義、業務規則、資源狀態與跨操作依賴之描述仍相對有限。因此，即使測試工具能夠產生大量請求，仍可能因參數值缺乏真實語義，而在 API 開道或後端驗證階段遭拒，形成大量 400 或 422 類型之無

效請求。

1. 語義增強 (Semantic Enhancement)：RESTLess 的核心貢獻之一，在於提出一個具備 62,250 個典型 REST API 操作的參數名稱和對應鍵值對的 RTSet 作為語義增強資料集，用以提升 REST API 測試過程中參數值生成的語義品質。其方法透過蒐集大量 REST API 常見操作、參數名稱與對應鍵值資料，並結合大型語言模型進行資料擴增，以產生具語義相似性與業務合理性的候選參數值。[4] 此一作法顯示，測試輸入資料不應僅滿足語法正確性，亦須盡可能符合 API 參數本身所隱含之語義要求。若參數名稱或描述涉及特定識別欄位、狀態欄位或結構化值，測試工具即應產生與其語境相符的輸入，而非僅以隨機字串或固定字典值代入。透過語義增強資料集改善參數生成品質，可使測試案例更貼近真實操作語境。當 RTSet 補充或替換原始 OpenAPI 規格中不足之參數範例後，系統便較有機會提升有效請求比例，使更多請求通過前端語法與語義檢查，進一步觸及後端服務邏輯。實驗結果顯示，經過 RESTLess 增強後的 SOTA 模糊測試工具，其有效請求序列 (Valid sequences, 即 20X 狀態碼) 的平均增長率達到 42.4%。對 RESTful API 測試而言，此點尤其重要，因為若測試資料在第一層驗證即被拒絕，則後續狀態轉移、跨資源依賴與深層錯誤皆無法被有效觸發。
2. 渲染順序優化 (Rendering Order Optimization)：測試序列生成方面，RESTLess 亦提出透過參數必要性與操作關係調整請求渲染順序之策略，使必要參數與較關鍵之操作優先被處理，以改善傳統測試工具容易產生短序列且缺乏多樣性操作鏈的問題。[4] 此方法有助於增加測試序列長度與操作組合多樣性，使測試流程更有機會觸及較深層的狀態與業務邏輯。在 RESTful API 測試情境中，許多操作並非彼此獨立，而是依賴先前請求所建立之資源或狀態。例如，若要測試查詢、更新或刪除某一資源，通常須先成功建立該資源並取得對應識別資訊。若測試序列無法妥善安排建立、查詢、更新與刪除等操作順序，便容易產生 404 或 403 等結果，進而使測試流程提早中斷。

整體而言，RESTLess 透過語義增強與渲染順序優化，提升了測試請求通過前端驗證與雲端開道檢查之機率，其主要貢獻仍集中於測試輸入與測試序列生成階段。此類方法有效改善了「如何產生較合理請求」與「如何延長測試序列」兩項核心問題，對提升黑

箱 RESTful API 測試之有效性具有重要意義，平均提升了 54.7%。然而，當測試流程進入真實 API 執行階段後，仍可能遭遇授權逾期、資源不存在、語義驗證失敗、伺服器錯誤、請求逾時或網路異常等動態執行狀態。這些情況並非皆可透過事前語義增強完全避免，亦不宜一概簡化為測試失敗。例如，401 可能意味需重新取得授權，404 可能反映前置資源尚未建立，422 則可能代表輸入資料仍需進一步修正。換言之，語義增強與序列優化雖能改善前置生成品質，但對於執行階段之後續流程處置，支援仍相對有限。因此，本節所回顧之文獻顯示，大型語言模型結合語義增強與序列優化技術，已有效推進 RESTful API 測試輸入與序列生成能力，但其研究焦點主要仍停留於請求生成前置階段。當測試進入真實執行環境後，若缺乏對失敗狀態之後續處理與流程延續機制，測試流程仍可能因單次非成功回應而提前終止。此一缺口說明，語義增強與序列優化應被視為自動化測試之必要基礎能力，但尚不足以完整支撐動態 API 生態中的持續測試需求；研究焦點亦因此自然延伸至執行階段之工作流程決策問題。

2.5 馬可夫決策過程與強化學習於工作流程決策之應用

既有 RESTful API 測試研究，如 RESTler[9]、RESTTestGen[11][10]、RESTLess[4]、KAT[5] 與 LlamaRestTest[3] 等研究，已分別推進有狀態 REST API 模糊測試、基於 OpenAPI 規格之測試生成、語義輸入增強、依賴感知測試與伺服器回應導向輸入修正等方向。然而，這些方法多半仍聚焦於如何產生更有效的請求、提高測試覆蓋率，或改善測試輸入之語義有效性；相較之下，對於 API 執行過程中失敗後如何根據當前狀態選擇下一步工作流程行為，相關研究仍相對有限。由於本研究的工作流程決策問題發生於 API 執行之後，因此非成功回應之語意不應僅被視為單純失敗訊號，而應被進一步理解為後續工作流程行動之判斷依據。表 2.3 整理本研究特別關注之 HTTP 狀態碼及其在 RESTful API 測試情境中的工作流程意涵。

在實際 RESTful API 工作流程中，非成功回應（Non-success Responses）不一定代表系統異常，也可能來自跨操作依賴、資源狀態不一致、動態參數需求或輸入資料語義不完整等因素。例如，請求可能因缺少有效資源識別碼、授權資訊不足或請求主體語

表 2.3: 本研究相關 HTTP 狀態碼與工作流程意涵

狀態碼	常見語意	於本研究之工作流程意涵
200/201/204	請求成功或資源建立完成	可視為成功狀態，流程得以持續推進或進入結果判定。
400/422	請求格式或語意驗證失敗	可能需重新生成測試資料、修正欄位值或調整輸入約束。
401	驗證失敗或憑證無效	可能需更新授權資訊後再執行。
403	權限不足	未必代表系統缺陷，亦可能屬於可接受失敗或權限限制情境。
404	資源不存在或依賴尚未建立	可能反映前置資源缺失，需先處理跨步驟依賴。
409	資源狀態衝突	可能反映資料狀態不一致，需結合流程脈絡判斷。
429	觸發速率限制	屬執行環境限制，可能需延後、重試或標記為環境因素。
500/502/503/504	伺服器或上游服務異常	可能屬暫時性環境問題，不宜直接等同產品功能缺陷。

義錯誤而回傳 422、404 或 401。此類情境在本質上更接近工作流程連續性（Workflow Continuity）問題，而非單純之測試失敗問題。另一方面，RestGPT[29] 及結合複雜 API 任務之大型語言模型測試集相關研究，已展示大型語言模型代理可將自然語言任務轉換為一系列 API 呼叫流程，並於執行過程中動態處理 API 回應，提取下一步所需參數，例如資源識別碼或權杖資訊。這研究證明了，根據 API 執行結果動態調整後續行動，對複雜 RESTful API 任務具有重要性。然而，其主要目標仍在於完成 API 任務本身與流程編排，而非將 API 測試執行過程中的工作流程決策行為，形式化為可重複驗證之狀態轉移問題。此類依據執行狀態持續調整後續行動的過程，本質上可視為一種序列決策（Sequential Decision-Making）問題，並適合以馬可夫決策過程（Markov Decision Process, MDP）加以建模。[30] 馬可夫決策過程提供一套形式化表示架構，可用以描述代理在環境中根據狀態、動作、狀態轉移與獎勵進行決策之過程，亦構成強化學習（Reinforcement Learning）方法的重要理論基礎。[30] 在軟體工程與智慧系統研究中，已有多項工作將複雜決策任務建模為狀態、動作與獎勵所組成之序列決策問題，以支援動態策略學習。例如，PyTester 將文本轉測試案例問題建模為馬可夫決策過程，並透過策略學習提升生成程式碼之可執行性與覆蓋能力；[21]AAMC 則將模型選擇問題形式化為多目標決策問題，以平衡效能、成本與延遲。[35] 在多代理決策研究中，近期亦有工作探討如何使大型語言模型與環境回饋對齊，以提升多代理強化學習中的探索、溝通與協同行為，顯示大型語言模型不僅可用於生成，也可作為決策輔助元件。[36] 此外，結合大型語言模型與多代理深度強化學習的研究亦指出，當決策問題涉及多步驟流程、資源限制與動態狀態轉移時，學習式策略有助於提升整體流程效率與決策品質。[20] 就 API 測試情境而言，若能將執行過程中的狀態變化、後續處置行動與結果回饋

加以形式化描述，則工作流程決策便可由固定規則控制，轉化為可分析、可比較且可學習之決策問題。相較於傳統以靜態規則驅動之工作流程控制方式，基於狀態轉移與回饋訊號之學習式方法，理論上更能根據歷史執行結果調整決策策略，並在不同環境狀態下選擇較適切之後續行動。然而，現有文獻對於此類工作流程決策問題之探討仍相對不足。多數 RESTful API 測試研究將改進重點放在請求生成、依賴推論與語義輸入修正，較少明確處理測試執行階段中失敗後應如何進行後續處置，以及這些處置行為如何形成可學習之決策過程。換言之，現有方法雖已改善測試輸入品質與序列可達性，但對於測試流程在動態環境中的持續推進能力，仍缺乏系統性建模。綜合上述文獻可知，馬可夫決策過程與強化學習提供了一套可用於分析工作流程連續性問題的理論基礎，使 RESTful API 測試之執行階段不再僅被視為單次請求成功與否的判定，而可進一步被理解為一個需要根據狀態持續選擇後續行動的序列決策問題。此一觀點亦構成本研究之研究定位：在既有語義增強與測試生成方法之外，進一步探討如何以學習式決策機制處理 API 測試執行階段之工作流程控制問題。

2.6 大型語言模型領域中 AI Agent 測試技術發展歷程與里程碑

大型語言模型（Large Language Models, LLMs）的出現，促使人工智慧系統由以往偏向被動式回應的工具，逐步轉向具備自主推理與任務執行潛力之系統。[37][14] 在軟體測試領域中，相關研究迅速將大型語言模型應用於測試用例生成、故障定位與測試輔助等任務，顯示其在處理自然語言需求、程式碼語境與測試資料生成方面具備明顯潛力。[38][39][16][12][23] 早期研究主要著重於利用大型語言模型輔助測試用例生成、輸入資料產生或故障定位等單一任務。[16][12] 例如，KAT 利用 GPT 模型與提示工程辨識 RESTful API 間的依賴關係，以提升測試覆蓋並減少誤報，證明了大型語言模型在生成測試腳本和資料方面的有效性；[5]FlexFL 則基於大型語言模型能夠在 Top-1（即可疑度第一名，以 Top-K 法評估定位演算法精確度的核心標準指標）排名中定位到 93 個傳統方法錯過的錯誤，並表現出更強的靈活性（Enhanced flexibility），同時利用錯誤報告與測試套件資訊，預測不穩定測試案例改善故障定位表現。[39][38] 此發展脈絡說

明，大型語言模型已不再僅是文字生成工具，而逐漸成為可支援複雜工程任務之推理元件。隨著大型語言模型能力提升，研究焦點亦逐步由單一模型處理單一任務，轉向更具自主性與協作性的代理式系統。所謂人工智慧代理（AI Agent），可視為能感知環境、進行推理並採取行動以完成特定任務之軟體實體。[14] 進一步地，代理式人工智慧（Agentic AI）則強調多個具專責角色之代理，透過協作、規劃與狀態交換，共同完成較複雜之高層次目標。[14][13] 此種發展對於 RESTful API 測試特別重要，因為相關任務往往同時涉及規格理解、測試生成、工具調用、結果分析與後續處置，單一模型未必能高效完成全部流程。

2.6.1 多代理系統與傳統架構的挑戰

多代理系統（Multi-Agent Systems, MAS）已被廣泛應用於自動化軟體測試、故障定位、資訊技術維運流程與其他複雜任務。[13][14] 其核心優勢在於可將複雜任務拆解為多個子任務，分派給不同代理處理，並透過工具調用與外部服務互動，形成較具彈性的工作流程。然而，現有第一代大型語言模型原生框架，例如 LangChain、LlamaIndex 與 AutoGen，雖然在代理開發與流程串接上展現高度潛力，但於大規模協作情境中仍暴露出若干架構限制。[13] LangChain 及其衍生框架 LangGraph 強調模組化工作流程、狀態轉移與圖形化執行能力，但其流程協調仍大量依賴模型推理，導致例行性流程控制也需付出高昂推理成本。[17][13] LlamaIndex 則偏重於知識整合與外部資料索引，較少處理多代理任務協調本身。[17] AutoGen 雖專為多代理互動設計，具備規劃、反思與協作能力，但其核心協調方式仍多依賴提示式對話機制，而非正式協定或可驗證之流程模型。[13][14] 此類以連續提示鏈與集中式模型協調為主的架構，雖可快速建構多代理流程框架，卻也帶來可擴展性不足、協調成本偏高與正式驗證困難等問題。[13] 相對而言，TB-CSPN 等形式化協調架構主張將語義處理與流程協調分離，以降低流程控制對大型語言模型推理之依賴，並提升協調過程的可追蹤性與可驗證性。實證評估顯示，TB-CSPN 比 LangGraph 風格的協調快 62.5%，大型語言模型 API 呼叫減少 66.7%，證明了架構分離的效率。[13] 這觀點對本研究具有重要啟示，因其指出多代理系統的價值不僅在於任務分工，更在於如何以合理架構支撐穩定之流程控制。

2.6.2 Multi-Agent 自動化生成測試案例

大型語言模型驅動的多代理系統，已逐漸成為自動化測試案例生成的重要發展方向，特別適用於需同時處理規劃、計算、工具調用與結果驗證之複雜任務。[16][14][37][13] 此類研究的核心想法，在於將測試生成流程拆解為具專責角色的子任務，由不同代理各自負責輸入生成、預期輸出推導、程式碼產生或結果檢查，藉此提升生成品質與流程效率。例如，TestChain 將測試案例生成問題拆分為測試輸入生成與測試輸出生成兩個子任務，Designer agent 專注於生成多樣化的測試輸入，包括基礎和邊緣案例，Calculator agent 則負責確定對應的預期測試輸出，並透過 ReAct 格式的對話鏈與 Python 解譯器進行互動。以 GPT-4 作為基礎模型的 TestChain 在 LeetCode-hard 資料集上的測試案例準確性（Accuracy）提高了 13.84%，證明了這種代理分工和外部工具協作的有效性，亦顯著減少了輸入及輸出映射中的複雜性和不準確性。[16]AutoGen 與 MetaGPT 等框架則進一步展示，多代理協作可透過規劃、反思與角色分工，提高處理複雜任務之能力。[14] 在微服務與端對端系統中，這種協作尤具意義，因為相關測試任務通常涉及多步驟資源依賴與多來源資訊整合。[25] 然而，既有多代理測試生成研究多聚焦於如何提高測試案例生成品質、改善工具調用能力與提升協作效率，對於測試執行過程中如何根據當前狀態調整後續工作流程，仍少有進一步建模與分析。換言之，多代理研究已展現其在前置生成與流程拆解上的優勢，但對於測試流程中斷後的決策機制與流程連續性支援，仍未形成一致且可驗證之研究主軸。

2.6.3 Hybrid Architecture 協作機制的發展與創新理念

（一）AI Agent 標準化協定概述與比較

隨著多代理系統逐步從研究原型走向實際應用，傳統臨時性整合（Ad-hoc Integration）已難以支撐大規模、多來源與跨平台之協作需求。為改善代理系統在互通性、安全性與擴展性上的限制，近年業界陸續提出標準化通訊協定，以建立較穩健之代理生態系統。[17] 其中，模型上下文協定（Model Context Protocol, MCP）主要著重於大型語言模型與外部工具、資源及上下文之垂直整合，提供一致的上下文攝取與工具調用介面；代理對代理協定（Agent-to-Agent Protocol, A2A）則著重於代理之間的水平

協作，支援任務委派、能力描述與結構化狀態交換。[17] 此外，代理通訊協定（Agent Communication Protocol, ACP）與代理網路協定（Agent Network Protocol, ANP）則分別代表通用整合與去中心化互聯方向，反映代理協定設計已逐漸由單點工具整合，延伸至更完整之分散式協作架構。就本研究主題而言，MCP 與 A2A 之價值尤為關鍵。前者有助於處理規格文件、知識資源與外部工具之存取一致性；後者則有助於代理之間的任務協作與狀態傳遞。二者分別對應垂直整合與水平協作需求，為代理式測試流程之職責分離提供了重要基礎。表 2.5 比較 MCP、A2A、ACP 與 ANP 在架構、應用範疇、互通性、安全性與擴展性上的差異。

表 2.5: MCP, A2A, ACP, ANP Protocol 模型架構、特點與應用範疇比較表

Protocol	Module	Scopes	Interoperability	Security	Scalability
MCP	客戶端-伺服器 (JSON-RPC)	垂直整合：LLM↔外部工具/資源整合。標準化上下文攝取和結構化工具調用。	高（LLM 與工具整合）。	Token-based auth, 需透過 mTLS、OAuth 2.1+ PKCE 緩解工具投毒和憑證竊取。	集中式伺服器假設，資源由客戶端管理。
A2A	類點對點 (Client↔Remote Agent)	企業協作：實現代理間的結構化互動與任務委派。使用 Agent Card 實現能力發現。	適合企業信任邊界內的協作。	DID-based 身份驗證，使用 JSON Web Signatures (JWS) 防止任務偽造。	支援 SSE/Push Notifications，適合分佈式生態系統。
ACP	代理中介客戶端-伺服器 (RESTful HTTP)	基礎設施層：通用通訊協定，支援多模態訊息和可擴展系統整合。	模型無關 (Model-Agnostic)，採用 RESTful HTTP，適合廣泛部署。	Bearer tokens, mutual TLS, JWS。	輕量且運行時獨立，支援可擴展的代理調用。
ANP	去中心化點對點 (P2P)	開放網路互聯：實現跨平臺、去中心化的代理發現和安全協作。	最強（開放網路），使用 JSON-LD 進行語義資料交換。	依賴 W3C DID 進行去中心化身份驗證。	適用於開放代理市場，但協商開銷高。

（二）雙協定協作的創新案例與優勢

此外，近期研究亦顯示，結合上下文感知與動態任務卸載機制的代理式架構，有助於提升系統在變動環境中的決策適應能力。[40] 多代理強化學習亦已被用於需要即時回饋與多目標協調的控制情境，顯示代理式決策架構不僅適用於流程編排，也適合處理動態狀態下的策略選擇問題。[41] 在多代理系統的發展中，單一協定往往難以同時解決工具整合、上下文一致性、代理協作與能力發現等問題，因此多協定整合逐漸成為一項可行方向。相關研究指出，MCP 適合處理大型語言模型與外部工具之垂直整合，而 A2A 則適合處理代理之間之水平協作與任務委派。[31][17] 以資訊技術事件響應案例為例，相關研究展示了 A2A 與 MCP 之整合方式：協調代理可透過 A2A 負責任

務委派與狀態追蹤，而診斷代理則透過 MCP 存取診斷工具與知識資源，以維持多步驟操作中的上下文一致性。[31] 在這案例中顯示，MCP 與 A2A 雙協定整合有助於克服傳統多代理系統在上下文分享、能力發現與訊息格式不相容方面的限制，並為具多步驟性與高互動性的任務提供較清楚的分層架構。然而，相關研究亦指出，多協定整合本身仍可能引入語義不匹配、安全性風險與協調複雜性，因而需要更明確之分工架構與協調機制。[13] 就本研究主題而言，這觀察具有重要意義：MCP 與 A2A 的價值不僅在於工具與代理可以互通，更在於其是否能於 RESTful API 測試場景中形成可重複驗證之流程結構。類似的多代理分工與協同規劃觀點，也已被應用於智慧製造與進階排程問題，顯示代理式系統在多步驟決策與任務協調方面具有良好擴展性。[42] 綜合而言，現有 AI Agent、多代理系統與標準化協定研究，已顯示大型語言模型系統可由單一模型操作，逐步發展為結合多代理分工、外部工具調用與標準化互通機制之複雜流程架構。然而，多數研究重點仍在於任務生成、協作效率或協定設計本身，對於 RESTful API 測試流程中如何整合多代理協作、上下文工具調用與執行階段後續決策，仍缺乏系統性驗證。現有方法主要改善請求生成或語意輸入品質，對執行後工作流程決策與恢復策略之系統性建模仍相對不足，此一缺口正是本研究進一步切入之基礎。

第三章 研究方法

3.1 研究架構與整體流程

本研究提出一套結合多代理協作、工作流程控制與執行恢復機制之 RESTful API 自動化測試框架。系統以 OpenAPI 規格與人工分析文件為主要輸入，透過大型語言模型之語意推理能力，支援參數約束推斷、請求-回應依賴推理（request-response dependency inference）與測試案例生成，以提升測試輸入之語意合理性與可執行性。在執行恢復層面，本研究將 API 執行後之工作流程決策形式化為馬可夫決策過程（Markov Decision Process），使系統能根據執行結果選擇後續行動，而非於首次失敗時即終止測試流程。本研究設計並比較兩種決策策略：rule-based 策略作為可解釋之基準，Offline-Q 策略則作為學習型決策之可行性探索，兩者共同構成工作流程恢復層之比較基礎。在技術架構層面，系統整合 AutoGen 與 LangGraph 代理框架，並引入 MCP 與 A2A 雙協定實現職責分離（Separation of Concerns）：MCP 負責工具與上下文之垂直存取，A2A 負責代理間之水平任務交接，LangGraph 則統一管理工作流程節點順序與狀態傳遞。此框架以 LangGraph 作為工作流控制層，負責控制測試流程中的節點順序、狀態保存、條件分支、重試與終止條件；以 AutoGen 作為協作推理層，於特定節點中執行多代理語意分析、依賴推斷、約束衝突檢查與測試案例審查；並透過 LangChain 整合 LLM 呼叫、工具使用與結構化輸出解析。為了支援跨代理與跨工具的資訊交換，本研究進一步引入 MCP 與 A2A 的概念。MCP 用於代理對外部工具、OpenAPI 規格、串接規格與執行外部資訊存取的垂直資訊存取；A2A 則用於代理之間的水平任務交接與結構化分析結果傳遞。透過此設計，多代理系統不直接控制整體流程，而是在 LangGraph 所定義的節點中執行受限的語意推理任務，以降低長流程任務中的不可預期行為。在 RESTful API 測試中，OpenAPI 規格雖然能描述結構化資訊，但對於欄位語意、業務規則、跨操作依賴與實際可用參數值的描述仍然有限。既有 RESTful API 測試研究指出，自動化測試工具在處理 REST API 時常遭遇測試輸入缺乏語意、操作序列過短、依賴推斷不足與測試 oracle 不明確等問題 [18][32][22]。RESTLess 進一步指出，傳統模糊測試工具若僅依賴隨機值或簡單字典產生參數，容易產生大量語意無效的請求，導致請求在 API 閘門或後端驗證階段即被拒絕 [4]。KAT 與 LlamaRestTest 則分別顯示，大型語

言模型可用於推斷 OpenAPI 中的操作依賴與參數依賴，並可利用伺服器回應改善測試輸入與依賴規則 [5][3]。基於上述問題，於測試工作流程真實 API 執行條件下是否能持續進行是本研究關注之核心問題，而非單次請求任務。實際 API 測試過程中，系統可能遭遇授權狀態改變、資源不存在、語義驗證失敗、伺服器錯誤、網路逾時或外部環境不可用等狀態。若測試流程僅以單次 HTTP 回應作為成敗判定，將容易使整個流程過早停止，進而降低測試資料之可用性與評估結果之穩定性。因此，本研究將 API 測試流程中的動態執行狀態視為工作流程決策問題，加上權衡各技術的優勢與限制，為 API 測試自動化設計整體系統架構採用了多階層功能模組的環境及工作流程。在研究方法上，本研究將 RESTful API 測試流程拆解為三種層次的架構面。在測試流程方面，本研究將 RESTful API 測試分為五個主要層級：規格語意理解、測試案例生成、測試程式生成、API 執行，以及自動化分析層與工作流程決策層。系統首先解析 OpenAPI 規格（Specification）、規格文件與歷史執行紀錄，取得 API 端點（Endpoint）、參數（Parameter）、資料格式（Schema）、身分驗證（Authentication）與回應狀態（Response Status）等資訊；接著透過 LLM 與多代理協作推斷語意約束、請求-回應依賴（Request-Response dependency）與潛在衝突；再根據語意約束與依賴推理（Dependency Inference）生成具商業邏輯合理性的測試案例與請求序列（Request Sequence）。最後，系統執行產生的測試程式，收集 HTTP 狀態碼（HTTP Status Code）、回應主體（Response body）、錯誤類型（Error Type）與延遲時間（Latency），並將執行結果轉換為工作流程決策狀態，以支援後續的測試調整與評估。第一為環境架構，說明 LLM 在 LangChain 框架下結合 AutoGen、LangGraph 如何利用 MCP、A2A 協定形成可控制的多代理傳輸與協作測試環境。第二為系統架構，將整體系統分為五個功能層級，包含規格語意理解、測試案例生成、測試程式生成、API 執行，以及分析與工作流程決策。第三為工作流程架構，使用 LangGraph 節點定義測試流程中的節點、輸入、輸出與狀態轉移方式。

3.1.1 環境架構之職責分離

OpenAPI 規格文件是描述 RESTful API 的標準化規格，提供標準、開放的格式定義 API 的資源與操作，常用格式為 JSON 或 YAML。通常能提供介面結構、參數格式與回應，但無法完整描述業務規則、跨操作依賴關係、資料約束條件以及實際使用情境，

因此僅依靠單一大型語言模型進行規格理解與測試案例生成時，容易造成上下文混亂、任務邊界不清與流程不可重現。容易產生語意理解不足、推論偏差或測試案例品質不一致等問題。由於 RESTful API 測試流程同時包含規格理解、語意推理、資料轉換、請求執行、結果分析與流程判斷等不同性質的任務，若全部交由單一大型語言模型或單一代理處理，將提高上下文混亂、任務邊界不清與結果不可重播之風險，因此有必要將語意推理、流程控制與執行決策加以分離。為提升規格理解能力與測試內容品質，本研究建置多代理協作模組（Multi-Agent Collaboration Module），透過多個具備不同職責之代理人（Agent）共同參與規格分析、需求推理、測試案例審查與決策討論，使系統能夠在測試案例生成前完成較完整之規格理解與語意約束分析。本模組以 AutoGen 作為多代理協作框架，並結合代理人對代理人協定（Agent-to-Agent Protocol, A2A）實現代理間資訊交換機制，使規格知識、測試策略與執行結果能於代理之間傳遞與共享。多代理協作模組位於 OpenAPI 規格解析模組與測試案例生成模組之間，其主要功能為將 OpenAPI 規格、Manual Document、Gold Path 與 MCP Context 轉換為可供後續測試生成使用之語意知識。其中 Agent Factory 負責建立各類型代理人；Multi-Agent Discussion Workflow 負責代理人討論流程；A2A Communication Layer 則負責代理之間的訊息交換與知識傳遞。不同代理人負責不同層級之推理工作，使系統能夠將規格分析、測試設計與執行分析進行職責分離，降低單一代理負責所有任務所造成之推理負擔。其中 Discussion Trace 用於記錄代理人之間的討論歷程；Specification Discussion Result 則保存討論後之規格分析結果；Finalized Specification 則作為後續測試案例生成之主要輸入。

規格討論流程對應工作流程中的 Semantic Constraint Reasoning 節點，當 OpenAPI 規格完成解析後，系統將 OpenAPI 內容、Manual Document、Gold Path 與 MCP Context 送入多代理協作模組。首先由 QA Agent 從測試觀點分析規格需求與驗證重點；接著由 Developer Agent 分析 API 設計邏輯、資源依賴與操作順序；最後由 Moderator Agent 整合各代理意見並形成最終規格結論。此流程之主要目的是建立比原始 OpenAPI 規格更完整的語意描述，並產生後續測試生成所需之約束條件。

本研究之環境架構建立於多代理協作與狀態式工作流程控制之上。因此，本研究採取混合式架構，將不同技術元件放置於不同職責層級中，如圖 3.1 所示：

API Automation Test Environment Architecture										
Auto Test Flow	LLM	Agent (LangChain)	Multi-Agent (AutoGen)			MCP Services			LangGraph (Workflow)	
OpenAPI Spec Parsing	✓	✓	✓	QA Agent	Dev Agent	Moderator Agent	✓	JSON Parsing	Schema Validation	✓
Generate Test Case	✓									✓
Generate Test Code	✓	✓	✓	Test Author Agent	↔	Test Reviewer Agent				✓
API Execution	✓	✓					✓			✓
Decision Making	✓	✓			Decision Agent		✓	Generate Report		✓

Symbolize:



圖 3.1: Environment Architecture Diagram

1. LangChain：代理與工具整合主框架

- 在本研究中 LangChain 為主要應用框架，負責建立代理、呼叫大型語言模型、整合工具、管理結構化輸出解析，以及連接 MCP 的工具與上下文服務，使整體系統更容易維護與擴充。
- 主要承擔整體代理應用的開發骨架，使不同代理能以一致方式呼叫 LLM、上下文推理結構化，並將不同工作流程節點與 LangGraph 結合形成模組化任務節點。

2. AutoGen：協作推理層

- AutoGen 為協作推理層，主要用於需要多角色討論、互評與共識形成的語意任務。主要的價值在於支援針對工作任務性的對話協作，使不同角色代理能從不同角度審查 API 規格、參數語意、請求-回應依賴關係與測試生成合理性。AutoGen 的優勢在於能夠透過不同角色代理進行多角度推理，檢查參數限制、規格與手動文件之間是否存在衝突。
- AutoGen 原始研究指出，多代理對話可支援代理之間的協作、工具使用與任

務分工，適合處理需要反覆討論與審查的 LLM 應用任務 [28]。

- 然而，不必多方討論的工作流程則並不適合 AutoGen 負責，例如 HTTP 執行或整體流程控制等確定性任務，以避免代理對話產生不必要的延伸。

3. LangGraph：工作流程控制層

- LangGraph 以狀態式的工作流程控制管理整體測試流程的節點順序、條件分支、狀態保存、重試控制與終止條件。RESTful API 測試並非單次請求即可完成，而是包含規格解析、測試策略制定、測試案例生成、請求實體化、HTTP 執行、結果分析與報告輸出等多階段流程。這些階段需要明確的執行順序與狀態轉移，因此必須由確定性的流程引擎負責控制，而不應完全依賴代理對話自行推進。
- 主要任務包括定義各測試節點的執行順序、保存每一階段產生的中間狀態、限制多代理推理僅在指定節點內執行及確保整體流程具備可追蹤與可重播能力。換言之，LangGraph 是整體系統的流程骨架，而不是語意推理核心。此設計呼應既有 Agentic AI 架構研究中對於流程協調、形式化狀態與可驗證工作流的重視 [13][14]。

4. Model Context Protocol (MCP)：工具與上下文垂直存取層

- 標準化工具與上下文的垂直存取傳輸媒介以 MCP 協定，測試模組開放企業對接只需指定端點即可。對於 RESTful API 測試而言，代理在不同階段需要存取 OpenAPI 規格、系統分析文件規則、語意種子值、資源候選值、歷史執行紀錄與測試報告資料。若這些資訊直接散落於各代理對話中，將容易造成上下文不一致與資料遺失。因此，本研究透過 MCP 概念將外部工具、資料與上下文包裝為一致的服務介面。
- MCP 可支援的工具存取包括查詢指定端點的 OpenAPI 數據、取得指定規格、取得候選參數值、查詢資源庫、讀取執行紀錄，以及彙整測試報告所需資料。MCP 相關研究指出，模型上下文協定的核心價值在於提供模型與外部工具、資料來源及上下文之間的標準化連接方式，但同時也需要注意資料權限、工具安全與上下文一致性問題 [15][17]。[43][44] 此設計使代理不需直接

管理所有資料來源，因此本研究設計主要是透過統一的上下文存取機制取得必要資訊，而非代理間自由溝通機制。

5. Agent-to-Agent Protocol (A2A)：代理間水平任務交接層

- A2A 為代理間水平任務交接，用於描述代理之間的任務委派、分析結果傳遞與狀態交換。例如，規格分析階段透過 AutoGen 代理間對端點清單與欄位限制傳遞的討論與推論傳輸；案例生成代理可將測試案例交給測試生成代理；執行代理可將 HTTP 回應狀態與錯誤類型傳遞給決策代理；決策代理則可根據結構化結果判斷下一步行動。

整合相關技術元件之職責如表 3.1 所示：

表 3.1: 環境架構之技術元件職責分離

技術元件	架構定位	主要職責
LangChain	代理與工具整合主框架	建立代理、呼叫 LLM、整合工具、解析結構化輸出，並連接 MCP 風格工具與上下文服務。
AutoGen	協作推理層	在指定節點中進行多代理語意分析、依賴推斷、衝突檢查與測試案例審查。
LangGraph	狀態式工作流程控制層	控制節點順序、狀態保存、條件分支、重試、終止條件與實驗重播。
MCP	工具與上下文垂直存取層	存取 OpenAPI 數據、取得指定規格、取得候選參數值、查詢資源庫、讀取執行紀錄。
A2A	代理間水平任務交接層	支援代理之間的任務委派、狀態交換與結構化分析結果傳遞。

換言之，LangChain 負責讓代理能「使用模型與工具」，LangGraph 負責讓流程能「依照狀態圖執行」，AutoGen 則負責讓特定節點能「進行多角色協作推理」。此種分工可避免將模型推理、工具調用與流程控制混在同一層處理，也能提升後續系統維護與擴充的可行性。既有研究亦指出，LLM 應用框架若將語意理解與流程協調混合在同一層，可能導致額外的協調成本與可驗證性問題，因此將語意處理與流程控制分離是多代理系統設計中的重要方向 [13][14]。因此，本研究將 AutoGen 限定於 LangGraph 指定

節點中使用，並要求其輸出必須轉換為結構化格式，例如 JSON、表格化欄位或明確的狀態物件，供後續節點使用。而MCP 與 A2A 在本研究中具有明確差異，MCP 處理的是代理對工具與資料的垂直存取，A2A 處理的是代理與代理之間的水平協作。前者確保上下文一致性，後者確保任務交接清楚。透過兩者分工，本研究能同時支援外部工具整合與多代理協作，並降低單一代理集中處理所有任務所造成的複雜度。既有研究亦指出，A2A、MCP 與其他代理通訊協定各自處理不同層次的互通問題，若能明確區分資料存取與代理協作，可提升多代理系統的可擴展性與互通性。[17][31][45][46] 為使代理人之間能夠共享推理結果，本研究透過 A2A 概念建立代理間資訊交換機制。在討論過程中，每個代理皆可接收前一代理所產生之分析結果，並於其基礎上進行後續推理。其互動流程可表示為：

$$Agent_{QA} \leftrightarrow Agent_{Developer} \leftrightarrow Agent_{Moderator} \quad (3.1)$$

於測試生成階段則進一步形成：

$$Agent_{Author} \leftrightarrow Agent_{Reviewer} \rightarrow Agent_{Decision} \quad (3.2)$$

透過此種鏈式協作方式，可使代理人之間逐步累積上下文知識，而非各自獨立推理。多代理協作模組之執行流程如演算法1所示。

Algorithm 1 多代理協作流程

Require: OpenAPI Specification, Manual Document, Gold Path, MCP Context

Ensure: Finalized Specification

- 1: Initialize Agent Set
 - 2: QA Agent analyzes testing requirements
 - 3: Developer Agent analyzes dependency constraints
 - 4: Moderator Agent integrates discussion results
 - 5: Generate Specification Discussion Result
 - 6: Generate Finalized Specification
 - 7: **return** Finalized Specification
-

多代理協作模組位於整體工作流程之中介層，其主要負責連接規格解析模組與測試生成模組。因此，多代理協作模組實際扮演語意推理中心（Semantic Reasoning Hub）之角色。其主要功能並非直接產生測試案例，而是在測試生成之前完成規格理解、知識整合與約束推理，使後續模組能建立於較完整之 API 語意模型之上。

3.1.2 Agent 角色與責任分工

本研究之代理設計採取以 AutoGen 的框架「群組式協作代理」與「單一決策代理」並行的方式。群組式協作代理主要用於需要多角度審查、語意推理與共識形成的任務；單一決策代理則用於根據 HTTP 執行狀態產生明確的工作流程決策。此設計的核心理由是：規格理解與測試生成需要多角色討論，但 HTTP 狀態導向的工作流程決策必須保持一致、可重播且可評估，因此不宜讓多代理自由發散。圖 3.2 更清楚說明代理人在整體測試流程中的配置位置與協作關係，整理了本研究由輸入、規格解析、測試生成、API 執行至報告輸出之代理分布方式。相較於前述環境架構圖著重技術元件分層，此圖進一步強調代理角色如何嵌入各工作階段，以及哪些階段採用多代理協作、哪些階段採用單一代理或確定性執行模組。

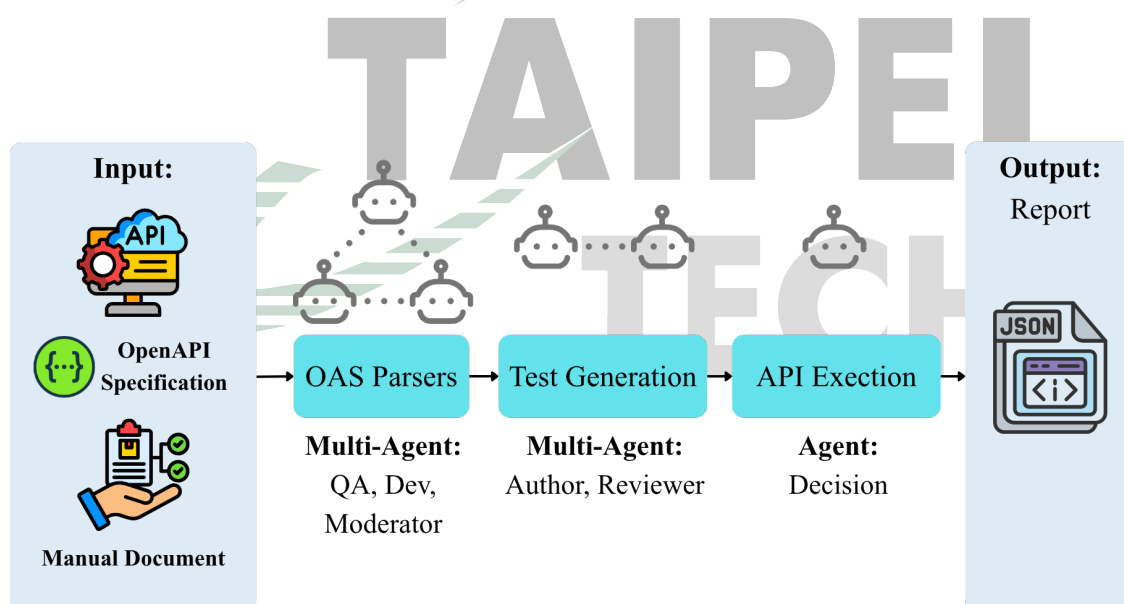


圖 3.2: 本研究代理架構與流程階段配置圖

由圖 3.2 可知，本研究之代理協作主要集中於 OpenAPI 規格解析與測試生成兩個階段。前者透過多代理討論形成較完整之規格語意、依賴關係與限制條件；後者則利用代理協作完成測試案例撰寫、審查與請求具體化。相對地，API 執行階段以確定性執行模組為主，避免代理對話直接介入 HTTP 執行流程；執行完成後，再由決策代理根據回應結果、錯誤類型與工作流程狀態決定後續行動，最後輸出結構化測試報告。

AutoGen 在本研究中負責協作代理。規格語意理解層為品質代理、開發代理與協調代理，主要用於規格語意推理、約束分析與衝突檢查。品質代理負責從測試與驗證角度檢查 API 規格是否足以產生測試案例；開發代理負責從後端實作與資料流程角度推斷可能的請求限制、資料依賴與業務規則；協調代理則負責整理前兩者之意見，形成一致的語意限制、依賴候選項目與衝突清單。AutoGen 原始研究指出，多代理對話可支援角色分工、工具使用與協作式問題解決，因此適合處理此類需要審查與共識形成的任務 [28]。測試程式生成層為測試撰寫代理與測試審查代理，主要用於測試案例轉換與測試程式審查。測試撰寫代理負責根據語意限制、依賴候選項目與輸入種子值產生測試案例、請求序列與測試程式草稿；測試審查代理則負責檢查請求參數是否完整、路徑參數是否正確替換、請求主體是否符合格式、檢查點是否合理，以及測試程式是否具備可執行性。既有大型語言模型軟體測試研究指出，模型生成測試程式時仍可能出現不可執行、缺少上下文或檢查點不完整等問題，因此透過審查代理形成第二層驗證，可提升測試生成品質與可執行性 [12][16]。決策代理在本研究中為單一代理，是專門處理 HTTP 狀態導向的工作流程決策，其輸入為 API 執行狀態、錯誤類型、端點、請求方法、授權狀態、資源可用性、重試次數、請求主體複雜度、前置檢查結果、環境訊號與前一個行動；其輸出為下一步工作流程行動。此設計參考 ARAT-RL 將 REST API 測試中的操作、參數與資料來源選擇視為強化學習探索問題，並根據 API 回應更新探索優先權的概念 [47]。本研究進一步將此思想延伸至工作流程層級，使決策代理可針對不同 HTTP 狀態選擇下一個測試流程行動。

表 3.2: 本研究代理角色與責任分工

框架	功能階層	代理角色	主要任務	主要輸出
AutoGen	規格語意理解層	品質代理	從測試觀點檢查規格完整性、測試目標與檢查點需求	測試需求、風險清單、檢查點建議
AutoGen	規格語意理解層	開發代理	從後端實作與資料流程觀點推斷欄位限制、資源依賴與可能錯誤來源	語意限制、資料依賴、資源需求
AutoGen	規格語意理解層	協調代理	彙整品質代理與開發代理意見，消除衝突並形成共識	衝突清單、依賴候選項目、語意約束摘要
AutoGen	測試程式生成層	測試撰寫代理	將測試案例轉換為請求序列與測試程式草稿	測試案例、請求序列、測試腳本草稿
AutoGen	測試程式生成層	測試審查代理	檢查測試程式、請求實體化與檢查點是否可執行且符合語意	修正建議、可執行性檢查、測試審查結果
LangChain 單一代理	分析與工作流 程決策層	決策代理	根據 HTTP 執行狀態與歷史狀態選擇下一步工作流程行動	工作流程行動、決策依據、信心指數、獎勵佔位

3.1.3 系統架構之五個功能模組

本研究將 RESTful API 自動化測試系統劃分為五個功能層級，分別為規格語意理解層、測試案例生成層、測試程式生成層、API 執行層，以及分析與工作流程決策層。此五層架構的目的在於將 API 測試從「規格輸入」、「測試執行」到「結果分析」的流程清楚分離，使每一層皆具有明確的輸入、輸出與責任範圍。

1. 第一層：規格語意理解層（Specification Parsing Layer）

- 規格語意理解層負責接收 OpenAPI 規格、系統分析文件與歷史執行紀錄，並萃取端點、HTTP 方式、路徑參數、請求參數、請求主體格式、回應格式、身份驗證要求與狀態碼定義等資訊。此層亦會分析每個欄位名稱、說明、類型與規格，以推斷可能的語意約束。

- 此層的設計基礎來自 OpenAPI 格式的 REST API testing 的相關研究。RESTTestGen、RESTler 與其他規格形式的 REST API 測試方法皆顯示，OpenAPI 規格可作為自動化測試產生的重要輸入 [10][11][9]。研究證明將需求文件與源碼函數嵌入到統一的特徵空間，解決「詞彙不匹配」問題，準確建立追蹤連結。[48]
- 然而，OpenAPI 規格本身通常不足以完整描述商業邏輯與依賴關係，因此本研究在此層加入 LLM 分析提取其語意與規格串接，以補足規格語意不足的問題。

2. 第二層：測試案例生成層（Test Case Generation Layer）

- 測試案例生成層負責根據規格語意理解層產生的端點參數、語意約束、規格形式與依賴關係，生成抽象測試案例與請求序列。此層不直接產生最終程式碼，而是先生成具商業邏輯且讓後續提高執行可達性的測試意圖，例如測試目標、測試資料、預期狀態、依賴來源與執行順序。
- 另外，本研究在此層採用基於 RESTLess 輸入種子生成方式。RESTLess 的研究指出，透過語意增強資料集與 LLM 生成具語意合理性的參數值，可改善傳統 REST API 模糊測試中大量無效請求的問題 [4]。本研究參考其語意增強概念，根據參數名稱、格式、說明、類型與規格產生測試參數資料。此外，本研究亦參考 KAT 對依賴性 API testing 的設計，將請求-回應依賴實施納入測試案例生成流程 [5]。

3. 第三層：測試程式生成層（Test Code Generation Layer）

- 測試程式生成層負責將抽象測試案例轉換為可執行測試程式。此層包含請求具體化、測試腳本生成、建立檢查點、設置與解除邏輯安排等任務。大型語言模型可協助產生初步測試程式，但測試程式是否可執行仍取決於路徑參數替換、查詢字串的組合、JSON 主體的建構、頁首的設定、身份驗證碼的注入與錯誤處理等工程細節。
- 因此，本研究將測試案例生成與測試程式生成分開處理。測試案例生成層負責決定「要測什麼」，測試程式生成層負責決定「如何轉換成可執行請求」。

此分離可避免 LLM 直接產生不可執行或缺乏上下文的測試程式，也有助於後續評估程式可執行率。既有 LLM 軟體測試研究亦指出，大型語言模型在測試生成上具有潛力，但仍需要工具、結構化流程與執行回饋來提升可執行性與可靠性 [12][16]。

4. 第四層：API 執行層（API Execution Layer）

- API 執行層負責實際送出 HTTP 請求，並記錄每筆測試案例的執行結果。此層收集的資訊包含 HTTP 狀態碼、回應主體、標頭、回應時間、錯誤類型、超時、重試次數與環境訊號。此層採黑箱測試觀點，不依賴服務內部原始碼，而是根據 API 外部行為、規格描述與預期結果進行觀察。
- API 執行層的輸出不只是成功或失敗，而是後續分析與工作流程決策的重要輸入。LlamaRestTest 的研究指出，伺服器回應可作為調整測試輸入與依賴規則的重要訊號 [3]。本研究延伸此觀點，將 HTTP 執行結果視為下一步工作流程 decision 的狀態來源，使測試流程不只停留在測試結果判斷，而能進一步支援狀態式分析。

5. 第五層：分析與工作流程決策層（Analysis and Workflow Decision Layer）

- 分析與工作流程決策層負責彙整 API 執行結果，判斷測試案例是否符合預期，並產生後續工作流程行動。此層會根據 HTTP 狀態碼、錯誤類型、端點、方式、驗證狀態、資源信號、請求主體複雜度、重試次數與執行歷史紀錄等資訊形成工作流程決策狀態。
- 本研究的工作流程決策策略與其強化學習形式化設計將於後續章節補充。本章先保留方法位置，說明其在整體架構中的角色：工作流程決策策略不是取代測試案例生成器，也不是單純的錯誤分類器，而是位於 API 執行結果之後、報告輸出之前的工作流程決策模組。其目的在於使系統能根據執行狀態判斷下一步應繼續、重試、重新產生輸入、更新授權、標記環境異常或進入報告流程。此設計可將 API 測試中的動態執行狀態轉換為可觀測、可比較的工作流程決策活動，並為後續 MDP 與策略比較留下資料基礎。

表 3.4: 自動化測試之五層功能架構

層級	主要輸入	主要輸出	核心方法
規格語意理解層	OpenAPI 規格、系統分析文件、歷史執行紀錄	端點資料、語意約束、依賴關係	LLM 語意執行、提取規格形式
測試案例生成層	語意約束、規格形式、依賴關係	具像化的測試案例、請求序列	RESTLess 式語義增強資料集、相依關係生成
測試程式生成層	測試案例、請求序列、驗證設置	可執行的測試腳本、HTTP 請求、檢查點	請求實體化、結構化測試腳本生成
API 執行層	HTTP 請求、測試腳本、API 環境	狀態碼、回應主體、延遲時間、錯誤類型	黑箱 HTTP 執行
分析與工作流程決策層	執行結果、預期行為、執行歷史紀錄	分析結果、工作流程決策狀態、工作流程行動	結果分析、工作流程決策策略

3.1.4 工作流程節點設計架構

本研究以工作流程引擎（Workflow Engine）作為整體系統執行核心，負責串接 OpenAPI 規格處理、多代理語意推理、測試案例生成、測試程式實體化、API 執行、工作流程決策與報告產生等模組。此設計的主要目的在於將 RESTful API 自動化測試流程拆解為可管理、可追蹤且可重播的節點式流程，避免將規格解析、模型推理、HTTP 執行與決策判斷混合於單一程序中，造成系統維護困難與實驗結果不可重現。本研究之工作流程引擎實作於 `workflow_engine.py`，核心類別為 `WorkflowEngine`。該類別封裝 LangGraph 工作流程建立、節點註冊、邊線設定、條件分支、編譯與執行等功能。系統執行時，所有節點皆透過共用之 `WorkflowState` 傳遞資料，各節點僅處理自身所需輸入欄位，並將執行結果回寫至狀態物件中，供後續節點使用。本研究採用 LangGraph 作為 RESTful API 測試工作流程編排框架（Workflow Orchestration Framework），建立具狀態管理能力之工作流程圖（Workflow Graph），將各功能模組封裝為獨立節點（Node），每個節點皆具有明確的輸入、處理邏輯與輸出狀態，並由 LangGraph 保存

跨節點的中間資料，並透過共享工作流程狀態（Workflow State）進行資料交換。此設計使系統能夠保留完整執行軌跡，並支援後續決策分析、策略比較與結果回溯。相對地適合 RESTful API 測試流程，因為 API 測試通常包含多個相依階段，例如規格解析、語意分析、輸入生成、測試程式產生、HTTP 執行與結果分析，各階段之間具有明確的資料依賴關係。在實驗上，WorkflowEngine 透過 StateGraph 建立工作流程圖，並以 WorkflowState 作為狀態型別。其中，create_workflow() 用於建立新的狀態圖；add_node() 將功能模組註冊為工作流程節點；add_edge() 定義節點之間的固定執行順序；add_conditional_edge() 則用於建立依據狀態結果決定之條件分支；compile() 將工作流程圖編譯為可執行物件；invoke() 則根據初始狀態啟動整體流程。本研究目前實作之主要工作流程節點如表3.6所示。這些節點並非僅為概念設計，而是對應至系統中實際可執行之節點函式。

1. 輸入收集與規格解析

- 收集 OpenAPI 規格、系統分析文件與既有執行紀錄。此節點會解析 API 端點、HTTP 方式、參數、請求主體格式、回應格式、身份驗證要求與狀態碼定義。其輸出為標準化的 API 規格，作為後續語意推理與測試案例生成的基礎。

2. 語意推理與約束抽取

- 負責執行語意推理與約束抽取。此節點啟動 AutoGen 協作推理，由品質代理（QA Agent）、開發代理（Dev Agent）與協調代理（Moderator Agent）分別負責規格語意分析參數約束推斷與規格一致性檢查。此節點輸出語意限制（例如是否符合資料格式）、隱藏限制（例如跨欄位商業邏輯約束、前置條件約束等）、衝突清單（例如請求的資料與系統現有狀態牴觸）與依賴候選項目（例如兩個欄位之間存在的商業邏輯約束）。
- 此節點的理论基礎來自 LLM 對自然語言規格與依賴關係的解析能力。KAT 顯示 LLM 可從 OpenAPI 規格中辨識操作依賴與參數依賴 [5]；RESTLess 與 LlamaRestTest 則顯示 LLM 可產生較具語意合理性的測試輸入，並利用回應資訊改善輸入規則 [4][3]。

3. 測試案例生成

- 根據語意限制、規格形式、依賴候選項目與關鍵參數生成測試案例。此節點會先形成抽象測試案例，再依據端點依賴與請求-回應關係產生請求序列。此階段重點在於讓測試案例不只符合 OpenAPI 格式，也盡可能符合 API 的商業邏輯的語意與操作順序。

4. 測試程式與請求實體化

- 將測試案例轉換為可執行測試程式與 HTTP 請求。此節點會完成路徑參數替換、查詢參數產出、請求主體建構、請求標頭設定、身分驗證注入與測試驗證點生成。此節點可透過 AutoGen 讓測試生成代理（Test Author Agent）與測試審查代理（Test Reviewer Agent）共同處理，並由 LangChain 負責工具調用與結構化輸出解析。
- 參考 RESTLess 所提出之語義增強概念，作為測試輸入生成策略，利用大型語言模型理解欄位語義與產生合理參數值的思想。根據 OpenAPI 規格的資料，以及規格文件中抽取出的欄位規則，產生較符合 API 語境的候選測試資料，將抽象測試案例轉換為可執行之 HTTP 請求。

5. API 執行與回應收集

- 實際執行 HTTP 請求，並記錄 API 執行結果。此節點輸出狀態碼、回應主體、回應標頭、延遲時間、錯誤類型、重試次數與執行軌跡。此節點直接由 HTTP 執行器與 LangGraph 工作流程控制，以確保執行結果可追蹤且可重播。

6. 結果分析與工作流程決策

- 接收 HTTP 執行結果後，分析與工作流程決策節點首先將執行上下文轉換為結構化決策狀態，涵蓋 HTTP 狀態碼、語意錯誤類型、資源依賴狀態、授權狀態與重試歷史等訊號。決策代理依據此狀態，從標準化行動空間中選擇下一步工作流程行動，包括重試（retry）、更新授權（refresh_token）、解析資源（resolve_resource）、重新產生資料（regenerate_data）或進入報告流程（report）。

- 此決策層之設計目的不在於重新生成整條測試流程，而在於對既有執行結果進行後執行行動選擇（post-execution action selection），使測試流程得以在部分失敗情境下持續推進，而非於首次非 2XX 回應時即終止。
- 當決策代理選擇恢復型行動時，系統會進入工作流程恢復執行層，依據行動類型從資源池（resource pool）、manual binding pool 或執行期訊號中取得補充資訊，並重新執行對應請求。恢復執行結果將再次進入決策節點，形成可觀測之閉迴路。
- 本研究於此層比較兩種決策策略：rule-based 策略依預定規則映射狀態至行動，作為可解釋之基準；Offline-Q 策略則依離線學習所得之 Q 值估計選擇行動，作為學習型決策之可行性探索。兩者共用相同之狀態定義與行動空間，差異僅在於行動選擇的決策機制。

7. 報告生成與指標產出

- 最終產出測試報告，報告內容包含測試摘要、API 執行結果、主要失敗類型、工作流程決策摘要與後續建議。報告生成階段會透過 MCP 串接工具存取前述節點累積的產出，並由 LLM 整理為結構化報告。此設計可提升測試流程的可追溯性，使最終報告不僅呈現成功或失敗，也能呈現決策依據與流程紀錄。

此設計使系統能將多代理推理限制在指定節點中執行，同時保留整體流程決定性的實施。綜合五個階層及工作節點的架構如圖 3.3 所示：

API Test Automation Workflow Architecture

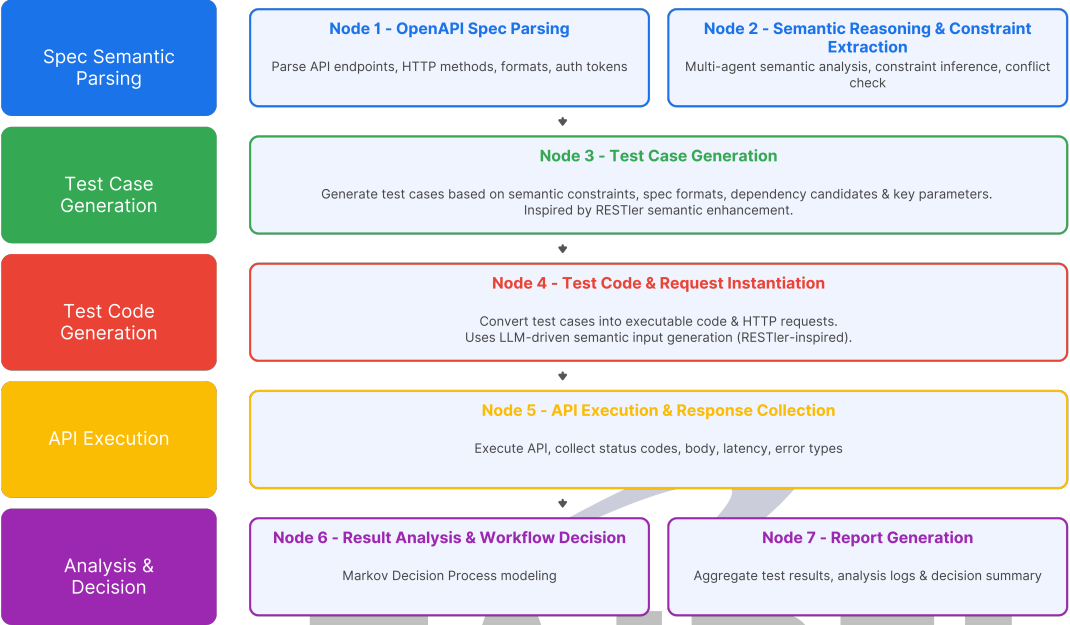


圖 3.3: Workflow & Layer Architecture Diagram

表 3.6: 工作流程引擎主要節點

節點名稱	主要目的	主要輸出
mcp_context_bootstrap	載入任務、Gold Path、Manual Rule 與規格上下文	source_task, gold_path, manual_rule, mcp_context_bundle
input_spec_parse	解析 OpenAPI 規格並整合人工文件規則	openapi_spec, endpoints, manual_doc_rules, reconciled_spec
semantic_constraint_reasoning	進行規格語意討論、manual parse 與 evaluator rule 產生	spec_discussion_result, manual_parse, evaluator_rule
seeded_input_generation	根據 manual parse 與規格內容產生 seeded input 資料	seeded_input_dataset
v2_seeded_execution	執行 seeded input 請求並解析跨步驟綁定	v2_test_cases, results, metrics
test_case_generation	產生抽象測試案例與測試策略上下文	test_cases, test_case_summary, candidate_parameter_context
test_code_materialization	將抽象測試案例轉換為可執行 pytest 程式碼	test_code, test_code_path, code_review, a2a_trace
api_execution	執行測試程式並蒐集 HTTP 結果與 reward breakdown	test_results, test_metrics, execution_context, reward_breakdowns
analysis_workflow_decision	根據執行結果產生工作流程決策	decision, decision_details, llm_telemetry_summary
report_generation	彙整測試結果、版本比較與策略分析報告	report, version_reports, version_comparison_report

工作流程狀態（Workflow State）是本研究系統中各節點交換資料的核心資料結構。每一個節點執行時，皆會從目前狀態中讀取所需欄位，完成任務後再將產生之資料寫回狀態中。此設計使整體流程具有狀態可追蹤性，並能保留每一階段產生之中間結果。WorkflowEngine 並不要求各節點直接互相呼叫，而是透過狀態傳遞進行間接溝通。本研究於明確定義每個節點的目的、輸入欄位、輸出欄位、下一個節點與下一節點所需輸入。此設定不僅用於輔助工作流程追蹤，也使系統在產生報告時能夠說明每一個節點的資料來源與資料流向。表3.8整理工作流程中主要狀態欄位之流動關係。工作流程引擎於執行時首先建立 Workflow Graph，接著依序執行各節點。執行過程中，每個節點完成後均會更新 Workflow State，並將結果傳遞至下一個節點。由於所有資料皆保留於共享狀態中，因此後續節點可直接存取前序節點之輸出結果。

表 3.8: 工作流程狀態主要欄位流動

狀態欄位	產生節點	後續使用節點
source_task, gold_path, manual_rule	mcp_context_bootstrap	input_spec_parse, semantic_constraint_reasoning, test_code_materialization
openapi_spec, endpoints	input_spec_parse	semantic_constraint_reasoning, test_case_generation
manual_doc_rules, reconciled_spec	input_spec_parse	semantic_constraint_reasoning, test_case_generation
manual_parse, evaluator_rule	semantic_constraint_reasoning	seeded_input_generation, api_execution
seeded_input_dataset	seeded_input_generation	v2_seeded_execution, test_case_generation
test_cases, test_case_summary	test_case_generation	test_code_materialization, api_execution
test_code, test_code_path	test_code_materialization	api_execution
test_results, test_metrics, test_execution_context	api_execution	analysis_workflow_decision, report_generation
decision, decision_details	analysis_workflow_decision	report_generation

此種狀態管理方式具有三項優點。第一，各節點之間不需直接相依，降低模組耦合度。第二，所有中間結果皆保存在 Workflow State 中，使實驗過程具備可追蹤性。第三，當某一節點產生錯誤時，系統可根據狀態欄位快速定位資料來源與錯誤位置，有助於除錯與後續分析。透過工作流程節點轉換決定各節點之執行順序與資料傳遞方向。本研究目前主要採用固定流程，並保留條件分支介面以支援後續擴充。此設計可在維持目前實驗可重現性的同時，保留未來加入錯誤恢復、重試控制與策略分支的彈性。目前主流程由 START 開始，依序通過 MCP 上下文初始化、規格解析、語意推理、seeded input 生成、測試案例生成、測試程式實體化、API 執行、工作流程決策與報告產生，最後進入 END。此流程可表示如下：

$$START \rightarrow N_1 \rightarrow N_2 \rightarrow \cdots \rightarrow N_k \rightarrow END \quad (3.3)$$

其中， N_i 代表第 i 個工作流程節點。除主流程外，系統亦保留基準版本執行節點，例如 v2_seeded_execution。這些節點用於版本比較與本研究實驗，使系統能在同一套工作流程架構中同時支援不同版本之測試流程。每個節點皆具有 objective、consumes、produces、next_node 與 next_inputs 等描述欄位，使系統能在執行後建立節

點級追蹤紀錄。此紀錄可用於檢查節點是否取得正確輸入、是否產生必要輸出，以及後續節點是否能接收完整資料。

Algorithm 2 工作流程引擎執行流程

Require: Initial workflow state S_0

Ensure: Final workflow report

- 1: Create StateGraph with WorkflowState
 - 2: Register workflow nodes
 - 3: Register node edges and optional conditional edges
 - 4: Compile workflow graph
 - 5: Invoke workflow with initial state S_0
 - 6: **for** each workflow node N_i **do**
 - 7: Read required fields from current Workflow State
 - 8: Execute node function
 - 9: Write produced fields back to Workflow State
 - 10: Record node trace metadata
 - 11: **end for**
 - 12: Generate final report and artifact outputs
 - 13: **return** final workflow state and report artifacts
-

透過上述設計，本研究能將多代理語意分析、確定性測試執行與工作流程決策整合於同一套可控制流程中。AutoGen 代理僅於指定節點中執行語意推理任務，LangGraph 則負責維持整體流程順序與狀態轉移。此設計可避免代理系統直接控制全域流程，降低不可預期行為，並提升實驗可重現性。

3.1.5 測試案例生成模組

測試案例生成模組（Test Case Generation Module）負責將前階段所產生之 OpenAPI 規格、Manual Document 解析結果、Gold Path 資訊、MCP Context 以及多代理規格討論結果，逐步轉換為可供執行之測試產出。本節所稱之測試案例生成模組，係指由語意種子生成（Seeded Input Generation）、抽象測試案例生成（Test Case Generation）以及測

試程式實體化 (Test Code Materialization) 所構成之上位測試生成流程。傳統 OpenAPI 測試工具多依據參數型別、Schema 結構或隨機策略產生測試輸入，容易產生大量語法正確但語意無效之請求。本研究則結合語意種子生成、測試案例推理以及測試程式實體化三個階段，使測試流程能夠從規格層逐步轉換為實際可執行之測試程式。本模組於整體架構中的位置介於多代理協作模組與 API 執行模組之間，負責完成從規格知識到測試執行程式之轉換。

依據目前系統實作，測試案例生成模組的三個工作流程階段構成節點，形成階段式轉換流程：

$$\text{Seeded Input} \rightarrow \text{Test Case} \rightarrow \text{Executable Test Code} \quad (3.4)$$

其中，Seeded Input 主要負責建立語意化測試資料；Test Case 負責建立抽象測試案例與請求序列；Test Code Materialization 則負責產生實際可執行之測試程式。

測試案例生成模組主要使用之資料結構如表3.10所示。

表 3.10: 測試案例生成模組核心資料結構

資料結構	用途
OpenAPISpecification	提供端點與參數定義
ManualParse	提供人工文件解析結果
SeededInputDataset	保存語意化輸入資料
TestCase	保存抽象測試案例
TestCaseSummary	保存測試案例統計資訊
CandidateParameterContext	保存候選參數來源資訊
GeneratedTestCode	保存可執行測試程式
CodeReviewResult	保存測試程式審查結果
A2A Trace	保存代理間協作紀錄

語意種子生成主要負責建立語意化測試輸入資料集，此設計受到 RESTLess 語意增強測試資料生成概念之啟發，但實作方式結合本研究之 Manual Document、Gold Path 與 MCP Context，使系統產生之輸入資料不僅符合參數型別要求，同時符合業務語意需求。其目的並非保證所有請求皆成功，而是降低明顯不合理輸入所造成之無效請求比例。Seeded Input 資料集之資料來源主要包含：OpenAPI Example Value、Enum Value、Document Rule、Gold Path Parameter 及歷史成功請求資訊。產出的 Seeded Input Dataset

將作為後續抽象測試案例生成之主要輸入來源。其後，Test Case Generation 會根據 Seeded Input Dataset 與規格知識建立抽象測試案例。此階段所建立之測試案例仍屬抽象表示，尚未轉換為實際程式碼。最終，Test Code Materialization 會將抽象測試案例轉換為可直接送入 Runtime Execution 節點之可執行測試程式。其中，Code Review Result 來自 Reviewer Agent 之審查結果，測試程式產生階段採用雙代理協作模式。其流程如下：

$$TestAuthor \leftrightarrow TestReviewer \quad (3.5)$$

Test Author Agent 負責建立測試程式內容；Test Reviewer Agent 則負責檢查 API 呼叫完整性、參數綁定合理性、驗證邏輯一致性以及 Manual Rule 符合性，審查完成後再輸出最終版本測試程式。測試案例生成模組實際扮演測試知識轉換層（Testing Knowledge Transformation Layer）之角色，其功能為將規格理解結果逐步轉換為可執行之 API 測試程式，作為後續 Execution 與 Workflow Decision 之基礎。

3.1.6 API 執行與結果蒐集模組

根據前一階段所產生之可執行測試程式與測試案例轉換此模組將為實際 RESTful API 呼叫，並於執行過程中蒐集回應結果、執行軌跡、錯誤資訊與決策訊號。相較於傳統僅驗證 HTTP 狀態碼之測試方式，本研究進一步保留完整執行上下文（Execution Context）、工作流程狀態轉移資訊以及可觀測性資料（Observability Data），使後續工作流程決策模組與離線強化學習模組能取得足夠之狀態資訊進行分析與學習。本模組於工作流程中對應之節點為 `api_execution`，主要輸入包含測試案例、測試程式碼、規格解析結果與執行環境資訊；此階段不採用模擬回應（Mock Response）或離線重播（Replay）方式，而是直接連線至實際 API 服務環境，以取得真實執行結果。所有測試案例均透過開發者授權取得之 Access Token 進行存取，並依據 OpenAPI 規格中定義之 HTTP Method、Path Parameter、Query Parameter 與 Request Body 建立請求內容。Execution 完成後，系統將建立標準化測試結果紀錄，其主要包含：測試案例識別碼（Case ID）、API 端點（Endpoint）、HTTP 方法（Method）、HTTP 狀態碼（Status Code）、回應內容（Response Payload）、執行時間（Latency）、執行結果（Pass / Fail）、錯誤分類（Error Type）及執行步驟資訊（Step Information）。Execution 模組接收測試案例生成

模組所產生之測試案例與測試程式碼，執行完成後將結果傳遞至 Telemetry Collection 與 Execution Trace Logging，並同步提供 Decision Agent 建立 Workflow Decision State 所需之執行資訊。Telemetry Collection 負責於 API 執行期間蒐集結構化觀測資料，其主要來源包含 HTTP 執行結果、依賴解析訊號、授權狀態與工作流程進度訊號。上述訊號經整合後形成統一之執行上下文（Execution Context），供後續三個用途使用：

1. 其一，作為決策代理建立工作流程決策狀態之輸入來源；
2. 其二，作為執行軌跡（Execution Trace）之記錄基礎，支援流程可追蹤性與結果回溯；
3. 其三，經狀態摘要、行動映射與獎勵計算後，轉換為狀態－行動－獎勵－下一狀態之轉移記錄，構成 Offline-Q 策略訓練所需之離線資料集。

需特別說明的是，Telemetry 資料與 Offline RL Dataset 並非同一層次之資料結構。前者為原始執行觀測資料，後者須經過額外的特徵編碼與獎勵計算才能用於策略訓練。此區分有助於釐清系統在執行觀測與策略學習兩個面向上的職責邊界。Telemetry 資料主要來源包含：HTTP Execution Result、Dependency Resolution Signal、Authentication Signal、Workflow Progress Signal 系統將上述訊號整合為統一之 Execution Context，以供後續 Workflow Decision State 建立使用。Telemetry 與 Execution Trace 屬於原始執行觀測資料；後續 Offline Reinforcement Learning 所使用之 transition dataset，仍需經過狀態摘要、標準化行動映射與獎勵計算後，才轉換為可供策略訓練之狀態－動作－獎勵－下一狀態資料。Telemetry 資料集主要包含表3.12所示欄位。

當API執行完成後，系統首先擷取 HTTP 回應與錯誤資訊，接著分析授權狀態、資源綁定情況、重試紀錄與流程進度，最後統一封裝為 Execution Context 結構。該 Execution Context 將作為 Decision Agent 建立 Workflow Decision State 之主要資料來源。而Trace Logging 負責記錄測試執行期間之完整工作流程軌跡（Execution Trace），以保留系統於每一階段所產生之中介結果與決策資訊。此設計之目的在於提供可重播（Replayable）、可追蹤（Traceable）與可分析（Analyzable）之執行紀錄，使研究者能夠追蹤單一測試案例於工作流程中的完整生命週期。Workflow Level Trace 記錄整體工作流程節點執行情形；Test Case Level Trace 記錄單一測試案例之執行結果；Step Level Trace 則記錄細部 HTTP 請求與回應資訊。最終產生之 Trace Artifact 將作為報告產生

表 3.12: Telemetry 主要蒐集欄位

欄位	說明
HTTP Status	API 回應狀態碼
Status Group	狀態群組分類結果
Semantic Error Type	語意錯誤分類結果
Retry Count	目前重試次數
Dependency Status	依賴資源狀態
Request Quality	請求品質資訊
Workflow Progress	工作流程進度資訊
Signal Quality	訊號品質資訊
Authentication State	驗證資訊狀態
Budget State	成本與資源狀態資訊

模組、策略分析模組與 Offline Reinforcement Learning 資料集建構模組之重要輸入來源。API 執行與結果蒐集模組位於整體工作流程之執行核心位置，負責將抽象測試案例轉換為實際 RESTful API 互動行為，並蒐集執行期間之各類可觀測性資料。透過 Execution、Telemetry Collection 與 Trace Logging 三個子模組，本系統能夠建立完整之執行上下文、工作流程訊號與決策分析資料，進一步支援後續 Workflow Decision、Offline Reinforcement Learning 以及報告產生等模組之運作。

3.1.7 實驗版本與策略演化設計

為說明各項模組逐步加入後對 RESTful API 自動化測試流程的影響，本研究設計 v1 至 v5 五個版本，作為累積式系統能力演進之描述。此版本設計並非嚴格單因素消融實驗，而是用於呈現規格理解、語意輸入建構、多代理協作、工作流程決策與恢復能力如何逐步被納入同一套測試框架中。本研究之主實驗以 100 筆人工文件與 gold path 定義之測試案例作為評估基準。每一筆測試案例皆包含任務描述、必要輸入、操作順序、預期結果與可接受失敗條件。因此，v1 至 v5 的比較重點不僅在於 HTTP 請求是否回傳 2XX 結果，也在於系統是否能依據 manual rule 完成任務層級判定。完整 OpenAPI 端點層級測試則保留為補充實驗，用於觀察系統於較大 API 集合上的執行穩定性與泛化情形。v1 為隨機執行基準版本。此版本不使用 manual rule、不使用 seeded input，也不導入多代理協作或工作流程決策策略。其主要目的在於建立最低基準線，觀察在缺乏規格語意與依賴資訊的情況下，系統直接產生請求並執行 API 時能達到的

基本通過率。此版本可用來說明隨機輸入在 RESTful API 測試中容易造成大量無效請求。v2 為規格感知與種子輸入版本。此版本導入 OpenAPI 規格解析與 seeded input，使系統能根據端點、HTTP 方法、參數型別、請求主體與欄位描述產生較合理的測試輸入。v2 的目的在於驗證 OpenAPI schema-aware request construction 是否有效改善 v1 隨機輸入造成的低通過率問題。相較於 v1，v2 開始具備基本規格理解能力，但尚未完整利用人工文件中的任務邏輯、gold path 與可接受失敗規則。v3 為 manual parsing、seeded input 與多代理測試實體化版本。此版本在 v2 基礎上加入 manual rule parsing、gold path 理解、多代理語意討論、測試案例生成與 pytest materialization。系統透過 QA Agent、Developer Agent、Test Author Agent、Test Reviewer Agent 與 Moderator Agent 進行任務分解、語意檢查與測試程式審查，使測試流程不僅依賴 OpenAPI 結構，也能參考人工文件所描述的任務語意與操作順序。v3 可視為本研究多代理 RESTful API 測試框架的基本完成版本。v4 在 v3 基礎上加入規則式工作流程決策策略。此版本導入 Decision Agent 與 rule-based workflow decision policy，使系統能根據 API 執行結果進行任務層級判定。當 API 回傳非 2XX 結果時，v4 不再直接將其視為失敗，而是根據 manual rule、acceptable failure 與錯誤分類判斷是否屬於可接受完成。此版本主要用於建立 decision-aware baseline，驗證 workflow-level verdict 與規則式決策邏輯對 RESTful API 測試評估之影響。v5 在 v3 基礎上導入離線 Q 值執行期決策選擇器。與 v4 依賴規則式策略不同，v5 使用離線工作流程轉移資料集訓練 Offline-Q Runtime Policy。此策略採用表格式擬合 Q 值迭代，根據歷史狀態、行動、下一狀態與獎勵估計不同狀態下各行動的價值，並於執行階段選擇 Q 值最高的行動。v5 的設計目的在於使系統能從歷史轉移資料中學習何時應重試、解析資源、更新授權、重新生成資料或進入報告流程，並驗證學習型工作流程決策策略是否能於相同工作流程環境下提供額外恢復能力。

在詮釋各版本數字時，有兩點前提需要說明。其一，v1 至 v2 之評估以 HTTP 狀態碼作為主要通過條件，v3 至 v5 則導入 manual rule 進行任務層級判定，評估標準的收緊會直接影響絕對通過率，因此 v2 至 v3 之間的數字變化不宜單純解讀為系統能力的退步。其二，v4 與 v5 並非在 v3 之上各自累積新能力，而是在相同測試生成基礎上替換工作流程決策機制，其比較重點在於兩種決策策略各自的有效情境，而非通過率的絕對高低。基於上述設計邏輯，本研究對各版本數字的詮釋重點如下：v1 至 v2 的提升反映規格導向輸入建構的效益；v3 相對於 v2 的變化反映評估標準收緊後的任務層級

校正；v4 與 v5 相對於 v3 的差異則反映工作流程決策層的介入效果，其中 v5 新增之 Workflow Recovered Pass 為本研究工作流程恢復能力的主要觀測指標。

表 3.13: v1 至 v5 實驗版本設計

版本	版本定位	主要設計目的
v1	Monkey Random Execution	不使用 manual rule、seeded input、多代理或決策策略，作為隨機執行基準線。
v2	Spec + Seeded Input	使用 OpenAPI 解析與 seeded input 改善請求建構品質，驗證規格感知輸入生成之效果。
v3	Manual Parsing + Seeded Input + Multi-Agent Materialization	整合 manual rule、gold path、多代理測試案例生成與 pytest 實體化，形成多代理 API 測試基本版本。
v4	v3 + Rule-based Workflow Decision Policy	加入規則式工作流程決策策略，根據 manual rule 與 acceptable failure 進行 workflow-level verdict。
v5	v3 + Offline-Q Runtime Decision Selector	使用離線 Q 值策略根據歷史轉移資料選擇下一步行動，支援資源解析、重試、授權更新、資料重建與報告行動。

綜合而言，v1 至 v5 之比較應視為同一套系統逐步加入不同能力模組後的描述性演進，而非演算法競賽式之單因素效能比較。特別是 v4 與 v5 的關係需要進一步說明。兩者在測試生成層面完全相同（均基於 v3），差異存在於兩個層面：其一為決策策略的表示方式，v4 採用規則映射，v5 採用 Offline-Q 值查表；其二為恢復行動的觸發條件，v4 之規則式策略在特定情境下可能未能觸發 resolve_resource 行動，而 v5 透過學習型策略在 404 資源依賴缺失情境中穩定選擇此行動，此為兩者產生不同恢復結果的主要原因。因此，v4 與 v5 之差異不應單純解讀為「rule-based vs. learning-based」的策略優劣，而應理解為兩種機制在特定錯誤情境下的觸發有效性差異。基於上述設計邏輯，本研究後續對版本結果之解讀以能力演進描述與決策策略可行性分析為主，Workflow Recovered Pass 為工作流程恢復能力之核心觀測指標。除決策策略本身外，本研究另行區分工作流程恢復層與工作流程決策策略兩個概念，並說明兩者之協作關係。工作流程決策策略負責在當前狀態 s_t 下，從既定動作空間 $\mathcal{A}$ 中選擇下一步行動。動作空間包含五種標準化行動，前四者為恢復型行動，後者為終止型行動。工作流程恢復層

則負責將決策策略所選擇之恢復型行動落地執行，提供具體的補救機制後再次執行。恢復執行完成後，其結果將再次進入決策節點，形成可觀測之閉迴路。換言之，決策策略回答「此刻應該選哪一種行動」，恢復層回答「所選行動如何被具體執行」，兩者分工使系統在架構上得以將策略選擇與補救執行解耦，並支援後續對不同決策策略（rule-based 與 Offline-Q）的替換比較。

3.2 工作流程決策策略設計

在 RESTful API 自動化測試在實際執行過程中，經常受到環境因素、授權限制、資源依賴關係以及測試資料品質等因素影響。非 2XX HTTP 回應不必然代表整體測試流程應立即停止，不同狀態碼與錯誤類型可能代表不同的後續處理需求。例如，401 可能與授權狀態有關，404 可能與資源依賴有關，422 可能與輸入資料語義有關，5XX 可能與伺服器暫時性狀態有關，而超時或網域名稱解析失敗（DNS Resolution Failure）則可能與外部環境有關。因此，單次請求失敗並不一定代表系統功能存在缺陷，而可能來自於測試環境或執行流程所造成之非產品缺陷（Non-product Failure）。若無法進一步判斷錯誤來源並採取適當補救措施，將難以支援長流程 API 測試與可重播評估，將造成大量誤判結果。因此，本研究提出將 RESTful API 測試流程建模為一個序列決策問題（Sequential Decision Problem），由決策代理（Decision Agent）負責在 API 執行後接收 API 執行器所產生的 HTTP 狀態碼、錯誤類型、執行軌跡、資源依賴狀態、重試次數與歷史執行結果，將其轉換為工作流程狀態（Workflow State），並選擇下一個工作流程行動，以提升測試流程完成率與結果可靠性。本研究所處理的問題在於某一步 API 請求執行完成後，下一步工作流程應採取何種行動。其決策單位為單一 test case 之單一步驟在一次 API 執行結果之後的一次 post-execution workflow action selection；決策時機發生於 API request 已送出且 response 或 error 已返回之後，亦即位於 execution 之後、report 之前。其輸入不是單一句法錯誤訊息，而是由 HTTP 狀態碼、語意錯誤類型、依賴綁定情況、重試歷史、授權狀態、工作流程進度與執行軌跡等資訊所構成之結構化工作流程狀態；其輸出則為可執行、可統計且可學習之標準化工作流程行動。此一設計之目標不是單純提升單次 HTTP 成功率，而是最大化 workflow continuation 與 final test-case completion，使系統能判斷在當前狀態下應重試、補足依賴資源、更新授權、

重新產生資料，或直接進入報告與歸因流程。為明確界定本研究的問題範圍，工作流程決策與相關方法有所不同。Retry Policy 通常僅處理單一請求是否重試；Test Repair 與 Self-Healing Test 主要著重失敗測試腳本或執行步驟之修補；Agent Planning 與 Task Planning 則著重任務分解、代理分工與執行規劃；一般 Recovery Strategy 多半僅描述補救原則，未必形成可比較之狀態轉移決策模型。相較之下，本研究的工作流程決策問題是以 API 執行後的狀態為核心，根據結構化執行上下文決定下一步工作流程行動，並將其形式化為可重播、可比較與可學習的序列決策問題 [29][28]。表 3.15 所比較之相關方法，涵蓋 RESTful API 測試中的請求生成與依賴推理方法 [4, 5, 3]、多代理協作與任務規劃方法 [28, 29]，以及序列決策建模之理論基礎 [30]。



表 3.15: Workflow Decision 與相關方法比較

方法類型	決策單位	輸入資訊	輸出行動	是否考慮歷史狀態	是否支援恢復機制	是否適合實時 API
Workflow Decision (本研究)	工作流程層級	HTTP 狀態碼、 錯誤類型、執行 軌跡、資源依 賴、授權狀態、 重試次數、歷史 結果	重試、資源解 析、更新授權、 重生資料、報告	是	是	是
Retry Policy	單次請求層級	當前錯誤或狀態 碼	retry/stop	否或弱	有限	部分適合
Test Repair	測試腳本層級	失敗腳本、定位 錯誤、斷言錯誤	修補腳本或斷言	部分	是	部分適合
Self-Healing Test	執行期修補層 級	執行失敗訊號、 替代定位資訊、 既有規則	替代元素、替代 步驟、重新執行	部分	是	部分適合
Agent Planning	任務規劃層級	任務目標、工具 描述、上下文	子任務序列、代 理分工	是	否或間接	部分適合
Task Planning	任務分解層級	高層任務描述、 文件、約束	執行步驟規劃	是	否或間接	部分適合
Recovery Strategy	錯誤處理層級	失敗類型、錯誤 規則	補救動作	部分	是	部分適合

表 3.17: Decision Input、Decision State、Feature State 與 Metadata 之對照

層級	形式化表示	內容	在本研究中的角色
Decision Input	X_t	原始執行上下文，包含 case/requestcontext、 executionoutcome、 manual-doccontext、 runtimepool、shorthistory 與 budget/nodecontext	作為決策前之原始輸入，不直接作為 策略查表狀態。
Decision State	$s_t = \phi(X_t)$	經摘要後之結構化工作流程狀態，例 如 HTTP 狀態、語意錯誤類型、 dependencystatus、 requestquality、retryhistory、 workflowprogress、 signalquality 與 budgetstate	作為工作流程決策之正式狀態表示， 用於規則式策略與後續離線狀態抽 象。
Feature State	$z_t = \psi(s_t)$	供 Offline-Q 使用之壓縮特徵狀態， 例如 statusbucket、 endpointgroup、 validationcluster、 retrybucket、 resourceavailabilitybucket 等	作為 v5 表格式 Q 值策略之查表索 引，以降低狀態空間稀疏性。
Metadata	—	原始 responsebody、完整 errormessage、fullmanualrule、 rawrequesttemplate、 resourcepool 原始內容、evidence 與其他輔助欄位	作為分析、追蹤與報告用途，不直接 視為正式 state variable。

表 3.15 與表 3.17 顯示，本研究所處理之工作流程決策問題，在決策單位、輸入資訊、輸出行動與恢復能力上，均不同於單純重試、測試修補或高階規劃方法；同時，本研究亦將原始執行上下文、正式決策狀態、離線特徵狀態與輔助性 metadata 加以區分，以避免概念混淆。

3.2.1 馬可夫決策過程建模

本研究將工作流程決策問題建模為馬可夫決策過程 (Markov Decision Process, MDP)，馬可夫決策過程可描述代理人在目前狀態下選擇行動，並根據環境回饋轉移至下一狀態與取得獎勵的序列決策問題 [30]。在本研究中，API 測試環境即為決策環境，Decision Agent 則為工作流程決策者。每一次 API 執行完成後，系統會根據執行結果形成狀態，策略選擇下一個工作流程行動，行動執行後再產生下一狀態與對應獎勵。就本研究的問題而言，決策輸入 (Decision Input) 為 API 執行完成後所蒐集的原始執行訊號，記為 X_t ，其可形式化表示如式 (3.6) 所示：

$$X_t = (C_t, Y_t, M_t, P_t, H_t, B_t) \quad (3.6)$$

其中， C_t 表示目前案例與請求上下文， Y_t 表示本次執行結果，例如 HTTP 狀態碼、回應內容與錯誤訊息； M_t 表示人工撰寫文件與任務規則相關資訊； P_t 表示執行期資源池、授權資訊與外部上下文； H_t 表示短期歷史紀錄，例如前一步狀態、先前動作與重試次數； B_t 示預算與節點上下文。上述原始輸入經摘要函數 ϕ 轉換為供策略使用之決策狀態，如式 (3.7) 所示：

$$s_t = \phi(X_t) \quad (3.7)$$

在Offline-Q 策略中，狀態 s_t 為將上述訊號摘要後形成的工作流程狀態，決策輸出 (Decision Output) 定義為在狀態 s_t 下所選擇之標準化工作流程行動 $a_t \in \mathcal{A}$ ；本研究之決策目標是在當前狀態下選擇最有助於後續流程延續與最終任務完成之行動。其最佳化目標可形式化表示，如式 (3.8) 所示：

$$a_t^* = \arg \max_{a \in \mathcal{A}(s_t)} \mathbb{E} \left[\sum_{k=t}^{T-1} \gamma^{k-t} r_k \mid s_t, a_t = a \right] \quad (3.8)$$

本研究將 MDP 定義如式 (3.9) 所示：

$$M = (S, A, P, R) \quad (3.9)$$

其中：

- S ：狀態空間（State Space）
- A ：動作空間（Action Space）
- P ：狀態轉移機率（State Transition Probability）
- R ：獎勵函數（Reward Function）

在傳統 RESTful API 測試工具中，測試案例執行後通常直接根據 HTTP 回應結果判定成功或失敗。然而實際系統環境中，測試失敗可能來自網路異常、授權失效、資源依賴缺失或測試資料錯誤等因素。因此，單一步驟的執行結果無法完整反映系統品質。本研究將工作流程視為一系列連續決策過程。當測試案例執行完成後，系統首先整合 API 執行狀態、錯誤類型、依賴關係、重試紀錄與歷史結果之結構化狀態 s_t ，接著由工作流程決策策略選擇可選擇的後續工作流程行動 a_t 。動作執行完成後，環境將產生新的執行結果並形成下一狀態 s_{t+1} ，同時計算該次決策所對應之獎勵值 r_t 。整體流程可形式化表示如式 (3.10) 所示：

$$s_t \rightarrow a_t \rightarrow s_{t+1} \rightarrow r_t \quad (3.10)$$

透過持續觀察狀態轉移與獎勵回饋，策略模型可逐步學習在不同測試情境下應採取何種補救行為，以提升測試流程完成率與決策品質。在一次測試流程中，工作流程軌跡可表示如式 (3.11) 所示：

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T) \quad (3.11)$$

其中， s_t 為第 t 步的工作流程狀態， a_t 為策略於該狀態下所選擇的工作流程行動。策略函數可表示如式 (3.12) 所示：

$$\pi(a|s) \quad (3.12)$$

其意義為在狀態 s 下選擇動作 a 的決策規則。本研究之目標並非單純最大化單一 API 呼叫的 HTTP 成功率，而是提升整體測試流程的可持續性與測試訊號品質。因此，策略最佳化目標可表示如式 (3.13) 所示：

$$\max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t R_t \right] \quad (3.13)$$

其中， γ 為折扣因子，用於控制未來獎勵對目前決策的影響程度。

3.2.2 狀態空間設計

狀態空間（State Space）用於描述 API 測試流程在某一時間點可被系統觀測到的狀態。本研究之狀態設計遵循馬可夫性質，即策略在進行決策時不直接讀取完整歷史軌跡，而是將與決策相關的歷史資訊摘要後納入目前狀態。此設計可避免策略依賴未結構化日誌，同時使相似狀態能在離線資料集中被重複比較與學習。在本研究中，正式的狀態變數是指由決策輸入摘要後形成，並納入 workflow 決策狀態之結構化欄位；狀態並非僅表示單一 HTTP 錯誤碼，而是描述在何種 workflow 狀態下出現該執行結果。因此，相同之 401、404 或 422，可能因授權狀態、資源依賴、重試次數或流程進度不同，而對應不同之後續 workflow 行動。原始的執行追蹤、完整的歷史結果與其他輔助證據則不直接作為狀態欄位，而是透過摘要形式反映於重試歷史、workflow 進度、相依性狀態、訊號品質與預算狀態等狀態特徵中。本研究所使用的狀態空間由 API 執行器所回傳之執行資訊組成結構化的決策狀態，主要包含式 (3.14) 所示之狀態變數：

$$s_t = \left(s_t^{task}, s_t^{outcome}, s_t^{dep}, s_t^{req}, s_t^{hist}, s_t^{prog}, s_t^{sig}, s_t^{budget}, s_t^{runtime} \right) \quad (3.14)$$

其中， s_t^{task} 表示任務與步驟識別資訊； $s_t^{outcome}$ 表示本次執行結果； s_t^{dep} 表示資源依賴狀態； s_t^{req} 表示請求品質； s_t^{hist} 表示重試與短期歷史； s_t^{prog} 表示 workflow 進度； s_t^{sig} 表示執行訊號品質； s_t^{budget} 表示呼叫與時間預算狀態； $s_t^{runtime}$ 表示執行期附加狀態。其中核心決策狀態可由案例識別、端點資訊、HTTP 狀態、狀態群組、語意錯誤類型，以及下列子結構共同組成。資源依賴狀態可表示如式 (3.15) 所示：

$$s_t^{dep} = (required_inputs_ready, missing_inputs, bound_from_previous_step) \quad (3.15)$$

請求品質可表示如式 (3.16) 所示：

$$s_t^{req} = (schema_valid, \\ required_fields_complete, \\ path_params_resolved, \\ payload_semantic_valid) \quad (3.16)$$

重試與短期歷史可表示如式 (3.17) 所示：

$$s_t^{hist} = (retry_count, same_action_count, recent_failure_pattern, previous_actions) \quad (3.17)$$

工作流程進度可表示如式 (3.18) 所示：

$$s_t^{prog} = (completed_steps, total_steps, progress_score, terminal) \quad (3.18)$$

$$progress_score = \begin{cases} \frac{completed_steps}{total_steps}, & total_steps > 0 \\ 0, & total_steps = 0 \end{cases} \quad (3.19)$$

執行訊號品質可表示如式 (3.20) 所示：

$$s_t^{sig} = (execution_signal_score, diagnostic_specificity, groundedness_score) \quad (3.20)$$

預算狀態可表示如式 (3.21) 所示：

$$s_t^{budget} = (api_call_count, remaining_call_budget, remaining_time_budget_sec) \quad (3.21)$$

除核心決策狀態外，系統亦保留執行期增補狀態，例如端點型態、是否需要授權、前次狀態碼、資源池命中情形、歷史成功率與目前節點等欄位，作為 $s_t^{runtime}$ 之一部分。此類欄位仍屬正式狀態變數，但主要用於執行期補充判斷。在Offline-Q 策略中，完整

結構化狀態 s_t 是進一步映射為緊湊特徵狀態 z_t 。其形式可表示如式 (3.22) 所示：

$$z_t = \psi(s_t, \text{signal}_t) \quad (3.22)$$

其中， z_t 主要由狀態群組、訊號分類、端點群組、驗證錯誤群組、缺失輸入區間、重試次數區間、是否具有更新權杖以及可用資源數量區間等特徵構成，用以降低狀態空間稀疏性。

3.2.3 動作空間設計

動作空間 (Action Space) 定義 Decision Agent 可選擇的後續工作流程行動。為降低策略學習複雜度，本研究定義五種標準化決策動作 (Canonical Actions) 作為強化學習策略之動作空間，如式 (3.23) 所示：

$$A = \text{retry}, \text{resolve}_{\text{resource}}, \text{refresh}_{\text{token}}, \text{regenerate}_{\text{data}}, \text{report} \quad (3.23)$$

這個動作空間是提供工作流程決策策略於執行期選擇下一步行動之用，其本質為 post-execution workflow action label，而非自由文字建議或高階任務規劃結果。透過將執行後決策離散化為有限個 canonical actions，系統可進一步統計不同狀態下之行動分布，並建立可供規則式策略與 Offline-Q 策略共同使用之決策介面。其中，*retry* 表示重新執行相同請求；*resolve_{resource}* 表示補齊或替換必要資源後重新執行；*refresh_{token}* 表示更新授權資訊後重新執行；*regenerate_{data}* 表示重新產生測試資料或請求主體；*report* 表示不再進行修復，直接進入報告產生與結果歸因階段。

表 3.19: Offline-Q 策略之標準化動作空間

Canonical Action	適用情境	決策意義
retry	暫時性伺服器錯誤、網路波動或可重試狀態	系統判斷目前請求本身可保留，僅需重新執行以排除暫時性因素。
resolve_resource	404 資源不存在、路徑參數缺失、資源池可用	系統嘗試從資源池、manual binding pool 或 runtime value pool 取得可用資源識別碼後重新執行。
refresh_token	401 授權錯誤、權杖失效、驗證資訊不足	系統更新授權資訊後重新送出請求，以避免因權杖過期造成誤判。
regenerate_data	400 或 422 驗證錯誤、請求主體不符合語意規則	系統重新產生測試資料，使請求更符合 schema 與 manual rule 所描述的語意需求。
report	權限不足、不可修復錯誤、環境異常或已具可解釋結果	系統停止嘗試修復，將結果分類並進入報告產生階段。

3.2.4 Rule-based Workflow Decision Baseline

為評估工作流程決策機制在不使用離線強化學習之情況下的基準表現本研究建立規則式工作流程決策策略（rule-based workflow decision policy）作為可解釋之決策基準，於本研究 100 筆主實驗中對應版本 v4。其決策邏輯依據錯誤分類、HTTP 狀態群組、依賴狀態與授權狀態，從標準化動作空間中選擇後續工作流程行動，不使用 Q 值表或離線策略學習。具體規則如表 3.21 所示。規則式策略之基本邏輯為：若狀態屬於授權錯誤，優先選擇更新授權資訊；若狀態屬於資源不存在，優先選擇解析資源並重新執行；若狀態屬於驗證錯誤，優先選擇重新產生測試資料；若狀態屬於權限不足、環境錯誤或已可解釋狀態，則進入報告階段。此策略可視為根據 RESTful API 測試經驗建立之專家策略，其優點是可解釋性高、行為穩定，且在錯誤類型明確的情境下通常能達到良好效果；缺點是難以根據歷史執行結果自動修正策略，也難以在多種候選行動皆可能合理時進行資料驅動的行動偏好比較。

表 3.21: Rule-based Workflow Decision Baseline 之決策規則

狀態群組	優先行動	說明
invalid_auth 或 auth_error	refresh_token	授權資訊可能失效，優先更新授權後重試。
insufficient_permission	report	權限不足通常不宜透過重試解決，應分類並產出報告。
not_found	resolve_resource	資源不存在可能由錯誤資源識別碼或前置資源缺失造成，應先嘗試資源解析。
validation_error	regenerate_data	驗證錯誤通常與請求資料語意或格式有關，應重新生成資料。
server_error 或 rate_limit	retry 或 report	暫時性錯誤可重試；若重複出現則進入報告階段。
environment_error	report	環境錯誤應隔離，不應污染 API 功能缺陷判定。
success	report	測試已取得可用結果，進入報告階段。

需說明的是，rule-based 策略在本研究中承擔兩項功能：其一為提供可解釋之決策邏輯，使工作流程決策過程具備可追蹤性；其二為作為學習型策略（Offline-Q）之

比較基準，以評估資料驅動方法相對於專家規則的增量效益與限制。值得注意的是，rule-based 策略之設計重點在於決策判定的正確性，而非直接提升通過率，是反映在當前測試情境下，規則式決策判定與無決策層之結果在任務層級上高度一致。v4 之主要貢獻在於建立可供 Offline-Q 訓練之工作流程決策事件資料集，並確立決策層的架構基礎供 v5 使用。

3.2.5 Offline-Q Workflow Decision Policy

本研究於工作流程決策模組之上建置離線強化學習（Offline Reinforcement Learning）模組，作為 v5 版本之學習型工作流程決策策略可行性探索。其目的並非提出新的強化學習演算法，亦非預設其已優於規則式策略，而是驗證工作流程決策問題是否可被形式化為離線狀態—動作價值學習問題，並觀察在當前訓練資料規模與單一 API 領域條件下，學習型策略能否習得部分有效之決策模式。此模組所處理的問題是在單一步驟 API 執行完成後，根據當前決策狀態選擇下一個標準工作流動作。Offline-Q 所學習的是「在既有恢復機制存在的前提下，於當前工作流程狀態 s_t 下應優先選擇哪一種標準化行動」。與 rule-based 策略依預定規則進行確定性映射不同，Offline-Q 透過離線轉移資料估計各狀態下不同行動的長期價值，並以 Q 值作為行動偏好的依據。此設計適合 RESTful API 測試情境，因為真實 API 執行具有成本、速率限制、權限限制與環境波動，不適合大量在線探索。其目的並非直接學習 API 操作序列生成，而是學習測試執行失敗後之恢復決策（Recovery Decision）與工作流程控制策略（Workflow Control Policy）。因此，Offline Reinforcement Learning 之作用範圍位於 API 執行完成之後，由 Decision Agent 根據執行結果建立決策狀態，並透過離線策略推論下一步工作流程行動。

圖 3.4 整理本研究於 v5 版本中的決策資料流，由測試生成、API 執行、工作流程轉移資料蒐集到 Offline-Q 決策之整體資料流，整理從輸入規格與人工文件、測試案例與測試程式生成、API 執行、轉移資料集建構，到 Offline-Q Policy 推論與最終報告輸出之流程。這說明了非僅於 API 執行後額外附加一個決策器，而是將前段測試生成、執行結果蒐集與後段決策學習串接為同一條可追蹤之資料流。

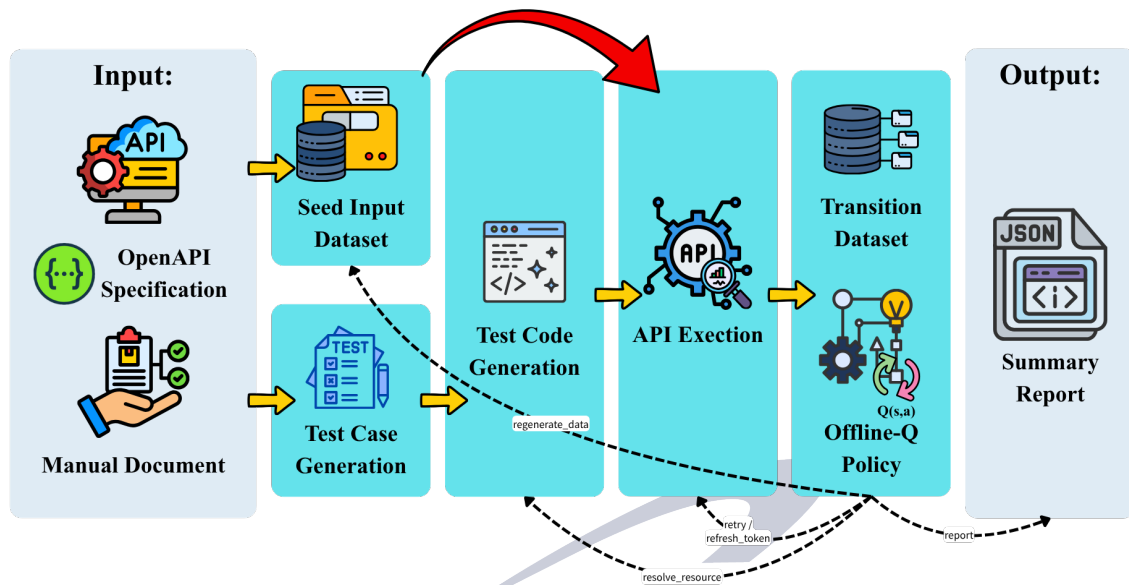


圖 3.4: v5 版本之測試生成、執行與 Offline-Q 決策資料流

如圖 3.4 所示，OpenAPI 規格與人工文件先作為前段輸入，分別支援 seeded input 建構與測試案例生成；其後系統完成測試程式生成並執行 API 請求，蒐集執行結果、狀態轉移與候選恢復行動。這些執行事件進一步被整理為 transition dataset，作為 Offline-Q Policy 之訓練與推論基礎。換言之，工作流程決策並非孤立存在，而是建立在前段測試生成品質、執行狀態觀測與轉移資料累積之上。圖 3.5 更說明了 Decision Agent 與 Offline-Q Policy 之關係，展示了 Decision Agent 內部的決策資料流與策略查表機制。相較於前述代理架構圖著重代理在整體流程中的配置位置，說明狀態、環境、行動與策略更新之對應關係。

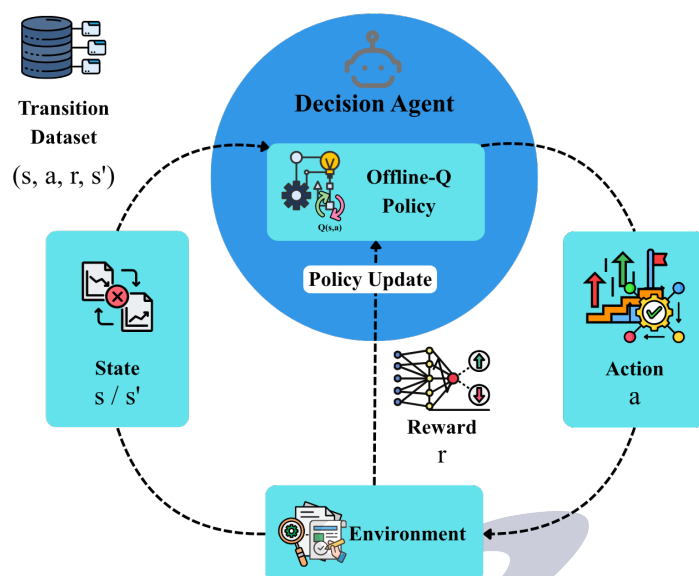


圖 3.5: Decision Agent 與 Offline-Q Policy 之決策資料流

圖 3.5 所示，Decision Agent 位於工作流程決策層，負責接收 API 執行後所形成之決策狀態，並將其轉換為可供 Offline-Q Policy 查表之特徵狀態。Offline-Q Policy 根據當前狀態對各候選行動之 Q 值估計，選擇下一步標準化工作流程行動，再由執行環境產生對應之下一狀態與回饋訊號。訓練階段則透過歷史工作流程轉移資料與獎勵值更新策略，使 Decision Agent 於執行階段能以既有 Q-table 進行推論，而不需於實時 API 環境中進行高成本之即時探索。與傳統線上強化學習需於真實環境中持續探索不同動作不同，本研究採用離線強化學習設計，所有訓練資料皆來自實際測試執行期間所蒐集之工作流程決策事件（Workflow Decision Event），並轉換為狀態－動作－獎勵－下一狀態（State-Action-Reward-Next State）轉移資料集。此設計可避免於真實 RESTful API 服務中進行大量探索所造成之成本、速率限制（Rate Limit）與權限風險，同時保留完整決策軌跡供後續策略訓練與重播分析。Offline Dataset 建構將工作流程執行期間所產生之決策事件轉換為離線強化學習資料集，其主要功能為擷取 Decision Agent 所需要之工作流程狀態、實際執行動作、獎勵值以及下一個狀態，建立可供 Offline-Q 訓練之轉移資料。Decision Agent 輸出之決策事件紀錄，並將產生之資料集提供 Offline-Q Training 模組進行訓練。由於工作流程狀態包含大量結構化資訊，因此本研究於訓練前先進行狀態特徵編碼（State Feature Encoding），將原始狀態轉換為固定特徵索引（Offline Feature Key），降低狀態空間維度並提升離線策略可訓練性。根據工作流程狀

態抽取多項決策相關特徵如下：

系統將每筆工作流程決策紀錄轉換為式 (3.24) 所示之轉移資料集：

$$D = \{(z_t, a_t, r_t, z_{t+1})\} \quad (3.24)$$

其中， z_t 表示目前特徵狀態， a_t 表示實際採取之標準化動作， r_t 表示轉移獎勵， z_{t+1} 表示行動後之下一個特徵狀態。接著，系統依據特徵狀態索引建立 Q 值表 $Q(z, a)$ ，用於估計在特徵狀態 z 下採取動作 a 之預期價值。其更新過程如式 (3.25) 所示：

$$Q(z, a) \leftarrow \text{mean} \left(r + \gamma \max_{a'} Q(z', a') \right) \quad (3.25)$$

- r 為即時獎勵
- γ 為折扣因子
- z' 為下一狀態
- a' 為下一步候選動作

而 γ 為折扣因子，表示後續狀態價值對目前決策仍具有重要影響； a' 為下一狀態下的候選動作。本研究設定訓練迭代次數，以反覆更新狀態-動作價值估計。依目前專案實作，v5 策略之狀態數及動作空間大小為訓練演算法標記為 `tabular_fitted_q_iteration`。

表 3.23: Offline-Q 主要狀態特徵

特徵類型	說明
Status Bucket	HTTP 狀態碼分類
Signal Classification	錯誤訊號分類
Endpoint Group	API 端點群組
Endpoint Depth	API 路徑深度
Retry Count Bucket	重試次數區間
Missing Input Bucket	缺失資源數量
Validation Cluster	驗證錯誤群組
Validation Signature	驗證缺失特徵
Available Resource Bucket	可用資源數量
Request Method	GET、POST、PUT、DELETE 等

特徵編碼完成後將產生唯一 State Key：

$$StateKey = f(z_t) \quad (3.26)$$

此索引作為 Q-table 查詢與更新依據。State Feature Encoding 為 Offline Dataset 與 Offline-Q Training 之共同基礎模組，同時於 Policy 中重複使用，以確保訓練與推論使用一致之特徵空間。Offline-Q Training 會根據歷史轉移資料估計各狀態－動作組合之長期價值，並建立工作流程決策策略。本研究之 Offline-Q 策略採用表格式擬合 Q 值迭代 (Tabular Fitted Q Iteration) 作為訓練方法。為避免符號混淆，訓練與推論階段皆以特徵狀態 z_t 作為 Q-table 之查表單位。

系統定義之 Canonical Action Space 如下：

表 3.24: Offline-Q Action Space

Action	說明
retry	重試原請求
resolve_resource	補齊依賴資源
refresh_token	更新授權資訊
regenerate_data	重新生成測試資料
report	直接進入報告流程

訓練完成之 Q-table 將輸出供 Policy 於執行期間載入使用。在測試執行期間，系統先將當前決策狀態 s_t 轉換為對應之特徵狀態 z_t ，再由 Offline-Q 策略查詢各候選動作之價值並選擇最佳行動。如式 (3.27) 所示：

$$a^* = \arg \max_a Q(z_t, a) \quad (3.27)$$

系統選擇分數最高的行動作為下一步工作流程決策，此機制使系統能根據歷史學習結果，自動判斷是否應重新執行請求、補齊依賴資源、更新授權資訊、重新生成測試資料，或直接結束測試流程並產生報告。本節所提出之離線 Q 值策略，定位為學習型決策策略於工作流程決策問題中之可行性驗證，而非以超越規則式策略為直接目標。此定位源於兩項設計前提：其一，本研究主實驗之核心觀測重點為工作流程恢復能力與流程延續性，而非單純之通過率提升；其二，離線強化學習策略之表現受訓練資料規模、狀態覆蓋與行動分布影響顯著，在當前單一 API 領域與有限資料條件下，策略學習結果需審慎解讀。因此，Offline-Q 策略相對於規則式策略之效益差異，應結合第四章實驗結果與研究限制共同分析，以區分「策略學習本身之可行性」與「在當前資料條件下之實際表現」兩個層次。

3.2.6 Process Reward Modeling 觀點之獎勵設計

傳統強化學習多以最終結果作為獎勵來源（Outcome-based Reward），即根據任務是否成功完成給予正向或負向回饋。然而在 RESTful API 自動化測試情境中，單純依賴最終測試結果往往無法充分反映決策品質，API 測試流程中的重試、資源解析、授權更新與資料重新生成皆屬於中間過程決策，其品質會直接影響後續測試能否繼續推進。例如，當系統收到 HTTP 404 錯誤時，若直接結束測試流程，最終結果可能為失敗；然而若系統能正確識別該錯誤源自於資源尚未建立，並執行資源補齊後重新測試，則可能成功完成後續驗證。此類情況顯示測試結果不僅受到最終狀態影響，更受到中間決策過程品質所影響。為使離線強化學習策略能學習工作流程決策品質，本研究之獎勵設計並非單純以 HTTP 2XX 作為唯一正向獎勵，而是參考近年大型語言模型推理優化中所提出之程序獎勵模型（Process Reward Modeling, PRM）概念 [49]，將獎勵評估重點由單一最終結果延伸至整體決策軌跡（Decision Trajectory），評估每一步工作流程狀態轉移是否改善測試流程品質。在本研究中的決策軌跡表示如式 (3.28) 所示：

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T) \quad (3.28)$$

其中每一次狀態轉移均代表工作流程決策模組對當前執行狀態所做出的處理選擇。與傳統 Outcome Reward 僅評估最終結果不同，本研究將每一步決策視為一個局部品質評估單元（Local Decision Unit），並根據決策後所產生之狀態改善程度計算轉移獎勵（Transition Reward），使系統能學習不同決策對流程品質所造成的影響。令轉移獎勵函數如式 (3.29) 所示：

$$r_t = f(s_t, a_t, s_{t+1}) \quad (3.29)$$

- s_t ：目前狀態
- a_t ：執行動作
- s_{t+1} ：下一狀態

本研究之獎勵設計並非僅根據單次 HTTP 回應是否成功給分，而是將工作流程行動視為一種狀態轉移決策，評估某一行動是否使測試流程朝向較可完成、較可綁定

且較符合人工文件之方向前進。因此，本研究之獎勵屬於狀態轉移獎勵（Transition Reward），其評估對象為 (s_t, a_t, s_{t+1}) ，而非單一時點之執行結果或固定行動偏好。

當決策能改善執行狀態時給予正向獎勵；若造成無效操作、重複失敗或不必要之工作流程擴張，則給予負向獎勵。雖然本研究最終採用表格式擬合 Q 迭代（Tabular Fitted Q Iteration）進行離線策略訓練，但其獎勵訊號設計已具有程序獎勵模型之特性。換言之，本研究並非僅學習最終測試結果是否成功，而是透過累積決策軌跡中每一步狀態轉移所獲得之獎勵，學習何種工作流程決策能有效改善測試執行品質。此設計使離線 Q 學習策略能夠利用歷史執行資料學習較佳之錯誤恢復與流程修正行為。

對於第 t 步工作流程決策，本研究先計算狀態 t 之語意轉移分數，再比較狀態 $t+1$ 相對於狀態 t 之改善程度。其語意轉移分數定義如式 (3.30) 所示：

$$S_{\text{transition}}(t) = 0.35 S_{\text{schema}}(t) + 0.35 S_{\text{binding}}(t) + 0.30 S_{\text{METEOR}}(t) \quad (3.30)$$

其中， $S_{\text{schema}}(t)$ 表示第 t 步請求在結構與欄位層面之正確性分數，用以衡量目前請求是否符合 OpenAPI 規格、必要欄位是否完整、路徑參數是否解析，以及 payload 結構是否有效； $S_{\text{binding}}(t)$ 表示第 t 步跨步驟輸出綁定與依賴解析的正確性分數，用以衡量目前請求所需資源是否已從前序步驟、manual rule、resource pool 或 runtime value pool 取得； $S_{\text{METEOR}}(t)$ 表示第 t 步轉移語意描述實際執行結果與人工文件或 gold path 參考描述的語意接近程度分數，用以衡量目前狀態轉移是否更接近人工文件所定義之預期流程。三者皆正規化於區間 $[0, 1]$ 。在計算語意改善量時，本研究先定義語意轉移差分如式 (3.32) 所示：本研究中， $S_{\text{METEOR}}(t)$ 所使用之參考描述，並非來自抽象文字假設，而是根據人工文件結構化檔案 semantic_restbench_tmdb_motified.csv 之欄位內容組成。對於每一筆測試任務，系統會綜合任務指令、gold path、步驟編號、API 步驟、依賴綁定、成功條件與可接受失敗條件，建立人工文件參考轉移描述。其概念

可形式化表示如式 (3.31) 所示：

$$\begin{aligned} T_t^{\text{ref}} = g(&instruction, \\ &gold_path, \\ &Step, \\ &api_step, \\ &dependency_bindings_content, \\ &success_condition, \\ &acceptable_failure_codes) \end{aligned} \quad (3.31)$$

其中， T_t^{ref} 表示第 t 步之人工文件參考轉移描述；函數 $g(\cdot)$ 表示將人工文件欄位轉換為文字化參考描述之組合函數。表 3.25 整理本研究用於建立 $S_{METEOR}(t)$ 參考描述之主要欄位與範例資料。

表 3.25: 人工文件欄位與 METEOR 參考描述來源範例

欄位名稱	於 METEOR 中的用途	範例欄位資料
task_id	識別任務與對應步驟集合	tmdb_001
instruction	提供任務語意目標	givemethenumberofmoviesdirectedbySofiaCoppola
gold_path	提供預期操作順序	Step1:GET/search/person->Step2: GET/person/{person_id}/movie_credits
Step	指定目前步驟	Step1
api_step	指定目前應執行之 API 操作	GET/search/person
output_bindings_content	定義前一步應抽取之輸出欄位	\$.results[0].idasperson_id
dependency_bindings_content	定義跨步驟依賴名稱	person_id
success_condition	提供成功判定條件	allrequiredstepsexecutedandpassed
oracle_actions_1_register_data	提供驗證錯誤之建議後續處置	[400,422]
oracle_actions_2_resolve_resource	提供資源錯誤之建議後續處置	[404]
acceptable_failure_codes	提供可接受失敗條件	[401,403]

以範例 tmdb_001 第一步為例，人工文件明確指出任務指令為搜尋特定導演，預期 gold path 為先執行 API，再以輸出的 person_id 執行第二步 GET/person/{person_id}/movie_credits。因此，當第 t 步之實際狀態轉移描述與此人工文件參考描述越接近時， $S_{METEOR}(t)$ 越高；反之，若執行結果偏離預期步驟、未正確綁定依賴，或未滿足人工文件所定義之成功條件，則其分數下降。

$$\Delta_{\text{semantic}}(t) = S_{\text{transition}}(t+1) - S_{\text{transition}}(t) \quad (3.32)$$

$\Delta_{\text{semantic}}(t)$ 表示某一工作流程行動執行後，狀態在語意品質上的改善或惡化程度。若行動後之 request 較完整、依賴綁定較正確，或與人工文件之轉移語意對齊程度更高，則 $\Delta_{\text{semantic}}(t)$ 為正值；反之，若行動後之依賴關係仍缺失、請求品質下降，或狀態轉移更偏離預期流程，則 $\Delta_{\text{semantic}}(t)$ 為負值。

除語意改善量外，本研究亦根據執行結果之語意狀態群組（Status Group）給定基礎狀態獎勵。系統不直接以原始 HTTP status code 比較，而是先將執行結果語意化分類為 success、invalid_auth、insufficient_permission、not_found、validation_error、rate_limit、server_error 或 environment_error 等群組，再依據狀態轉移方向給定 R_{status} 。其中，由 non-success 轉為 success 給予較高正向獎勵；由 unknown 或 environment_error 轉為較可診斷之非環境錯誤狀態給予中度正向獎勵；重複採取相同行動且仍停留於相同失敗狀態，則給予負向獎勵；若由 success 退化為 non-success，則給予較大負向獎勵。

綜合基礎狀態獎勵與語意轉移改善量後，第 t 步總獎勵定義如式 (3.33) 所示：

$$R_t = \text{clip}\left(R_{\text{status}} + \Delta_{\text{semantic}}(t), -1, 1\right) \quad (3.33)$$

其中， R_t 表示第 t 步工作流程決策之即時總獎勵； R_{status} 表示由狀態群組轉移方向所決定之基礎狀態獎勵； $\Delta_{\text{semantic}}(t)$ 表示語意轉移品質之差分； $\text{clip}(\cdot, -1, 1)$ 表示將總獎勵限制於區間 $[-1, 1]$ ，避免極端值對離線 Q 值估計造成過度放大影響。

從方法意義而言， R_{status} 負責評估狀態轉移之整體方向是否改善，例如失敗是否轉為成功、未知錯誤是否轉為可診斷錯誤，或是否反而退化為更差之失敗狀態； $\Delta_{\text{semantic}}(t)$ 則負責衡量細部品質是否提升，例如 request 是否更符合 schema、dependency binding 是否更正確，以及狀態描述是否更貼近人工文件所定義之預期流程。因此，本研究之 reward 不只是獎勵「某一步 API 是否成功」，而是獎勵「某一工作流程行動是否讓測試流程朝向可完成、可綁定且可對齊人工文件之方向前進」。

本研究之策略最佳化仍以折扣累積獎勵為目標，其目標函數可表示如式 (3.34) 所示：

$$\max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t R_t \right] \quad (3.34)$$

其中， γ 為折扣因子，用以控制未來獎勵相對於目前決策之重要程度。當 γ 較高時，策略將更重視工作流程長期延續與最終完成性；當 γ 較低時，策略則較偏向重視短期狀態改善。需特別說明者為，本研究目前之 reward 設計尚未正式納入 execution time cost、token cost 或 backend pressure 等成本項，因此其本質仍屬以 workflow correctness、continuity 與 manual-doc alignment 為主之獎勵設計，而非成本敏感式獎勵函數。為說明上述獎勵計算如何反映於實際執行紀錄，本研究以 v5 輸出紀錄為例。該檔案保存每一筆測試案例於 v5 決策模式下之 HTTP 結果、語意判定、最終判定、轉移分數與決策中繼資料。其核心資料結構可抽象表示如式 (3.35) 所示：

$$\begin{aligned} ResultRecord = (&strict_http_pass, semantic_manual_verdict, final_verdict, \\ &manual_doc_match_score, transition_score_t, transition_score_{t+1}, \\ &reward_total, decision_metadata) \end{aligned} \quad (3.35)$$

其中，strict_http_pass 表示是否滿足嚴格 HTTP 成功條件。semantic_manual_verdict 表示根據人工文件語意規則所得之判定，final_verdict 表示最終任務層級判定，manual_doc_match_score 表示當前結果與人工文件描述之對齊程度。另，transition_score_t 與 transition_score_{t+1} 分別表示行動前後之狀態轉移品質，reward_total 表示該次狀態轉移之總獎勵；decision_metadata 則記錄 v5 所選擇之標準化動作、執行期動作與對應特徵狀態。

表 3.27 整理 v5_task_results.json 中一筆輸出紀錄之主要欄位值。

表 3.27: v5 輸出紀錄範例

欄位	範例值
test_case_id	V12_0001
status	passed
strict_http_pass	true
semantic_manual_verdict	strict_pass
final_verdict	pass
final_verdict_reason	ObservedoutcomesatisfiedthestrictHTTPexpectation.
manual_doc_match_score	0.24320551411246344
transition_score_t	0.39773009950248756
transition_score_t1	0.772961654233739
reward_total	1.0
decision_metadata.canonical_report	report
action	
decision_metadata.runtime_action	continue_to_report
decision_metadata.policy_version	v5

若以原始 JSON 片段觀察，其內容可表示如下：

```
{
  "test_case_id": "V12_0001",
  "status": "passed",
  "strict_http_pass": true,
  "semantic_manual_verdict": "strict_pass",
  "final_verdict": "pass",
  "manual_doc_match_score": 0.24320551411246344,
```

```
"transition_score_t": 0.39773009950248756,  
"transition_score_t1": 0.772961654233739,  
"reward_total": 1.0,  
"decision_metadata": {  
  "policy_version": "v5",  
  "canonical_action": "report",  
  "runtime_action": "continue_to_report",  
  "adapter_kind": "offline_q_runtime"  
}  
}
```

此例顯示，v5 不僅輸出最終任務判定，亦同步保存狀態轉移分數、人工文件對齊分數與決策中繼資料，以作為決策軌跡、語意判定與獎勵值的實際範例，使後續 Offline-Q 資料集建構可直接使用 `reward_total` 作為 r_t ，並以 `decision_metadata` 追蹤該次工作流程行動之決策來源。獎勵設計的主要目的在於為工作流程恢復與狀態延續提供可學習之回饋訊號。本研究參考程序獎勵模型（Process Reward Modeling, PRM）之概念，將獎勵評估單位由最終測試結果延伸至每一步狀態轉移，使系統能評估中間決策步驟是否使測試流程朝向可完成、可綁定且符合人工文件之方向前進。本研究借用「逐步評估決策品質」之核心概念，應用於工作流程決策之轉移獎勵計算，因此以「PRM 導向獎勵設計」稱之，以區別於標準 outcome-based 獎勵。換言之，此獎勵設計在本研究中首先服務於工作流程決策問題之形式化——使每一步決策具備可計算之回饋訊號；其次才是支援 Offline-Q 之策略訓練——提供可用於 Q 值估計之轉移獎勵。在當前實驗規模下，獎勵設計之有效性應理解為「提供形式化所需之訊號結構」。

第四章 實驗過程與結果

本章的版本比較為同一套 RESTful API 自動化測試系統逐步加入不同能力模組後之描述性結果，涵蓋規格理解、語意輸入建構、多代理測試實體化、工作流程決策與執行後恢復能力之累積演進。本章重點在於呈現系統能力演進與主要有效機制，而非演算法競賽式比較。v1 至 v2 的評估以 HTTP 狀態碼為主要通過條件，v3 至 v5 則導入 manual rule 進行任務層級判定，評估標準的收緊直接影響絕對通過率，因此不能直接說明系統能力。v4 與 v5 是在相同測試生成基礎上替換工作流程決策機制，兩者共用相同之恢復執行層，差異僅在決策策略之選擇方式。因此，主要在變因控制上的比較，其結果可直接地反映兩種決策策略之有效情境差異。主要觀測指標為 Workflow Recovered Pass，用以衡量工作流程恢復層於執行失敗後之實際恢復能力，此指標為 v5 相對於 v3、v4 之核心差異所在。

4.1 開發環境與系統配置

本研究以 Python 作為主要開發語言，建置一套結合大型語言模型、多代理協作機制、工作流程引擎與離線強化學習決策策略之 RESTful API 自動化測試框架。整體系統採模組化架構設計，各功能模組透過工作流程狀態（Workflow State）進行資料交換，並由工作流程引擎統一管理執行順序。系統實作主要包含 OpenAPI 規格解析、多代理規格討論、測試案例生成、測試程式實體化、API 執行、工作流程決策、離線策略訓練以及測試報告產生等模組。各模組皆以 Python 類別與函式實作，並透過 LangGraph 建立可追蹤之工作流程圖，以支援節點級執行紀錄與狀態管理。在大型語言模型整合方面，本研究採用可替換式模型介面設計，透過統一提供者層（Provider Layer）支援不同模型供應商。多代理協作部分則以 AutoGen 作為代理對話框架，負責規格討論、測試案例審查與測試程式生成等任務。工作流程控制則由 LangGraph 負責節點編排與狀態傳遞。此外，為支援工作流程決策策略之研究，本系統額外建置離線轉移資料集模組，用於記錄工作流程狀態、決策行動、獎勵值與下一狀態，並透過表格式擬合 Q 值迭代（Tabular Fitted Q Iteration）訓練離線工作流程決策策略。

表4.1整理本研究主要開發環境與系統元件。

表 4.1: 系統開發環境與主要元件

項目	內容
程式語言	Python 3.14
工作流程框架	LangGraph
多代理框架	AutoGen
大型語言模型介面	OpenRouter / OpenAI Compatible API
API 規格格式	OpenAPI Specification 3.0.3
測試執行方式	HTTP Runtime Execution、Pytest Materialization
狀態管理	Workflow State
決策策略	Rule-based Policy、Offline-Q Policy
離線學習方法	Tabular Fitted Q Iteration
資料格式	JSON、JSONL、OpenAPI YAML
報告輸出	JSON Report、Execution Trace、Policy Analysis Report

4.2 資料來源與測試目標

本研究之 RESTful API 測試環境主要採用 RESTGPT 研究所公開之測試資料作為基礎來源 [29]。RESTGPT 研究整理多個具備 OpenAPI 規格文件之公開 RESTful API 服務，並針對不同 API 建立對應之任務描述（Task Description）與執行路徑（Gold Path）作為大型語言模型與真實世界 API 操作互動之驗證環境。考量本研究聚焦於 RESTful API 測試流程、跨操作資源依賴以及工作流程決策策略之驗證，而非重新建構大型 API 資料集，因此選擇 RESTGPT 所提供之 TMDb（The Movie Database）資料集作為主要

實驗環境。TMDB 資料集提供完整 OpenAPI 規格文件、公開開發者平台以及涵蓋多種 API 情境，資料集數量充足且涵蓋查詢、建立、修改與刪除等多種操作類型，同時存在跨操作資源依賴關係。RESTGPT 已整理 100 筆 Gold Path 任務案例，可作為測試案例驗證基準，適合作為工作流程導向 API 測試之研究案例。相較於重新建構任務資料集，直接採用 RESTGPT 已驗證之 Gold Path 能降低人工標註成本，並提高不同研究之間的可比較性。因此，本研究以 RESTGPT 之 TMDB 資料集作為主要測試來源，並以其 100 筆 Gold Path 作為測試任務基準。在實際執行環境方面，由於 TMDB API 需要授權驗證，本研究則自行申請 TMDB 開發者帳號並取得 API Access Token，以建立可執行之 Live Execution 環境。所有 API 測試皆透過實際 HTTP 請求與 TMDB 公開服務進行互動，而非使用模擬回應（Mock Response）或離線 Replay 環境，以確保測試流程符合服務提供者之使用規範。本研究將 RESTGPT 提供 tmdb dataset 之 query 與 solution 納為 Gold Path 作為 MCP Context 建構流程，使後續規格討論、多代理推理、測試案例生成與工作流程決策皆能建立於相同知識來源之上。

表4.2整理本研究所使用之 API 資料來源。

表 4.2: 研究使用之 API 資料來源

項目	內容
資料來源	TMDB API
來源研究	RESTGPT [29]
API 類型	Public RESTful API
規格格式	OpenAPI Specification
認證方式	Bearer Token Authentication
測試環境	Live Production API
取得方式	開發者帳號申請與授權取得
用途	OpenAPI 解析、測試案例生成與工作流程決策驗證

本研究之主要測試單位並非 TMDB OpenAPI 規格中的全部 API 端點，而是 RESTGPT 已整理完成之 100 筆 Gold Path 任務案例，後續工作流程生成、測試執行與策略比較皆以任務案例（Task Case）為主要分析單位。

4.3 Benchmark 設計

本研究之實驗評估採用的 Benchmark 設計是任務導向測試基準 (Task-Oriented Benchmark)。若僅以單一 HTTP 請求是否成功作為評估標準，將難以衡量跨操作依賴、授權流程以及工作流程決策策略對整體測試任務之影響。本研究之主要實驗基準採用 RESTGPT 所提供之 TMDb Task Dataset 與 Gold Path 資料，並結合人工規則文件 (Manual Rules) 形成 100 筆任務導向測試案例 (Task-Oriented Test Cases)。每一筆測試案例均包含：Task Description、Gold Path Solution、對應 Manual Rule、預期輸入參數、預期輸出結果、可接受失敗條件 (Acceptable Failure Conditions) 不同於一般 API Benchmark 以單一 HTTP 請求作為評估單位，本研究以完整任務 (Task) 作為分析單位。每一個 Task 可能包含多個 API 操作與跨步驟資源依賴，因此更能反映真實 API 測試情境。於實驗執行過程中，系統根據 Task Description 產生測試案例並執行實際 HTTP 請求，最終結果由 Manual Rule 進行判定，而非僅依賴 HTTP Status Code。此設計可避免將語意錯誤之請求誤判為成功案例。

表4.3整理主實驗 Benchmark 之定義。

表 4.3: 100-Case Manual Document Benchmark 定義

項目	內容
Benchmark 名稱	Manual Document Benchmark
資料來源	RESTGPT TMDb Dataset
測試單位	Task Case
案例數量	100
評估方式	Manual Rule Evaluation
執行環境	Live Execution
主要用途	v1~v5 主實驗比較

為評估不同技術元件對 API 測試流程之影響，本研究建立五個逐步演進版本 (v1~

v5)，並於相同測試環境下執行比較。

表 4.4: v1~v5 版本定義

Version	定義
v1	Random / Monkey Execution Baseline
v2	OpenAPI Specification + Seeded Input Generation
v3	Manual Parsing + Multi-Agent Test Materialization
v4	v3 + Rule-based Workflow Decision Policy
v5	v3 + Offline-Q Workflow Decision Policy

Version 1 : Random Execution Baseline Version 1 為最基礎之 Baseline。系統僅根據 OpenAPI 端點資訊產生隨機測試請求，不使用 Manual Rule、不建立語意輸入，也不進行跨步驟依賴推理。此版本主要用於觀察完全缺乏語意資訊時之基礎通過率。

Version 2 : Specification-Guided Execution Version 2 加入 OpenAPI Specification 解析與 Seeded Input Generation。系統可利用規格資訊產生較合理之測試輸入，降低無效請求比例，但仍未導入多代理協作與工作流程決策機制。

Version 3 : Multi-Agent Test Materialization Version 3 於 Version 2 基礎上加入多代理協作機制。系統透過 Specification Discussion 分析規格限制與依賴關係，並由 Test Author 與 Test Reviewer 共同產生可執行測試程式。此版本代表本研究完整測試生成能力，但尚未具備工作流程恢復能力。

Version 4 : Rule-based Workflow Decision Policy Version 4 於 Version 3 基礎上加入工作流程決策模組。系統根據 HTTP Status Code、Semantic Error Type、Dependency Status 以及 Retry History 等訊號，透過預先定義之規則選擇恢復行動。典型決策包括：

Version 5 : Offline-Q Workflow Decision Policy Version 5 保留 Version 3 之所有測試生成能力，並以 Offline-Q Policy 取代 Rule-based Policy 作為學習型決策選擇器。系統首先利用既有 Workflow Decision Events 建立 Offline RL Dataset，再透過 Tabular Fitted Q Iteration 訓練 Workflow Decision Q-Policy。於執行期間，系統將 WorkflowDecisionState 轉換為 State Feature Key，並查詢 Q Table 取得預期回報最高之恢復行動。Version 5 之目的在於驗證工作流程決策是否可透過離線狀態-動作價值學習加以建模，並探索學習型策略在執行後恢復情境中的可行性與限制。

4.3.1 Evaluation Metrics

為客觀評估不同版本之 RESTful API 自動化測試能力，本研究建立統一之任務導向（Task-Oriented）評估指標。所有版本均使用相同 Task Dataset、相同 Manual Rule、相同 TMDb API 環境以及相同身份驗證設定，以降低外部因素對比較結果之影響。本研究主要使用 Task-Level Success 作為評估基礎，並配合 Workflow Recovery 相關指標進行分析。由於本研究之測試單位為 Task Case，而非單一 HTTP 請求，因此評估結果以任務層級（Task-Level）進行統計。每一個 Task 可能包含多個 API 操作、跨步驟資源依賴以及多次工作流程決策，因此單一 HTTP 狀態碼不足以作為最終判定依據。本研究主要採用 Pass Rate、Strict Pass、Acceptable Failure Pass 以及 Workflow Recovered Pass 作為評估指標。

4.3.1.1 Pass Rate

Pass Rate 用於衡量整體任務成功比例，其定義如下：

$$PassRate = \frac{N_{Pass}}{N_{Total}} \times 100\% \quad (4.1)$$

其中， N_{Pass} 為成功完成之 Task 數量。其中， N_{Total} 為 Benchmark 總 Task 數量。Pass Rate 為本研究主要比較指標，用於評估不同版本整體 API 測試能力之差異。

4.3.1.2 Strict Pass

Strict Pass 代表測試案例完全符合 Gold Path 與 Manual Rule 所定義之預期結果。判定條件包含：HTTP 回應成功、必要欄位存在、回應內容符合 Manual Rule。Strict Pass 代表系統成功完成預期測試目標，且不存在語意偏差。

4.3.1.3 Acceptable Failure Pass

實際 RESTful API 測試環境中，部分 API 操作可能因權限限制、帳號限制或服務端策略而無法成功執行。若系統成功識別此類限制，並依據 Manual Rule 判定其屬於預期可接受失敗（Acceptable Failure），則該案例可被標記為 Acceptable Failure Pass。例如：權限不足（403 Forbidden）、TMDB 帳號限制操作、不允許匿名建立資源、非公開 API 功能。此指標可避免將環境限制誤判為測試失敗。

4.3.1.4 Workflow Recovered Pass

Workflow Recovered Pass 用於衡量工作流程決策策略之恢復能力（Recovery Capability）。若初始測試執行失敗，但系統透過工作流程決策模組採取適當恢復行動後成功完成任務，則該案例將被標記為 Workflow Recovered Pass。例如：Refresh Token 後重新執行成功、Resolve Resource 後重新執行成功、Regenerate Test Data 後成功、Retry 後成功。此指標主要用於評估 Version 4 與 Version 5 決策策略之效果。

其定義如下：

$$RecoveryRate = \frac{N_{Recovered}}{N_{InitialFailure}} \times 100\% \quad (4.2)$$

4.4 實驗流程

本研究之實驗主要分為四個部分，包含基準資料準備、版本比較實驗、工作流程決策資料蒐集以及離線策略訓練與驗證。第一部分為測試資料準備（Dataset Preparation）。本研究採用 RESTGPT 所提供之 TMDB 資料集作為測試環境，包含 OpenAPI 規格文件、100 筆 Gold Path 任務案例、Task Dataset 以及對應之 Manual Rule。所有測試案例皆透過 TMDB 官方 API 進行實際執行（Live Execution），並使用開發者帳

號所取得之授權資訊完成身份驗證。第二部分為版本比較實驗 (Version Benchmarking)。本研究依序執行 v1 至 v5 五種版本，並於相同測試案例集合下比較各版本之測試通過率與執行結果。Version v1 至 v3 主要用於驗證規格解析、輸入生成與多代理測試生成機制之效果；Version v4 則加入 Rule-based Workflow Decision Policy；Version v5 則進一步導入 Offline-Q Workflow Decision Policy。第三部分為工作流程決策事件蒐集 (Workflow Decision Event Collection)。於 v4 版本執行期間，系統將所有工作流程決策過程記錄為 Workflow Decision Event，內容包含當前工作流程狀態 (State)、採取之決策行動 (Action)、後續狀態 (Next State) 以及對應獎勵值 (Reward)。此資料將作為離線強化學習訓練資料集之來源。第四部分為離線策略訓練與驗證 (Offline Policy Training and Evaluation)。系統根據蒐集之 Workflow Decision Event 建構 Offline RL Dataset，並使用 Tabular Fitted Q Iteration 訓練 Offline-Q Policy。完成訓練後，將策略載入 v5 工作流程決策模組進行推論，並於相同測試環境下重新執行 Benchmark，以評估 Offline-Q Policy 於工作流程決策階段之表現。透過上述四個部分，本研究建立由基準測試、決策事件蒐集、離線策略訓練到策略驗證之完整實驗，使工作流程決策策略之訓練與評估皆建立於相同之測試環境與資料來源上。

本研究提出之多代理 RESTful API 自動化測試框架，目標在於建立一套可從 API 規格輸入開始，自動完成規格解析、語意推理、測試案例生成、測試程式實體化、實際 API 執行、工作流程決策以及測試報告產生之完整測試流程。與傳統以 OpenAPI 規格直接產生測試請求之方法不同，本研究將大型語言模型 (Large Language Model, LLM)、多代理系統 (Multi-Agent System, MAS)、模型上下文協定 (Model Context Protocol, MCP)、代理人對代理人協定 (Agent-to-Agent Protocol, A2A) 以及工作流程決策策略整合於同一執行架構中，使系統能在測試過程中保留規格語意、人工文件知識以及執行歷史資訊，並根據實際執行結果選擇後續處理策略。整體系統架構採用工作流程導向 (Workflow-Oriented) 設計，所有模組均透過共享工作流程狀態 (Workflow State) 交換資料，並由工作流程引擎統一管理執行順序與狀態傳遞。

整體執行流程可分為六個主要階段：工作流程控制層 (Workflow Control Layer)、規格解析與知識建構層 (Specification Processing Layer)、多代理協作層 (Multi-Agent Collaboration Layer)、測試生成與執行層 (Test Generation and Execution Layer)、工作流程決策層 (Workflow Decision Layer)、報告與可觀測性層 (Reporting and Observability)

Layer)。其中，工作流程控制層負責協調各模組執行順序；規格解析層負責建立 API 語意知識；多代理協作層負責規格討論與測試內容審查；測試生成與執行層負責產生並執行測試案例；工作流程決策層負責分析執行結果並決定下一步行動；報告層則負責產生測試結果與策略分析報告。

系統啟動後，首先由 MCP Context Bootstrap 模組載入 OpenAPI 規格、任務資料集 (Task Dataset)、Gold Path、Manual Rule 與相關文件內容，建立後續測試所需之上下文資訊。完成初始化後，OpenAPI Parser 解析 API 規格內容，擷取端點 (Endpoint)、請求參數、回應格式、身分驗證需求以及相關結構資訊。規格解析完成後，系統進入 Specification Discussion 階段。此階段透過 AutoGen 多代理協作機制，由不同角色代理共同分析 OpenAPI 規格與人工文件之間的差異，推導測試限制條件、參數依賴關係以及語意約束，並產生統一的規格解讀結果。接著進入 Seeded Input Generation 階段。系統根據規格解析結果、人工規則與既有語意種子資料建立候選輸入集合 (Seeded Input Dataset)，作為後續測試案例生成之基礎。此步驟可降低隨機輸入導致之無效請求比例，提升測試案例之語意合理性。在 Test Case Generation 階段中，系統根據候選輸入資料與規格限制條件產生抽象測試案例 (Abstract Test Cases)，並建立測試案例摘要與參數上下文資訊。之後由 Test Code Materialization 模組將抽象測試案例轉換為可執行之 Python 測試程式碼，並同步產生代理審查紀錄與協作追蹤資訊。完成測試程式產生後，API Execution 模組負責執行實際 HTTP 請求，收集回應狀態碼、回應內容、執行時間與錯誤資訊。執行期間亦同步產生測試遙測資料 (Telemetry)、執行軌跡 (Execution Trace) 與工作流程決策訊號 (Workflow Decision Signal)。當測試執行完成後，系統進入 Workflow Decision 階段。此階段將執行結果轉換為 WorkflowDecisionState，並交由工作流程決策策略進行分析。根據目前研究版本不同，系統可選擇 Rule-based Workflow Policy 或 Offline-Q Workflow Policy 決定下一步行動，例如重新執行請求、重新產生測試資料、重新整理授權資訊、重新建立依賴資源，或直接產生報告。最後，Report Generation 模組整合測試結果、執行紀錄、決策資訊與策略分析結果，輸出 JSON 格式測試報告、版本比較報告、策略分析報告以及工作流程決策紀錄，作為後續實驗分析與離線策略訓練之基礎資料來源。為支援跨模組資訊交換，本研究於系統內部統一採用 Workflow State 作為資料傳遞媒介。各節點不直接存取其他節點之內部資料，而是透過 Workflow State 中所維護之結構化欄位進行資訊交換。此設計可降低模組耦合度，並

提升工作流程之可追蹤性、可維護性與可重播能力。表4.5整理本研究主要模組與其功能。

表 4.5: 系統主要模組與功能

模組	主要功能
WorkflowEngine	管理節點執行順序、狀態傳遞與控制
OpenAPIProcessing	解析 OpenAPI 規格、建立端點資訊與欄位語意
MCPContextBuilder	整合 OpenAPI、Manual Document 與 Gold Path 資訊
Multi-AgentCollaboration	執行規格討論、需求推理與測試內容審查
SeededInputGenerator	建立語意感知測試輸入資料
TestCaseGenerator	產生抽象測試案例與測試策略
TestCodeMaterializer	產生可執行測試程式
RuntimeExecutor	執行實際 API 請求與測試流程
WorkflowDecisionModule	分析執行結果並決定後續工作流程行動
Offline-QModule	訓練與載入離線決策策略
RewardEvaluationModule	計算狀態轉移獎勵與 PRM 分數
ReportGenerator	產生測試報告與策略分析結果

此設計使所有節點皆能透過共享狀態存取前序節點產生之資訊，而不需直接耦合特定模組。整體執行實驗流程由 LangGraph 工作流程圖驅動，主要執行流程如下：

本研究架構並非單一線性流程，而是以 Workflow State 作為共同資料交換中心。

OpenAPI 處理模組、多代理協作模組、測試生成模組與執行模組皆透過共享狀態進行資訊交換。Decision Agent 則負責接收執行結果並觸發工作流程決策策略，而 Offline-Q 模組與 Reward 模組則提供決策分析與策略推論能力。此種設計可降低模組間直接耦合程度，同時保留工作流程執行歷程，使後續測試分析、策略比較與執行回放皆能建立於相同資料結構之上。其中規格資訊流主要由 OpenAPI 規格、Manual Rule 與 Manual Parse 組成；執行資訊流則包含測試案例、測試程式碼、HTTP 回應與執行紀錄；決策資訊流則包含 WorkflowDecisionState、WorkflowDecisionSignal、Reward Breakdown 以及 Offline Reinforcement Learning Transition Record。透過三種資料流的整合，本系統能夠將 API 測試由單次請求驗證擴展為具備決策能力與恢復能力之工作流程測試架構。

4.4.1 Offline-Q 工作流程決策策略設定

表 4.7: Offline-Q 工作流程決策策略設定

項目	設定	說明
Policy Version	offline_q	v5 使用之離線 Q 值策略版本。
Training Algorithm	tabular_fitted_q_iteration	以表格式擬合 Q 值迭代更新狀態-動作價值。
Training Status	trained_tabular_fitted_q	表示策略已完成離線 Q 值訓練。
Discount Factor	0.9	控制未來獎勵對目前決策的影響程度。
Iterations	25	Q 值反覆更新次數。
Action Space Size	5	包含 retry、resolve_resource、refresh_token、regenerate_data 與 report。

4.5 Manual Document Benchmark 主實驗結果

本研究於相同 TMDb Task Dataset、相同 Manual Rule、相同人工文件結構化檔案 semantic_restbench_tmdb_modified.csv 以及相同 Live Execution 環境下，分別執

行 Version 1 至 Version 5 之測試流程，以評估不同技術元件對 RESTful API 自動化測試能力之影響。本研究以 100 筆由 manual document 與 gold path 定義之測試案例作為主要實驗基準。此實驗不同於完整 OpenAPI 端點層級測試，其分母並非所有可解析 API 端點，而是具備明確任務目標、必要輸入、預期結果與可接受失敗條件之 manual-doc-grounded test cases。每一個測試案例皆依據 manual rule 判定最終結果，並分為 strict pass、workflow recovered pass 與 fail 三種類型。表4.9顯示 v1 至 v5 於 100 筆測試案例上的主要結果。v1 採用隨機測試方式，僅通過 25 筆，顯示未考慮規格與語意資訊時，測試請求容易產生無效參數與錯誤路徑。v2 導入 OpenAPI 規格與 seeded input 後，通過率提升至 72%，表示規格感知與語意輸入建構能有效改善請求品質。v3 加入 manual parsing、seeded input 與 multi-agent materialization 後，通過率為 69%。相較於 v2，v3 雖未進一步提升通過率，但其任務層級判定較嚴格，顯示更完整的測試實體化與 manual rule 檢查會重新過濾部分僅於 HTTP 層面成功之案例。v4 加入 rule-based workflow decision policy 後，整體通過率仍維持 69%，顯示本輪主實驗中，規則式決策層尚未對最終任務完成率帶來額外增益。v5 進一步導入 offline-Q runtime decision selector 後，通過率提升至 79%。值得注意的是，v5 之 Strict Pass Rate 仍為 0.69，與 v3 及 v4 相同；其 Final Pass Rate 提升至 0.79，且新增之 10 筆通過皆來自 Workflow Recovered Pass。此結果表示，v5 的主要效益不在於提升前段測試生成能力，而在於執行後根據工作流程狀態恢復原本會失敗之任務。本輪主實驗中，Acceptable Failure Pass 未觀察到通過案例，因此後續結果分析將以 Strict Pass、Final Pass 與 Workflow Recovered Pass 作為主要指標。值得注意的是，本章之 v1 至 v5 結果應理解為系統逐步加入規格感知輸入、多代理測試實體化、工作流程決策與恢復能力後的能力演進。特別是 v4 與 v5 之差異，除了決策選擇器形式不同外，亦涉及恢復行動能否於執行期被有效觸發，因此其結果解讀應以機制分析為主。

表 4.9: Version Benchmark Results

Version	Passed	Failed	Pass Rate	Strict Pass Rate	Final Pass Rate	Acceptable Failure Pass	Workflow Recovered Pass
v1	25	75	0.25	0.25	0.25	0	0
v2	72	28	0.72	0.72	0.72	0	0
v3	69	31	0.69	0.69	0.69	0	0
v4	69	31	0.69	0.69	0.69	0	0
v5	79	21	0.79	0.69	0.79	0	10

為量化不同版本之效益差異，本研究定義版本間通過率差異如式 (4.3) 所示：

$$\Delta_{\text{pass}}(v_i, v_j) = \text{PassRate}(v_i) - \text{PassRate}(v_j) \quad (4.3)$$

另為量化工作流程恢復對 v5 之貢獻，本研究定義恢復增益如式 (4.4) 所示：

$$\text{RecoveryGain}(v5) = \text{FinalPassRate}(v5) - \text{StrictPassRate}(v5) \quad (4.4)$$

依本輪結果可得：

$$\text{RecoveryGain}(v5) = 0.79 - 0.69 = 0.10 \quad (4.5)$$

此結果表示，v5 相較於 v3 與 v4 所增加之 10% 通過率，應優先解讀為執行後工作流程恢復層在特定錯誤型態下發揮作用的結果；其中，學習型決策策略扮演的是在既有恢復行動集合中進行狀態導向選擇的角色，而非單獨產生全部增益來源。

進一步而言，本輪觀察到之恢復效益具有明確錯誤型態集中性。所有 Workflow Recovered Pass 均來自 404 資源依賴缺失，表示目前最穩定被驗證之能力，是系統在資源識別碼失效或跨步驟綁定缺失情境下，仍可透過資源解析與第二次執行完成任務。

對其他錯誤型態之恢復能力，則仍需更多資料與對照分析支持。其可形式化表示如下：

$$\mathcal{X}_{\text{decision}} = \left\{ \begin{array}{l} HTTPStatus, ErrorType, AuthRequired, DependencyStatus, \\ ResourceAvailability, SemanticValidation, RetryCount, TokenState, \\ NeedRegenerateData, NeedRegenerateCode \end{array} \right\} \quad (4.6)$$

公式(4.6)中，各決策變數之意義如下：

- *HTTPStatus* 與 *ErrorType*：用於辨識當前回應型態與錯誤來源。
- *AuthRequired* 與 *TokenState*：用於判斷是否屬於授權相關問題。
- *DependencyStatus* 與 *ResourceAvailability*：用於判斷是否存在可恢復之跨步驟資源缺失。
- *SemanticValidation*：用於識別是否屬於語意驗證錯誤。
- *RetryCount*：用於避免重複採取無效重試。
- *NeedRegenerateData* 與 *NeedRegenerateCode*：分別對應重新產生測試資料與重新產生測試程式之候選恢復方向。

從主實驗結果觀察，工作流程決策的主要有效變數包含 HTTP Status、Error Type、Dependency Status、Resource Availability 與 Retry Count。其中，HTTP Status 與 Error Type 負責區分成功、驗證錯誤、授權錯誤、資源錯誤與環境錯誤；Dependency Status 與 Resource Availability 則決定目前失敗是否可透過補足跨步驟資源而恢復；Retry Count 則用於避免無效重試。相較之下，Token State 與 Semantic Validation 雖仍屬正式決策變數，但在本輪 100 筆主實驗中，並非造成主要恢復增益之核心來源。為量化 Benchmark Stability，本研究進一步定義基準執行穩定度如下：

$$Stability = \frac{N_{\text{completed}}}{N_{\text{scheduled}}} \quad (4.7)$$

其中， $N_{\text{completed}}$ 表示實際完成執行之任務數， $N_{\text{scheduled}}$ 表示原定應執行之任務總數。於本輪主實驗中， $N_{\text{completed}} = 100$ ， $N_{\text{scheduled}} = 100$ ，故 $Stability = 1.0$ ；且

failed_task_runs=0，表示至少在 benchmark 執行穩定性層面，本研究之實驗流程未因外部因素導致任務中止或批次失敗。從效益角度觀察，v1 至 v2 之提升主要反映規格導向輸入建構對請求品質的改善；v2 至 v3 則顯示更嚴格之 manual rule 與 materialization 會重新校正任務層級通過率。v3 至 v4 代表系統開始具備 decision-aware baseline，但其 Final Pass Rate 仍未超出 v3，顯示規則式決策層於本輪資料下尚未形成額外恢復增益。真正與本研究主題直接相關的效益發生於 v5。雖然 v5 之 Strict Pass Rate 仍為 0.69，與 v3 及 v4 相同，但其 Final Pass Rate 提升至 0.79，表示 decision layer 並未改善前段生成品質，而是透過執行後之後續行動選擇改善 workflow continuity。其次，就 Workflow Recovery 而言，v5 新增 10 筆 Workflow Recovered Pass，而 v1 至 v4 均為 0，顯示離線決策策略已在部分情境下提供實質恢復能力。再次，就 Oracle Match 與 Final Pass 而言，v5 之最終任務判定高於其 Strict Pass，表示部分案例雖未於初始執行即成功，但能透過後續恢復步驟轉為最終成功。最後，就 Benchmark Stability 而言，本輪 100 筆任務均完成執行，且無失敗批次，表示此比較結果具備基本完整性。

4.5.1 Workflow Recovery Analysis

Version 5 導入 Offline-Q Workflow Decision Policy 後，共有 10 筆案例於初始執行失敗後成功恢復，最終完成測試任務。表4.11整理所有成功恢復案例。

表 4.11: Workflow Recovery Cases in Version 5

Task ID	Step	Endpoint	Recovery Action	Before	After	Resolved Binding	最終結果
tmdb_006	3	/person/{person_id}/images	資源解析後重跑	404	200	person_id=81 247	恢復通過
tmdb_011	3	/movie/{movie_id}/credits	資源解析後重跑	404	200	movie_id=123 6482	恢復通過
tmdb_020	3	/movie/{movie_id}/images	資源解析後重跑	404	200	movie_id=123 6482	恢復通過
tmdb_023	3	/movie/{movie_id}/release_dates	資源解析後重跑	404	200	movie_id=123 6482	恢復通過
tmdb_025	3	/movie/{movie_id}/similar	資源解析後重跑	404	200	movie_id=123 6482	恢復通過
tmdb_044	3	/person/{person_id}/movie_credits	資源解析後重跑	404	200	person_id=81 247	恢復通過
tmdb_085	3	/person/{person_id}	資源解析後重跑	404	200	person_id=81 247	恢復通過
tmdb_086	3	/person/{person_id}	資源解析後重跑	404	200	person_id=81 247	恢復通過
tmdb_087	3	/company/{company_id}	資源解析後重跑	404	200	company_id=8 1247	恢復通過
tmdb_091	3	/person/{person_id}	資源解析後重跑	404	200	person_id=81 247	恢復通過

為量化 v5 之恢復效益，本研究定義恢復增益如式 (4.8) 所示：

$$RecoveryGain(v5) = FinalPassRate(v5) - StrictPassRate(v5) \quad (4.8)$$

$$RecoveryGain(v5) = 0.79 - 0.69 = 0.10 \quad (4.9)$$

另定義恢復率如式 (4.10) 所示：

$$RecoveryRate(v5) = \frac{N_{Recovered}}{N_{StrictFail}} \quad (4.10)$$

其中， $N_{Recovered} = 10$ ， $N_{StrictFail} = 31$ ，故可得：

$$RecoveryRate(v5) = \frac{10}{31} \approx 0.3226 \quad (4.11)$$

分析結果顯示，所有成功恢復案例皆屬於資源識別碼失效所導致之 404 Not Found 錯誤。當 Offline-Q Policy 判斷當前狀態符合資源缺失特徵時，系統會選擇 `resolve_resource_and_rerun` 作為恢復行動，並透過 Resource Pool 重新取得有效資源識別碼後再次執行測試。此結果表示，本輪主實驗中最具辨識力之決策變數組合為 HTTP Status、Error Type、Dependency Status 與 Resource Availability。換言之，v5 目前最主要學到的是「404 + 資源依賴缺失」情境下之工作流程恢復模式，而非全面性涵蓋所有錯誤類型之恢復策略。

4.5.2 v4 與 v5 之差異分析

因此，v4 與 v5 之差異不宜簡化為 rule-based 與離線強化學習之純策略比較。較精確地說，v4 主要代表 decision-aware verdict baseline；v5 則是在工作流程恢復機制存在時，進一步以學習型決策選擇器決定是否觸發資源解析、更新授權、重新產生資料或直接報告。故本研究目前最直接成立之結論，是工作流程恢復層與狀態導向決策可共同提升流程延續性，而非單獨宣稱 Offline-Q 已全面優於規則式策略。雖然 v4 與 v5 於本研究主實驗中之研究定位皆屬工作流程決策層版本，但兩者所代表的研究意義不同。v4 主要驗證 rule-based workflow decision policy 能否根據 manual rule 與錯誤分類，判斷非 2XX 結果是否仍屬於任務可接受結果。換言之，v4 的功能重點在於 workflow-level verdict，而非將失敗請求重新執行為成功結果。相較之下，v5 導入 offline-Q runtime decision selector，其目標是根據離線轉移資料集所學得之狀態-行動價值，於執行時選擇較合適的工作流程行動。就本輪主實驗結果而言，v4 之 Final Pass Rate 為 0.69，v5 則提升至 0.79；然而，v5 之 Strict Pass Rate 仍為 0.69，與 v4 相同。此結果表示，v5 之額

外提升並非來自前段測試生成品質改善，而是來自執行後工作流程恢復能力。更具體地說，v4 使系統可在語意層級上避免將環境限制或資源缺失直接視為任務失敗；v5 則進一步在部分 404 資源缺失情境中，透過資源池、manual binding pool 與 runtime value pool 執行 second-pass rerun，使原先失敗之請求重新取得成功結果，並轉為 workflow recovered pass。綜合而言，本研究目前最直接成立之結論為：工作流程恢復層與狀態導向決策可共同提升流程延續性。此差異對 RESTful API 測試具有實務意義——在具操作間依賴之 API 任務中，系統從失敗狀態恢復並繼續執行後續步驟，其測試訊號價值高於單純將失敗結果標記為可接受。

4.5.3 Offline-Q 策略學習之深度診斷

本節說明 Offline-Q 策略學習之訓練結果、Q-table 結構診斷，以及策略學習之有效邊界與限制。完成 Offline Dataset 建構後，本研究採用 Tabular Fitted Q Iteration 進行離線策略訓練，折扣因子（Discount Factor）設定為 $\gamma = 0.9$ ，共進行 25 次迭代更新。訓練結果如表 4.13 所示，最終 Q-table 包含 60 個 states 與 62 組 state-action pairs。

表 4.13: Offline-Q 訓練結果摘要

項目	數值
Policy Version	v5_offline_q_contrastive
Training Algorithm	tabular_fitted_q_iteration
Discount Factor (γ)	0.9
Training Iterations	25
Train Record Count	139
Validation Record Count	29
Train State Count	60
Validation State Count	18
Final Q-table States	60
Final State-Action Pairs	62

進一步觀察資料分布可知，目前訓練資料具有明顯之 action 不平衡。訓練集之行動分布為：report=113、resolve_resource=21、regenerate_data=3、refresh_token=1 與 retry=1。訓練集大多來自「成功後進入報告」之記錄，低頻恢復型行動之覆蓋相對有限。

Q-table 結構診斷 對完整 Q-table（圖 4.1）進行結構性分析後，發現 60 個訓練狀態中，58 個狀態僅含單一動作之 Q 值記錄，其餘行動欄位均為空值，代表這些狀態在訓練資料中從未觀察到其他行動被執行。在單一動作狀態下， $\max_{a'} Q(s', a')$ 退化為確定性常數，Fitted-Q 迭代無法引入跨行動之相對價值比較。僅有 2 個狀態具備多動作對比記錄，可作為策略評估之對比狀態（contrastive states）。其中，S059（VALIDATION / status_400 / auth_bootstrap）中 regenerate_data 之 Q 值為 0.60、report 為 0.10，策略正確偏好恢復型行動；S060（VALIDATION / status_400 / rating_write）中 report 之 Q

值為 0.55、retry 為 -0.05 ，為整個 Q-table 唯一出現負值之 state-action pair，顯示策略已學得「對相同失敗狀態直接重試通常無效」之局部判斷。此結構直接解釋了 $\Delta Q^{(k)}$ 在全 25 次迭代均為 0.0 的現象（式 (4.12)）：策略在第一次迭代後即收斂至由 empirical reward 與 logged transition 決定之固定點，現階段主要瓶頸並非迭代次數不足，而是 dataset coverage。

$$\Delta Q^{(k)} = \max_{s,a} |Q^{(k)}(s,a) - Q^{(k-1)}(s,a)| \quad (4.12)$$



Offline-Q Q-Learning Table

Generated from workflow_decision_q_policy.json (60 rows)

state	state_label				retry	resolve	refresh	regen	report	best_action	best_q
S001	api_execution	INSUFFICIENT_PERMISSION	status_401	__none__					0.6000	report	0.6000
S002	api_execution	INSUFFICIENT_PERMISSION	status_401	__none__					0.6000	report	0.6000
S003	api_execution	INSUFFICIENT_PERMISSION	status_401	__none__					0.6000	report	0.6000
S004	api_execution	INSUFFICIENT_PERMISSION	status_401	__none__					0.6000	report	0.6000
S005	api_execution	INVALID_AUTH	status_401	__none__	POST		0.6000			refresh_token	0.6000
S006	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6000				resolve_resource	0.6000
S007	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.5680				resolve_resource	0.5680
S008	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.5963				resolve_resource	0.5963
S009	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6500				resolve_resource	0.6500
S010	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6000				resolve_resource	0.6000
S011	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6000				resolve_resource	0.6000
S012	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6000				resolve_resource	0.6000
S013	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6500				resolve_resource	0.6500
S014	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6500				resolve_resource	0.6500
S015	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6500				resolve_resource	0.6500
S016	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6000				resolve_resource	0.6000
S017	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6000				resolve_resource	0.6000
S018	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6000				resolve_resource	0.6000
S019	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6000				resolve_resource	0.6000
S020	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6000				resolve_resource	0.6000
S021	api_execution	NOT_FOUND_RESOURCE	status_404	__none__		0.6000				resolve_resource	0.6000
S022	api_execution	SUCCESS	status_200	__none__	DELETE				1.0000	report	1.0000
S023	api_execution	SUCCESS	status_200	__none__	DELETE				1.0000	report	1.0000
S024	api_execution	SUCCESS	status_200	__none__	DELETE				1.0000	report	1.0000
S025	api_execution	SUCCESS	status_200	__none__	GET	acc			1.0000	report	1.0000
S026	api_execution	SUCCESS	status_200	__none__	GET	acc			1.0000	report	1.0000
S027	api_execution	SUCCESS	status_200	__none__	GET	acc			1.0000	report	1.0000
S028	api_execution	SUCCESS	status_200	__none__	GET	aut			1.0000	report	1.0000
S029	api_execution	SUCCESS	status_200	__none__	GET	aut			1.0000	report	1.0000
S030	api_execution	SUCCESS	status_200	__none__	GET	cer			1.0000	report	1.0000
S031	api_execution	SUCCESS	status_200	__none__	GET	con			1.0000	report	1.0000
S032	api_execution	SUCCESS	status_200	__none__	GET	con			0.9979	report	0.9979
S033	api_execution	SUCCESS	status_200	__none__	GET	cor			1.0000	report	1.0000
S034	api_execution	SUCCESS	status_200	__none__	GET	cor			1.0000	report	1.0000
S035	api_execution	SUCCESS	status_200	__none__	GET	dis			1.0000	report	1.0000
S036	api_execution	SUCCESS	status_200	__none__	GET	fir			1.0000	report	1.0000
S037	api_execution	SUCCESS	status_200	__none__	GET	ger			1.0000	report	1.0000
S038	api_execution	SUCCESS	status_200	__none__	GET	lis			1.0000	report	1.0000
S039	api_execution	SUCCESS	status_200	__none__	GET	mov			0.9995	report	0.9995
S040	api_execution	SUCCESS	status_200	__none__	GET	mov			0.9974	report	0.9974
S041	api_execution	SUCCESS	status_200	__none__	GET	mov			1.0000	report	1.0000
S042	api_execution	SUCCESS	status_200	__none__	GET	mov			0.9907	report	0.9907
S043	api_execution	SUCCESS	status_200	__none__	GET	mov			1.0000	report	1.0000
S044	api_execution	SUCCESS	status_200	__none__	GET	net			1.0000	report	1.0000
S045	api_execution	SUCCESS	status_200	__none__	GET	net			0.9988	report	0.9988
S046	api_execution	SUCCESS	status_200	__none__	GET	per			1.0000	report	1.0000
S047	api_execution	SUCCESS	status_200	__none__	GET	per			1.0000	report	1.0000
S048	api_execution	SUCCESS	status_200	__none__	GET	per			0.9995	report	0.9995
S049	api_execution	SUCCESS	status_200	__none__	GET	per			0.9946	report	0.9946
S050	api_execution	SUCCESS	status_200	__none__	GET	sea			1.0000	report	1.0000
S051	api_execution	SUCCESS	status_200	__none__	GET	tre			1.0000	report	1.0000
S052	api_execution	SUCCESS	status_200	__none__	GET	tv			1.0000	report	1.0000
S053	api_execution	SUCCESS	status_200	__none__	GET	tv			1.0000	report	1.0000
S054	api_execution	SUCCESS	status_200	__none__	GET	tv			1.0000	report	1.0000
S055	api_execution	SUCCESS	status_200	__none__	GET	tv			1.0000	report	1.0000
S056	api_execution	SUCCESS	status_200	__none__	GET	wat			1.0000	report	1.0000
S057	api_execution	UNKNOWN	status_201	__none__	POST	tv			1.0000	report	1.0000
S058	api_execution	UNKNOWN	status_201	__none__	POST	tv			1.0000	report	1.0000
S059	api_execution	VALIDATION	status_400	auth_bootstrap				0.6000	0.1000	regenerate_data	0.6000
S060	api_execution	VALIDATION	status_400	rating_write	WF	-0.0500		0.5500		regenerate_data	0.5500

圖 4.1: Offline-Q Policy 之 Q-table

Q-value 分布與行動偏好 在Q-value 整體分布方面，本輪 62 個 state-action pairs 之統計結果為：最小值 -0.05、最大值 1.00、平均值 0.8217、中位數 0.9995，其中正值 61 筆、負值 1 筆。Q-value 高度集中於 1.0 附近，顯示多數狀態下策略已穩定偏好「完成當前流程並進入報告」之行動。

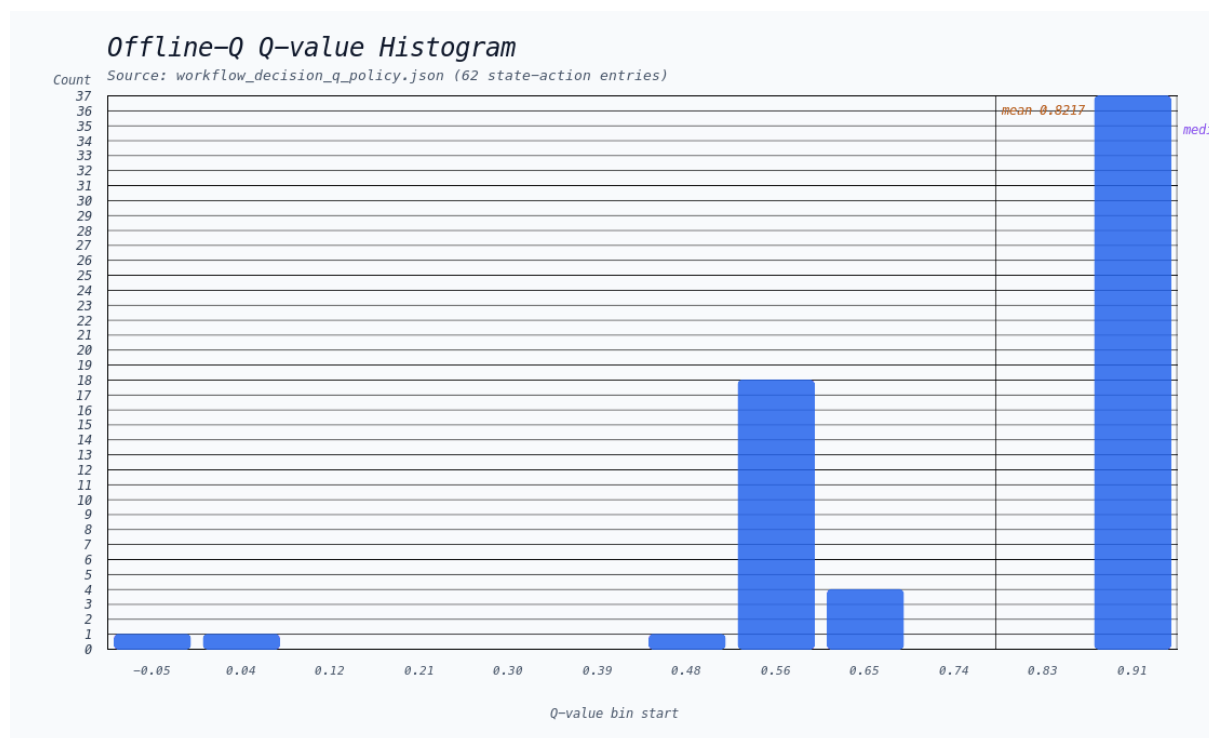


圖 4.2: Offline-Q Policy 之 Q-value 分布

表 4.14: Offline-Q 各決策行動 Q-value 統計

Action	Count	Mean Q	Max Q	Positive	Negative
retry	1	-0.05	-0.05	0	1
resolve_resource	16	0.6103	0.65	16	0
refresh_token	1	0.60	0.60	1	0
regenerate_data	2	0.5750	0.60	2	0
report	42	0.9400	1.00	42	0

表4.14顯示，不同行動之 Q-value 分布具有明顯差異。report 擁有最高平均值與最大值，反映策略已穩定學到「於成功或已可明確歸因之狀態下結束流程」的偏好。於

真正恢復型行動中，resolve_resource 之平均 Q-value 最高，regenerate_data 次之，而 retry 為唯一負值，表示在目前資料下，對相同失敗狀態直接重送相同請求通常無法帶來正向回報。

Offline-Q Per-Action Q-value Summary
Generated from workflow_decision_q_policy.json (5 rows)

action	count	min	q1	median	q3	max	mean	pos	neg
retry	1	-0.0500	-0.0500	-0.0500	-0.0500	-0.0500	-0.0500	0	1
resolve_resource	16	0.5680	0.6000	0.6000	0.6125	0.6500	0.6103	16	0
refresh_token	1	0.6000	0.6000	0.6000	0.6000	0.6000	0.6000	1	0
regenerate_data	2	0.5500	0.5625	0.5750	0.5875	0.6000	0.5750	2	0
report	42	0.1000	0.9995	1.0000	1.0000	1.0000	0.9400	42	0

圖 4.3: Offline-Q 各決策行動之完整 Q-value 統計摘要

若將圖4.3與圖4.4併同觀察，前者提供各行動之分位數摘要，後者則提供分布形狀與離散程度之視覺化證據。兩者共同顯示，目前 Offline-Q 所學得之主要有效模式，集中於「成功後報告」與「404 資源依賴缺失下進行資源解析」兩類狀態。為補充表4.14所呈現之平均值與正負值分布，圖4.3進一步列出各決策行動之最小值、第一四分位數、中位數、第三四分位數、最大值與平均值。由圖可觀察到，report 之第一四分位數已達 0.9995，顯示多數成功後報告狀態之 Q-value 高度集中於 1.0 附近；resolve_resource 則集中於約 0.60 至 0.65 之區間，反映其為目前最穩定的恢復型行動；retry 則仍為唯一負值行動，進一步支持其在現有資料覆蓋下並非有效策略。



圖 4.4: 不同決策行動之 Q-value 分布

圖4.4進一步依決策行動分析 Q-value 分布。結果顯示，`resolve_resource` 具有最高的恢復型 Q-value 分布，且與主實驗中 10 筆 Workflow Recovered Pass 皆來自 404 資源依賴缺失情境之觀察一致。這表示目前 Offline-Q 所學得的主要有效模式，集中於「404 Not Found + Dependency Missing」狀態下優先選擇資源解析與重新執行。

另從最佳 action 分布觀察，每個 state 之最佳 action 分布為：`report`=42、`resolve_resource`=16、`refresh_token`=1 與 `regenerate_data`=2。此結果再次顯示，目前 policy 已明確區分兩類主要情境：其一為成功或已完成狀態，最佳行動為 `report`；其二為可恢復之資源依賴錯誤狀態，最佳行動為 `resolve_resource`。

離線策略評估指標 為評估離線策略與 logged behavior 之相容程度，本研究採用 matched action rate (式 (4.13)) 作為輔助指標。訓練集 MAR 為 $137/139 = 0.9856$ ，驗證集為 $24/29 = 0.8276$ ；train direct-method policy value 為 0.9003，validation 為 0.8149。此結果顯示策略與既有行為高度一致，驗證集覆蓋落差則反映低頻行動在驗證資料中的分布更為稀疏。在當前資料規模與行動分布偏斜條件下，上述數值較適合用於說明

策略與 logged behavior 之相容程度，而不宜單獨視為策略效能之強力證據。

$$MAR = \frac{N_{\text{matched}}}{N_{\text{records}}} \quad (4.13)$$

測試成本與資料蒐集偏誤之結構性分析 當前 Q-table4.1 呈現 58/60 狀態為單一動作記錄之結構，其根本成因是 Live API 測試環境中探索成本高昂所引發的資料蒐集偏誤 (data collection bias)。在真實 API 測試情境中，主動探索低頻恢復行動（如對同一請求反覆嘗試 retry 或 regenerate_data）會引發一系列現實代價：測試資料庫狀態污染、額外的網路延遲與 API rate 有限觸發，以及潛在的服務端異常風險。這些成本迫使資料蒐集過程採取保守的行為策略，僅記錄系統判斷為安全的行動，導致動作支撐集 (action support) 在多數狀態下退化為單一選項。從統計學角度而言，此現象構成選擇性偏誤 (selection bias)：Q-table 所觀察到的轉移資料是來自被現實成本過濾後的保守行為策略。在此條件下，Fitted-Q 函數逼近器失去了在完整動作空間建立比較性決策邊界的能力——模型只觀察到「被選擇執行的動作」之結果，對於「未被執行的替代動作」之代價一無所知。此即離線強化學習文獻中所指的離線可識別性不足 (off-policy identifiability insufficiency) 在本研究情境下的具體表現形式。上述分析係基於 Q-table 結構觀察與理論推論，本研究未設計受控實驗以直接量化探索成本對資料分布的影響，此為方法論層面的限制之一。

綜合本節結果，Offline-Q 目前最清楚學得的模式可概括為兩類：其一是成功或已可明確歸因狀態下優先 report；其二是 404 資源依賴缺失狀態下優先 resolve_resource。因此，本研究對 Offline-Q 較穩健之定位應為：其已在目前 TMDb 單一領域與既有資料覆蓋下，驗證學習型工作流程決策策略之可行性，並對特定恢復情境提供實質幫助，但尚不足以宣稱其已全面超越 rule-based baseline。

第五章 結論與未來展望

5.1 實驗成果總結

本研究針對現有 RESTful API 自動化測試工具在語意理解能力不足、跨操作依賴推理能力有限、測試流程恢復能力缺乏，以及缺少工作流程層級決策機制等問題，提出一套結合多代理協作、工作流程恢復層與學習型決策探索之 RESTful API 自動化測試框架。研究架構整合大型語言模型 (Large Language Model, LLM)、多代理系統 (Multi-Agent System, MAS)、AutoGen、LangGraph、模型上下文協定 (Model Context Protocol, MCP)、代理人對代理人協定 (Agent-to-Agent Protocol, A2A)、離線強化學習 (Offline Reinforcement Learning) 以及流程獎勵建模 (Process Reward Modeling, PRM) 等技術，建立由規格理解、測試案例生成、測試程式產生、實際 API 執行、工作流程決策至結果分析之完整自動化測試流程。研究結果顯示，僅依賴隨機輸入或傳統規格導向方法所產生之測試請求，容易因缺乏語意資訊而導致大量無效請求。透過 OpenAPI 規格解析與 Seeded Input Generation 機制，可有效提升測試輸入品質，降低不合理參數組合所造成之失敗案例，顯示規格導向語意輸入建構對 RESTful API 測試具有實質價值。在測試案例生成階段，本研究導入多代理協作機制，透過品質代理、開發代理與協調代理共同參與規格分析與約束推理，使測試案例生成不再僅依賴單一大型語言模型之推論結果。實驗結果顯示，多代理討論機制能夠協助系統發現規格中的潛在依賴關係與隱含限制條件，進而產生較符合實際業務邏輯之測試案例與請求序列。此結果驗證多代理協作於 API 測試領域中不僅能作為內容生成工具，更能作為語意知識整合與規格推理機制。此外，本研究將工作流程決策問題形式化為馬可夫決策過程 (Markov Decision Process, MDP)，並建立工作流程決策策略模組，使系統能根據 HTTP 狀態碼、錯誤類型、依賴狀態與資源可用性等執行訊號，自動選擇適當之恢復行動。研究結果顯示，工作流程決策與恢復機制的主要效益體現在執行後工作流程延續能力上。於 100 筆主實驗中，v5 之 Strict Pass Rate 與 v3、v4 同為 0.69，但 Final Pass Rate 提升至 0.79，新增之 10 筆通過全部來自 Workflow Recovered Pass，且皆集中於 404 資源依賴缺失情境。此結果表示，本研究目前最直接被驗證之核心貢獻，在於後執行工作流程恢復層能避免測試流程因單一錯誤而提前終止，進一步提高整體測試流程之可完

成度。在離線強化學習部分，本研究透過工作流程轉移事件建立 Offline Reinforcement Learning Dataset，並利用表格式擬合 Q 值迭代 (Tabular Fitted Q Iteration) 訓練 Offline-Q Policy。實驗結果顯示，Offline-Q Policy 已能成功學習部分工作流程決策模式，尤其是在成功後優先進入報告，以及在 404 資源依賴缺失情境下優先選擇資源解析等局部模式。然而，由於訓練資料規模、行動覆蓋率與狀態空間覆蓋度仍有限，其整體表現尚不足以作為全面優於 Rule-based Policy 之證據。儘管如此，本研究已驗證工作流程決策問題可被轉換為離線強化學習問題，並證明 Workflow Decision Policy 具備進一步導入學習型決策策略之可行性。

整體而言，本研究成功建立一套可實際執行之多代理 RESTful API 自動化測試框架，並驗證大型語言模型、多代理協作機制、後執行工作流程恢復層以及工作流程決策形式化於 API 測試自動化領域之整合可行性，為未來智慧化 API 測試系統之發展提供具體參考架構。

5.2 研究貢獻與限制

5.2.1 研究貢獻

本研究之第一項貢獻在於建立後執行工作流程恢復層 (Post-Execution Workflow Recovery Layer)。過去多數 RESTful API 測試研究主要聚焦於單一請求之產生、輸入參數探索或測試覆蓋率提升，而較少關注測試流程在面對實際執行錯誤時之後續恢復行為。本研究將 API 測試視為一個包含多個狀態轉移與決策行動之動態流程，使系統在 404 資源缺失等情境下，仍可透過資源解析與再次執行繼續完成任務。第二項貢獻在於建立結合大型語言模型、多代理系統、MCP 與 A2A 之協作式 RESTful API 測試架構。相較於傳統單一模型生成測試案例之方法，本研究透過多代理協作機制將規格分析、依賴推理、測試審查與決策分析等任務進行職責分離，使系統能以較結構化方式處理複雜規格理解與測試案例生成問題，同時提高測試流程之可追蹤性與可維護性。第三項貢獻在於將工作流程決策問題形式化為馬可夫決策過程。透過將 API 執行後之狀態、候選行動、狀態轉移與獎勵納入統一表示，本研究使工作流程決策不再只是經驗性錯誤處理，而能以可重播、可比較且可學習之序列決策問題加以分析。第四項貢獻在於

驗證學習型工作流程決策策略之可行性。雖然目前 Offline-Q 策略尚未構成全面優於 Rule-based Policy 之證據，但研究結果顯示，工作流程決策問題可被轉換為離線強化學習問題，且學習型策略已能在既有資料覆蓋下學得部分局部有效之恢復模式。第五項貢獻在於提出適用於工作流程決策問題之 Process Reward Modeling 設計。不同於僅評估最終測試結果之獎勵設計方式，本研究將每一次狀態轉移視為評估單位，透過狀態改善程度、錯誤可處理性與語意資訊增益等指標建立轉移獎勵機制，使系統能評估中間決策步驟之品質。此方法提供 Workflow Decision Learning 一種較符合實際測試流程之評估方式。此外，本研究亦得到一項重要觀察：真實 Live Execution 環境下所產生之執行結果具有高度隨機性與環境依賴性，因此較適合作為描述性能力展示與失敗型態觀察來源，而不宜直接作為不同演算法效能優劣之強證據。另一方面，實際執行過程所蒐集之失敗案例集合能夠反映真實系統之錯誤分布，而經整理與標準化後之測試集合則具備可重播與可比較之特性。此發現對未來 RESTful API 測試 Benchmark 之建構具有重要參考價值。

5.2.2 研究限制

為使本研究之限制分析更具系統性，本文將可能影響結果解讀與跨時間重現性之干擾因素整理如下。首先，Live API 本身具有狀態變動特性，不同時間點之回應內容、資源可用性與服務端行為可能發生差異。其次，授權資訊可能因 Token 過期而失效，進而影響部分需驗證身分之操作。再次，外部服務不穩定與 API rate limit 可能造成暫時性失敗，使同一測試案例於不同執行輪次中產生不同結果。此外，前次可用之測試資料或資源識別碼亦可能於後續實驗中被刪除或失效。另一方面，不同版本在測試程式實體化程度上的差異，亦可能影響表面通過率與執行嚴格度。最後，若 OpenAPI 規格與實際服務行為不一致，則可能產生規格漂移問題。上述因素雖不必然改變研究方法本身之有效性，但會影響 Live API 情境下之結果穩定度與跨時間重現性。雖然本研究驗證所提出方法之可行性，但仍存在若干限制。首先，本研究主要採用 TMDB 資料集作為實驗環境。雖然 TMDB 提供完整 OpenAPI 規格與真實服務環境，且具備足夠之端點數量與跨操作依賴關係，但其本質仍屬於媒體內容查詢領域之應用服務。因此，本研究所得結果未必能完全代表金融、醫療、工業控制或企業內部系統等其他領域 API 之行為特性。其次，本研究主要驗證單一 API 領域環境下之工作流程決策能力。不

同 API 服務在認證方式、業務規則、資料結構與錯誤模式上可能存在顯著差異，因此目前尚無法直接證明所提出之 Workflow Decision Policy 具備跨領域泛化能力。未來仍需於更多公開 API 平台與企業系統環境中進一步驗證。第三，本研究所建立之 Offline Reinforcement Learning Dataset 規模仍相對有限。由於資料來源主要來自工作流程執行過程中所蒐集之 Workflow Decision Events，因此狀態空間與行動空間之覆蓋率仍不足以支撐更複雜之強化學習模型訓練。部分狀態與行動組合於訓練資料中出現次數有限，導致離線策略學習結果仍呈現較高程度之稀疏性。此外，本研究觀察到 Rule-based Policy 於工作流程決策問題中已形成相當強勢之基準模型。由於許多 RESTful API 錯誤模式具有明顯且可解釋之對應關係，例如 401 錯誤通常對應授權更新、404 錯誤通常對應資源解析等，因此基於專家知識建立之規則式策略已能達成相當穩定之決策效果。另一方面，本輪主實驗中所有成功恢復案例皆集中於 404 資源依賴缺失情境，表示目前被驗證之恢復能力仍偏向特定錯誤型態，而非全面覆蓋所有恢復型行動。此現象使得 Offline-Q 策略即使成功學習部分決策模式，也未必能於有限資料規模下取得顯著優勢。因此，較穩健之解讀應為：本研究目前已證明學習型策略可於特定恢復情境中提供額外效益，但尚不足以宣稱其已全面超越 Rule-based baseline。最後，由於本研究採用 Live API 執行環境，結果仍可能受授權狀態、資源可用性、外部服務波動與規格漂移影響。即使同一測試案例於不同時間點執行，也可能產生不同回應內容與恢復結果。因此，Live API 情境雖有助於反映真實環境下之測試挑戰，但其結果亦較不易完全重現，未來仍需配合更完整的 Benchmark 與可重播資料集加以補強。

5.3 未來研究方向

未來研究可朝向更具代表性與可重播性之 Benchmark 建構方向發展。研究過程中觀察到，真實 API 執行環境雖然能反映實際系統運作情況，但其結果容易受到服務端更新、外部環境變動與資料狀態改變之影響。因此，可考慮建立 Observed Set 與 Curated Set 雙層資料架構。Observed Set 用於持續蒐集真實環境中所觀察到之失敗案例與執行事件，以反映真實錯誤分布；Curated Set 則透過整理、標準化與分類後形成可重播之 Benchmark 資料集，使不同研究能在一致條件下進行比較與驗證。在強化學習方面，未來可擴大 Workflow Decision Dataset 之規模與多樣性。隨著狀態空間覆蓋率提

高，可逐步由目前之表格式離線學習方法擴展至函數逼近（Function Approximation）或深度強化學習（Deep Reinforcement Learning）架構，以提升策略對高維度狀態空間之處理能力。同時亦可探索離線策略評估（Offline Policy Evaluation）與保守型離線強化學習方法，以降低策略學習過程中之分布偏移問題。

成本感知漸進式資料集擴充 針對當前離線資料集動作覆蓋不足之限制，未來研究可引入成本感知之漸進式資料集擴充機制（incremental dataset expansion），以在控制測試成本的前提下逐步拓寬動作支撐集。其核心思路並非在單次實驗中解鎖完整動作空間，而是設計基於不確定性的探索機制（uncertainty-aware exploration）。在每次自動化測試執行過程中，系統可根據當前 Fitted-Q 模型對各狀態之估計信心，識別 Q 值估計較不穩定之狀態，並於這些關鍵節點選擇性地嘗試非主流行動。新產生之轉移紀錄 (s, a, r, s') 可再納入歷史資料集，供下一輪訓練使用。隨著執行輪次累積，資料集得以漸進式擴大，逐步活化 Q-table 在低頻行動上的估計能力，同時將對服務端環境之衝擊控制在可接受範圍內。此方向本質上屬於 Batch RL 架構下之主動資料蒐集問題（active data collection in offline RL），其實施仍需解決兩個前置問題。其一為不確定性估計方法之選擇。表格式 Q 學習本身不直接提供預測變異數，因此可能需要引入 ensemble 方法或 bootstrap 估計。其二為探索行動之成本上界設定，也就是在何種條件下允許系統嘗試非主流行動，而不致引發服務端副作用。上述問題仍待後續研究結合具體 API 測試場景加以設計與驗證。

此外，本研究目前採用離線強化學習架構進行策略訓練，未來可進一步研究線上強化學習（Online Reinforcement Learning）機制，使系統能於實際執行過程中持續更新決策策略。透過即時回饋與增量學習機制，系統可根據不同 API 環境之變化動態調整恢復策略，提高長期適應能力。另一值得探討之方向為情境式多臂老虎機（Contextual Bandit）模型。由於工作流程決策問題本身具有明顯之單步決策特性，部分恢復行動之效果可於短期內立即觀察，因此 Contextual Bandit 可能比完整強化學習架構更符合部分 Workflow Decision 場景。未來可進一步比較 Contextual Bandit 與 Offline Reinforcement Learning 於工作流程決策問題中之適用性與成本效益。在泛化能力方面，未來研究可將實驗環境擴展至多個不同領域之公開 API 與企業系統，包括電子商務平台、金融服務平台、物聯網管理平台及企業資源規劃系統等。透過跨領域驗證，可進一步評估多代

理推理機制、Workflow Decision Policy 以及 Process Reward Modeling 之通用性與可擴展性。最後，隨著代理系統逐漸朝向標準化與互通化方向發展，未來可進一步研究 A2A 與 MCP 於多代理測試環境中的標準化應用模式。目前代理系統仍多以框架內部協定進行整合，不同平台之代理間缺乏一致之互通機制。若能建立以 A2A 負責代理協作、MCP 負責工具與上下文存取之標準化代理生態系統，將有助於未來形成可重用、可擴充且具互操作性之智慧化測試平台，進一步推動大型語言模型與代理技術於軟體測試領域之實務應用與研究發展。



參考文獻

中文文獻

目前文獻資料尚未設定 keyword=cjk 等可供自動分區之欄位，因此本版先保留中英文分區標題，待後續補齊文獻標記後再啟用中文文獻自動分類。

英文文獻

- [1] Anu Bajaj and Om Prakash Sangwan. “A Systematic Literature Review of Test Case Prioritization Using Genetic Algorithms”. In: *IEEE Access* 7 (2019), pp. 126355–126375. DOI: 10.1109/ACCESS.2019.2938260.
- [2] C. Ebert, D. Bajaj, and M. Weyrich. “Testing Software Systems”. In: *IEEE Software* 39.4 (2022), pp. 8–17. DOI: 10.1109/MS.2022.3166755.
- [3] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. “LlamaRestTest: Effective REST API Testing with Small Language Models”. In: *Proceedings of the ACM on Software Engineering* 2.FSE (FSE 2025). To appear in FSE 2025, FSE022.
- [4] Tao Zheng et al. “RESTLess: Enhancing State-of-the-Art REST API Fuzzing With LLMs in Cloud Service Computing”. In: *IEEE Transactions on Services Computing* 17.6 (2024), pp. 4225–4238. DOI: 10.1109/TSC.2024.3489441.
- [5] Vu Nguyen, Tien N. Nguyen, et al. “KAT: Dependency-Aware Automated API Testing with Large Language Models”. In: *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. Authors listed in [5]. 2024, pp. 82–92. DOI: 10.1109/ICST60714.2024.00017.
- [6] Chunhui Wang et al. “Automatic Generation of Acceptance Test Cases From Use Case Specifications: An NLP-Based Approach”. In: *IEEE Transactions on Software Engineering* 48.2 (2022), pp. 585–616. DOI: 10.1109/TSE.2020.2998503.
- [7] J. C. Alonso et al. “ARTE: Automated Generation of Realistic Test Inputs for Web APIs”. In: *IEEE Transactions on Software Engineering* 49.1 (2023), pp. 348–363. DOI: 10.1109/TSE.2022.3150618.
- [8] A. Arcuri. “Automated Black- and White-Box Testing of RESTful APIs With EvoMaster”. In: *IEEE Software* 38.3 (2021), pp. 72–78. DOI: 10.1109/MS.2020.3013820.

- [9] V. Atlidakis, P. Godefroid, and M. Polishchuk. “RESTler: Stateful REST API Fuzzing”. In: *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 2019, pp. 748–758. DOI: 10.1109/ICSE.2019.00083.
- [10] E. Viglianisi, M. Dallago, and M. Ceccato. “RESTTESTGEN: Automated Black-Box Testing of RESTful APIs”. In: *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. 2020, pp. 142–152. DOI: 10.1109/ICST46399.2020.00024.
- [11] D. Corradini et al. “RestTestGen: An Extensible Framework for Automated Black-box Testing of RESTful APIs”. In: *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2022, pp. 625–629. DOI: 10.1109/ICSME55016.2022.00068.
- [12] J. Wang et al. “Software Testing With Large Language Models: Survey, Landscape, and Vision”. In: *IEEE Transactions on Software Engineering* 50.4 (2024), pp. 911–936. DOI: 10.1109/TSE.2024.3368208.
- [13] U. M. Borghoff, P. Bottoni, and R. Pareschi. “Beyond Prompt Chaining: The TB-CSPN Architecture for Agentic AI”. In: *Future Internet* 17.8 (2025), p. 363. DOI: 10.3390/fi17080363.
- [14] A. Bandi et al. “The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges”. In: *Future Internet* 17.9 (2025), p. 404. DOI: 10.3390/fi17090404.
- [15] X. Hou et al. “Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions”. In: *arXiv preprint arXiv:2503.23278* (2025). URL: <https://arxiv.org/abs/2503.23278>.
- [16] K. Li and Y. Yuan. “Large Language Models as Test Case Generators: Performance Evaluation and Enhancement”. In: *arXiv preprint arXiv:2404.13340* (2024). URL: <https://arxiv.org/abs/2404.13340>.
- [17] A. Ehtesham et al. “A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)”. In: *arXiv preprint arXiv:2505.02279* (2025). Based on

- details in [1], [2], likely a recent survey or preprint. URL: <https://arxiv.org/abs/2505.02279>.
- [18] A. Golmohammadi, M. Zhang, and A. Arcuri. “Testing RESTful APIs: A Survey”. In: *ACM Transactions on Software Engineering and Methodology* 33.1 (2024), 27:1–27:41. DOI: 10.1145/3617175.
- [19] Anjana Sarkar and Soumyendu Sarkar. “Survey of LLM Agent Communication with MCP: A Software Design Pattern Centric Review”. In: (2025).
- [20] Wenbin Gu et al. “Large language model-empowered dynamic scheduling for intelligent hybrid flow shop using multi-agent deep reinforcement learning”. In: *Advanced Engineering Informatics* 71 (2026), p. 104294.
- [21] W. Takerngsaksiri et al. “PyTester: Deep Reinforcement Learning for Text-to-Testcase Generation”. In: *The Journal of Systems and Software* 224 (2025), p. 112381. DOI: 10.1016/j.jss.2025.112381.
- [22] E. T. Barr et al. “The Oracle Problem in Software Testing: A Survey”. In: *Survey or Review Journal* 41.5 (2015). Full author/publication details not available in excerpt [10]., pp. 507–525. DOI: 10.1109/TSE.2014.2372785.
- [23] T. Şimşek, A. Ç. Gülşen, and G. A. Olcay. “The Future of Software Development With GenAI: Evolving Roles of Software Personas”. In: *IEEE Engineering Management Review* 53.4 (2025). DOI confirms journal and dates [8]., pp. 34–45. DOI: 10.1109/EMR.2024.3454112.
- [24] J. Navarro and R. Ibarra. “Automatic test case generation using natural language processing: A systematic mapping study”. In: *Information and Software Technology* 189 (2026), p. 107929. DOI: 10.1016/j.infsof.2025.107929.
- [25] C.-A. Sun et al. “Detecting Inconsistencies in Microservice-Based Systems: An Annotation-Assisted Scenario-Oriented Approach”. In: *IEEE Transactions on Services Computing* 17.5 (2024), pp. 2194–2209. DOI: 10.1109/TSC.2024.3399652.
- [26] 游文瑄. “以知識圖譜結合大型語言模型之檢索增強生成於 RESTful API 自動化測試”. MA thesis. 新北市: 天主教輔仁大學資訊管理學系碩士班, July 2025.

- [27] L. Meunier et al. “Black-Box Optimization Revisited: Improving Algorithm Selection Wizards Through Massive Benchmarking”. In: *IEEE Transactions on Evolutionary Computation* 26.3 (2022), pp. 490–500. DOI: 10.1109/TEVC.2021.3108185.
- [28] Qingyun Wu et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. 2023. arXiv: 2308.08155. URL: <https://arxiv.org/abs/2308.08155>.
- [29] Yifan Song et al. “RestGPT: Connecting Large Language Models with Real-World RESTful APIs”. In: *arXiv preprint arXiv:2306.06624* (2023).
- [30] Ashwin Rao and Tikhon Jelvis. *Foundations of Reinforcement Learning with Applications in Finance*. Chapman & Hall/CRC Machine Learning & Pattern Recognition. CRC Press, 2022. ISBN: 9781032124124.
- [31] Vaibhav Tupe and Shrinath Thube. “Demonstrating Multi-Agent Collaboration via Agent-to-Agent and Model Context Protocols: An IT Incident Response Case Study”. In: *2025 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. Project Demo: <https://youtu.be/2YCs6Axgifs>. 2025, pp. 1–5. DOI: 10.1109/SOSE67019.2025.00013.
- [32] M. Zhang and A. Arcuri. “Open Problems in Fuzzing RESTful APIs: A Comparison of Tools”. In: *ACM Transactions on Software Engineering and Methodology* 33.1 (2024), 27:1–27:41. DOI: 10.1145/3597205.
- [33] S. Karlsson, A. Čaušević, and D. Sundmark. “QuickREST: Property-based Test Generation of OpenAPI-Described RESTful APIs”. In: *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. 2020, pp. 131–141. DOI: 10.1109/ICST46399.2020.00023.
- [34] O. Baniş et al. “Automated Specification-Based Testing of REST APIs”. In: *Sensors* 21.8 (2021), p. 2699. DOI: 10.3390/s21165375.
- [35] Gaith Rjoub et al. “Adaptive agentic meta-controller (AAMC): A deep reinforcement learning framework for intelligent SLM/LLM orchestration”. In: *Neurocomputing* 678 (2026), p. 133192.

- [36] Zhuohui Zhang, Bin Cheng, and Bin He. “EALLMs: Environment-Aligned LLMs for Enhanced Exploration and Communication in Multi-Agent Reinforcement Learning”. In: *IEEE Transactions on Automation Science and Engineering* 23 (2026), p. 1009.
- [37] S. Hosseini and H. Seilani. “The role of agentic AI in shaping a smart future: A systematic review”. In: *Array* 26 (2025), p. 100399. DOI: 10.1016/j.array.2025.100399.
- [38] S. Fatima, T. A. Ghaleb, and L. Briand. “Flakify: A Black-Box, Language Model-Based Predictor for Flaky Tests”. In: *IEEE Transactions on Software Engineering* 49.4 (2023), pp. 1912–1927. DOI: 10.1109/TSE.2022.3201209.
- [39] C. Xu et al. “FlexFL: Flexible and Effective Fault Localization With Open-Source Large Language Models”. In: *IEEE Transactions on Software Engineering* 51.5 (2025), pp. 1455–1471. DOI: 10.1109/TSE.2025.3553363.
- [40] Klea Elmazi, Donald Elmazi, and Jonatan Lerga. “Proactive context aware task offloading in digital twin driven federated IoT systems with large language models”. In: *Internet of Things* 35 (2026), p. 101826.
- [41] Ye Tao et al. “Personalized dynamic lighting via LLM-Empowered multi-Agent reinforcement learning: From user intent to photometric control”. In: *Expert Systems With Applications* 313 (2026), p. 131633.
- [42] Mingxing Li et al. “AP4S: Agentic AI-assisted advanced planning and scheduling with large language models for smart manufacturing”. In: *Journal of Manufacturing Systems* 85 (2026), pp. 207–226.
- [43] I. Radeva et al. “Web Application for Retrieval-Augmented Generation: Implementation and Testing”. In: *Electronics* 13.7 (2024), p. 1361. DOI: 10.3390/electronics13071361.
- [44] D. Brodimas et al. “Intent-Based Infrastructure and Service Orchestration using Agentic-AI”. In: *IEEE Open Journal of the Communications Society* (2025). DOI: 10.1109/OJCOMS.2025.3600706.
- [45] Lei Wang et al. “A Survey on Large Language Model based Autonomous Agents”. In: *Frontiers of Computer Science* 0.0 (2025), pp. 1–42.
- [46] Baoxia Du et al. “TaskON: Task-Oriented Networking for Agentic AI”. In: *IEEE* (2026).

- [47] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. “Adaptive REST API Testing with Reinforcement Learning”. In: *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 326–338. DOI: 10.1109/ASE56229.2023.00218.
- [48] Peng Dai et al. “Constructing Traceability Links between Software Requirements and Source Code Based on Neural Networks”. In: *Mathematics* 11.2 (2023), p. 315. DOI: 10.3390/math11020315.
- [49] Hunter Lightman et al. “Let’s Verify Step by Step”. In: *arXiv preprint arXiv:2305.20050* (2023).
- [50] Alejandro Sánchez Guinea, Grégory Nain, and Yves Le Traon. “A systematic review on the engineering of software for ubiquitous systems”. In: *The Journal of Systems and Software* 118 (2016), pp. 251–276. DOI: 10.1016/j.jss.2016.05.024.
- [51] Roy T. Fielding, Mark Nottingham, and Julian Reschke. *HTTP Semantics*. RFC 9110. Obsoletes RFC 7231, 7230, etc. IETF, June 2022. URL: <https://www.rfc-editor.org/rfc/rfc9110.html>.